Orientation

What we have seen:

- RLMPC, why does it work & different flavors

Orientation

What we have seen:

- RLMPC, why does it work & different flavors

What we will do now:

- The theory is not about MPC, it is about MDPs (MPC is a special case)
- It has broader implications for AI and model-based decision making
- Provide (tentative) practical ideas for in Sim2Real

AI for Operational Decisions
Some perspectives from recent research

Prof. Sebastien Gros

Department of Cybernetic
Faculty of Information Technology
NTNU

Freiburg PhD School

Outline

1. Decisions from data
2. AI models for Decision
3. Differentiating Policy Training
4. Examples



Model-Free Pathway - "Pure" Reinforcement Learning



Actions

State

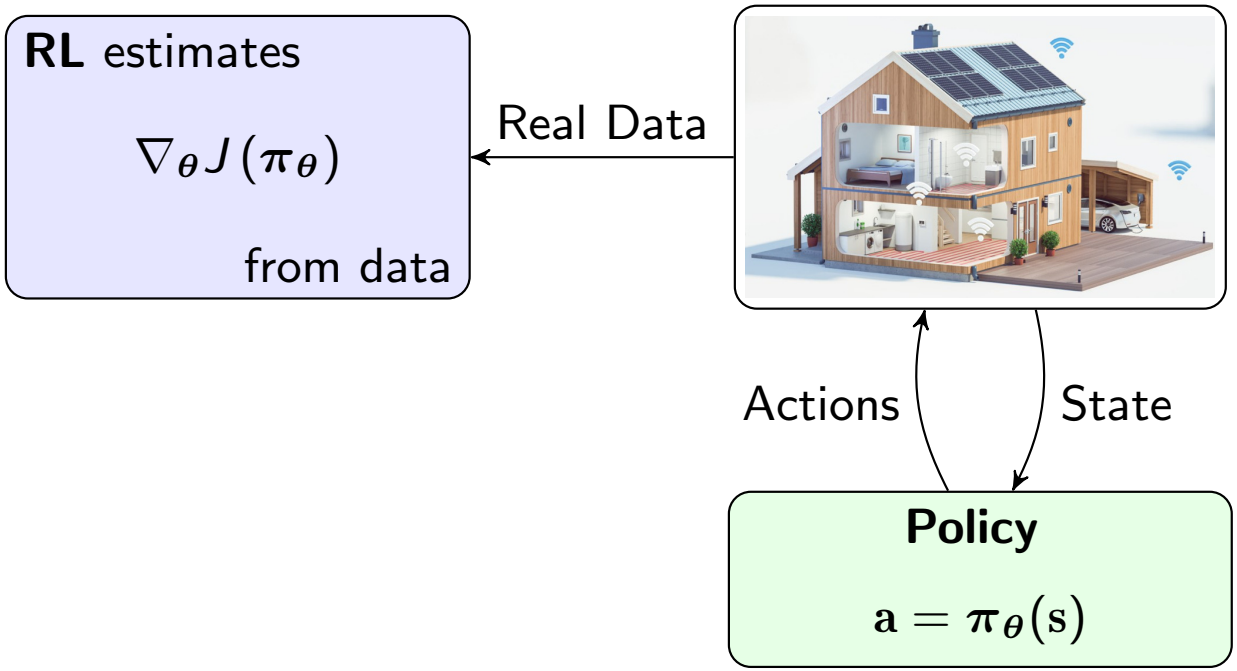
Policy

$$\mathbf{a} = \pi_\theta(\mathbf{s})$$

- Define policy π_θ as a parametrized function
- Optimize policy as:

$$\min_\theta \; J(\pi_\theta)$$

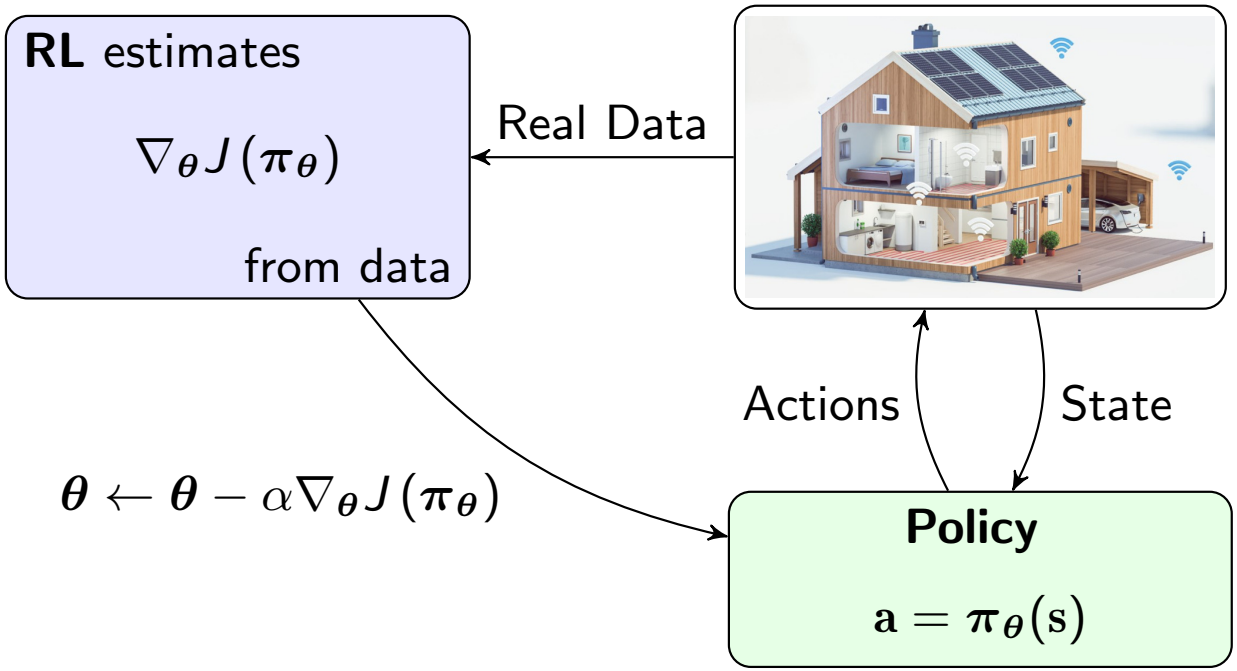
Model-Free Pathway - "Pure" Reinforcement Learning



- Define policy π_θ as a parametrized function
- Optimize policy as:

$$\min_{\theta} \; J(\pi_\theta)$$

Model-Free Pathway - "Pure" Reinforcement Learning

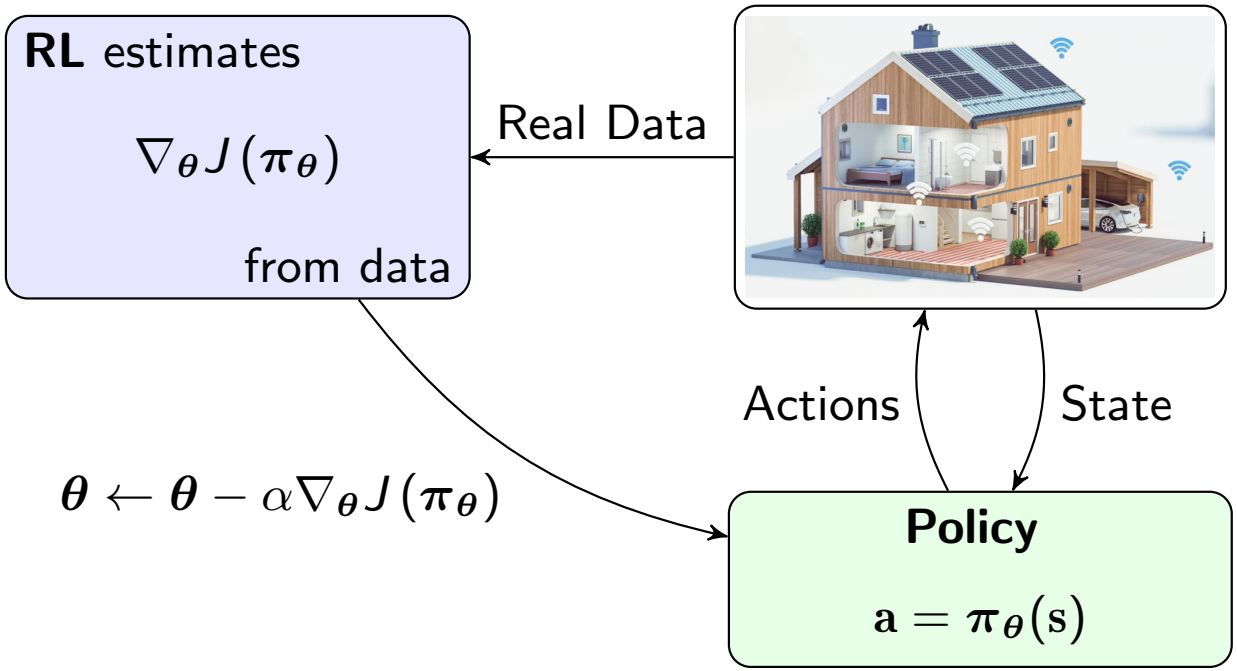


- Define policy π_θ as a parametrized function
- Optimize policy as:

$$\min_\theta \; J(\pi_\theta)$$

- Gradient descent can be used to optimize policy

Model-Free Pathway - "Pure" Reinforcement Learning



- Define policy π_θ as a parametrized function
- Optimize policy as:

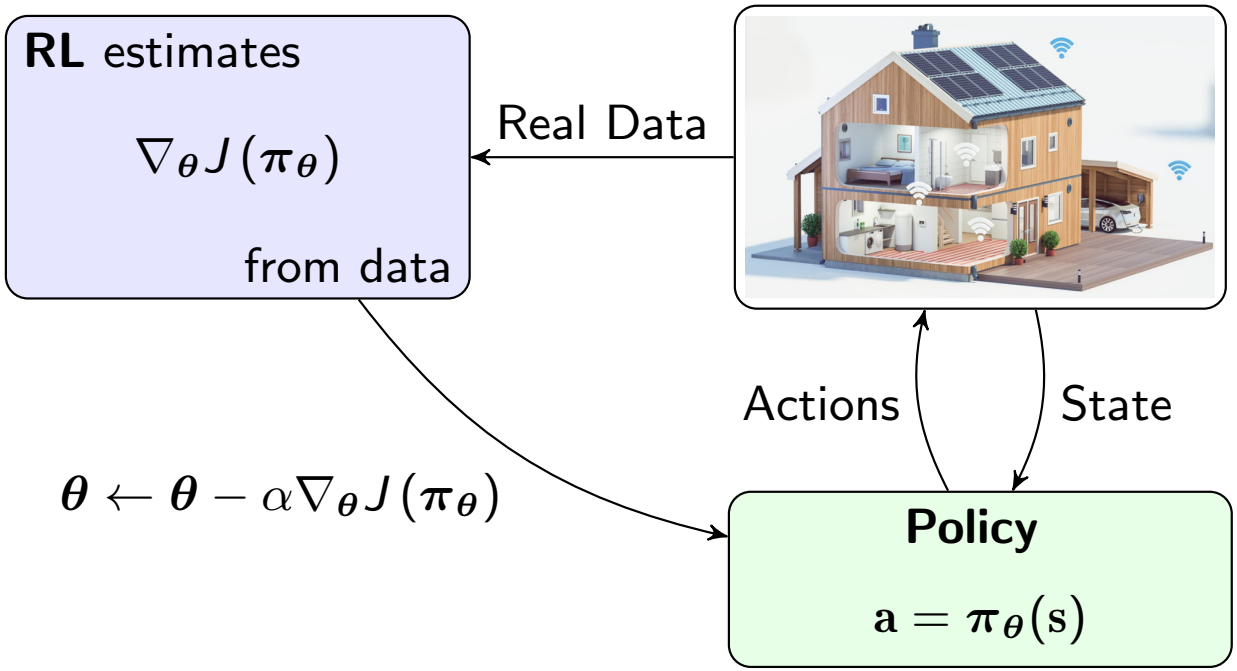
$$\min_\theta \; J(\pi_\theta)$$

- Gradient descent can be used to optimize policy

Remarks

- $\pi_\theta(\mathbf{s})$ often from DNN
- End-to-end training requires a lot of data and effort
- Difficult to provide explainability and guarantees on the policy we obtain
- Poor track record of industrial adoption (exception chatbots)

Model-Free Pathway - "Pure" Reinforcement Learning



- Define policy π_θ as a parametrized function
- Optimize policy as:

$$\min_\theta \; J(\pi_\theta)$$

- Gradient descent can be used to optimize policy

Remarks

- $\pi_\theta(\mathbf{s})$ often from DNN
- End-to-end training requires a lot of data and effort
- Difficult to provide explainability and guarantees on the policy we obtain
- Poor track record of industrial adoption (exception chatbots)

$\rightarrow$policy is often trained on **high-fidelity simulations**: embed knowledge, reduce costs, manage safety, promote explainability, control the training

Model-based Pathway - Predictive AI Models (data-driven, ML, etc)

Real world: $\mathbf{s}_{k+1} \sim \rho(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$

- Data with enough "richness"

Model-based Pathway - Predictive AI Models (data-driven, ML, etc)

One-step model

AI model e.g.

$$\hat{\mathbf{s}}_{k+1} \sim \rho_{\boldsymbol{\theta}}(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$$

fitting data

Real world: $\mathbf{s}_{k+1} \sim \rho(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$

- Data with enough "richness"

Here we no longer need the model $\rho_{\boldsymbol{\theta}}$ to be "optimization-friendly"

Model-based Pathway - Predictive AI Models (data-driven, ML, etc)

One-step model

AI model e.g.

$$\hat{\mathbf{s}}_{k+1} \sim \rho_{\boldsymbol{\theta}}(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$$

fitting data

Real world: $\mathbf{s}_{k+1} \sim \rho(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$

- Data with enough "richness"

Here we no longer need the model $\rho_{\boldsymbol{\theta}}$ to be "optimization-friendly"

Model parameters $\boldsymbol{\theta}$

- Selected such that model $\rho_{\boldsymbol{\theta}}$ "ressembles" reality ρ, using data (+ physics)
- Deterministic models are a special case of $\rho_{\boldsymbol{\theta}}$
- Many methods, from Least Squares and MLE to Bayesian and Adversarial Learning
- Still, in most cases $\rho_{\boldsymbol{\theta}}$ "simplifies" ρ because
  - need for huge amounts of data to decide $\boldsymbol{\theta}$ if model is very rich
  - computationally demanding simulations if $\rho_{\boldsymbol{\theta}}$ is expensive to sample from
- Then
  - biases in case of insufficiently rich model structure
  - validity is limited to some parts of the state-action space
  - only few first moments are correct (typ. mean + variance)

Model-based Pathway - Predictive AI Models (data-driven, ML, etc)

One-step model

AI model e.g.

$$\hat{\mathbf{s}}_{k+1} \sim \rho_{\boldsymbol{\theta}}(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$$

fitting data

Virtual data from "simulating"
$\hat{\mathbf{s}}_{k+1} \sim \rho_{\boldsymbol{\theta}}(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$ forward (Monte Carlo)

Here we no longer need the model $\rho_{\boldsymbol{\theta}}$ to be "optimization-friendly"

Model parameters $\boldsymbol{\theta}$

- Selected such that model $\rho_{\boldsymbol{\theta}}$ "ressembles" reality ρ, using data (+ physics)
- Deterministic models are a special case of $\rho_{\boldsymbol{\theta}}$
- Many methods, from Least Squares and MLE to Bayesian and Adversarial Learning
- Still, in most cases $\rho_{\boldsymbol{\theta}}$ "simplifies" ρ because
  - need for huge amounts of data to decide $\boldsymbol{\theta}$ if model is very rich
  - computationally demanding simulations if $\rho_{\boldsymbol{\theta}}$ is expensive to sample from
- Then
  - biases in case of insufficiently rich model structure
  - validity is limited to some parts of the state-action space
  - only few first moments are correct (typ. mean + variance)

Model-based Pathway - Predictive AI Models (data-driven, ML, etc)

One-step model

AI model e.g.

$$\hat{\mathbf{s}}_{k+1} \sim \rho_{\boldsymbol{\theta}}(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$$

fitting data

Virtual data from "simulating"
$\hat{\mathbf{s}}_{k+1} \sim \rho_{\boldsymbol{\theta}}(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$ forward (Monte Carlo)

In most cases, $\rho_{\boldsymbol{\theta}}$ is a fairly simple representation of reality ρ, easy to sample from

Here we no longer need the model $\rho_{\boldsymbol{\theta}}$ to be "optimization-friendly"

Model parameters $\boldsymbol{\theta}$

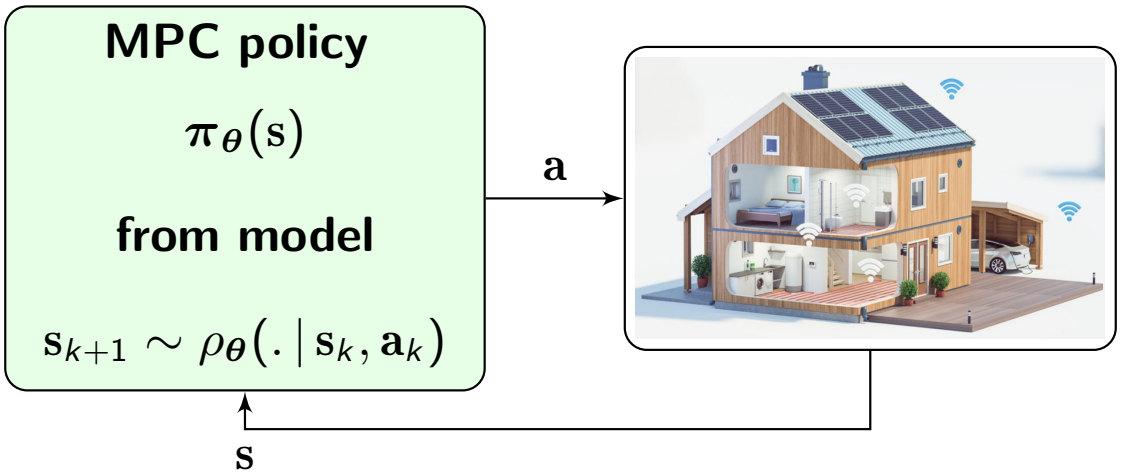
- Selected such that model $\rho_{\boldsymbol{\theta}}$ "ressembles" reality ρ, using data (+ physics)
- Deterministic models are a special case of $\rho_{\boldsymbol{\theta}}$
- Many methods, from Least Squares and MLE to Bayesian and Adversarial Learning
- Still, in most cases $\rho_{\boldsymbol{\theta}}$ "simplifies" ρ because
  - need for huge amounts of data to decide $\boldsymbol{\theta}$ if model is very rich
  - computationally demanding simulations if $\rho_{\boldsymbol{\theta}}$ is expensive to sample from
- Then
  - biases in case of insufficiently rich model structure
  - validity is limited to some parts of the state-action space
  - only few first moments are correct (typ. mean + variance)

Back to MPC - A "Classical" View on Decision Making

Policy $\pi_\theta(\mathrm{s})$ given implicitly by

$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \mathbb{E}\left[T\left(\mathbf{x}_N\right)+\sum_{k=0}^{N-1} L\left(\mathbf{x}_k,\mathbf{u}_k\right)\right] \\ \text{s.t.} \quad & \mathbf{x}_{k+1} \sim \rho_{\boldsymbol{\theta}}\left(\mathbf{x}_k,\mathbf{u}_k\right) \\ & \mathbf{x}_0 = \mathbf{s} \end{aligned}$$

use $\pi_\theta(\mathrm{s}) = \mathbf{u}_0^\star$ on the system

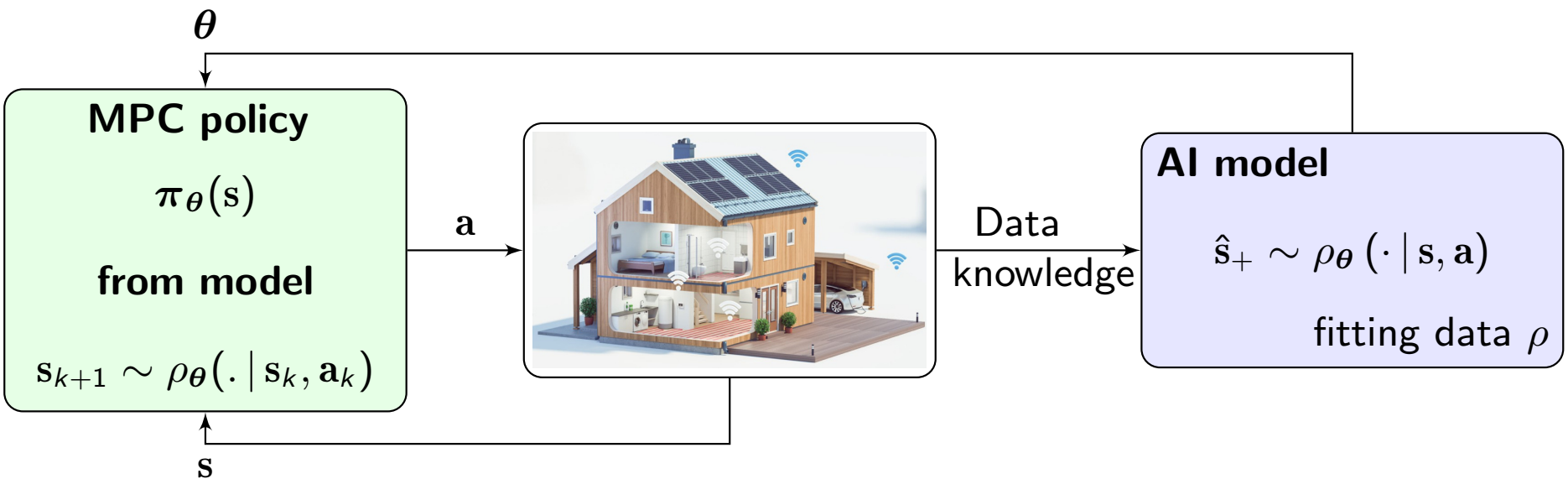


Back to MPC - A "Classical" View on Decision Making

Policy $\pi_\theta(\mathrm{s})$ given implicitly by

$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \mathbb{E}\left[T\left(\mathbf{x}_N\right) + \sum_{k=0}^{N-1} L\left(\mathbf{x}_k, \mathbf{u}_k\right) \right] \\ \text{s.t.} \quad & \mathbf{x}_{k+1} \sim \rho_{\boldsymbol{\theta}}\left(\mathbf{x}_k, \mathbf{u}_k\right) \\ & \mathbf{x}_0 = \mathbf{s} \end{aligned}$$

use $\pi_{\boldsymbol{\theta}}(\mathrm{s}) = \mathbf{u}_0^\star$ on the system



Back to MPC - A "Classical" View on Decision Making

Policy $\pi_\theta(\mathrm{s})$ given implicitly by

$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \mathbb{E}\left[T\left(\mathbf{x}_N\right) + \sum_{k=0}^{N-1} L\left(\mathbf{x}_k, \mathbf{u}_k\right) \right] \\ \text{s.t.} \quad & \mathbf{x}_{k+1} \sim \rho_{\boldsymbol{\theta}}\left(\mathbf{x}_k, \mathbf{u}_k\right) \\ & \mathbf{x}_0 = \mathbf{s} \end{aligned}$$

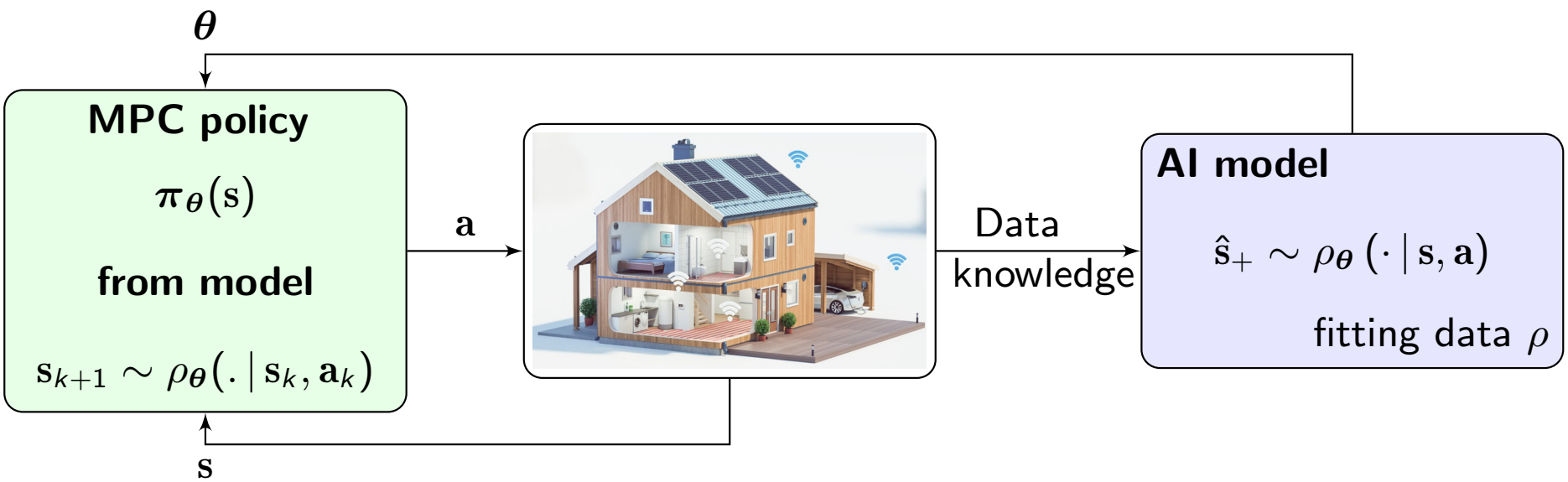
use $\pi_{\boldsymbol{\theta}}(\mathbf{s}) = \mathbf{u}_0^\star$ on the system

Defines a paradigm...

- Performance from model accuracy

  i.e. $\rho_{\boldsymbol{\theta}}$ "close enough" to ρ

- "Ignore" that we replan all the time



Back to MPC - A "Classical" View on Decision Making

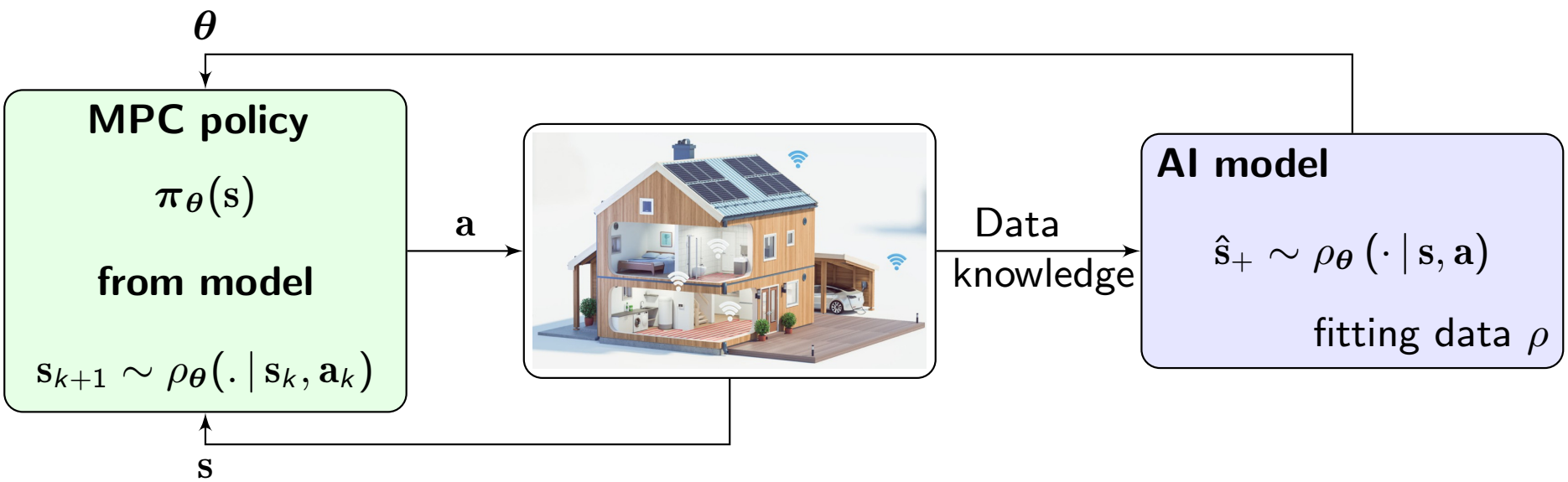
Policy $\pi_\theta(\mathrm{s})$ given implicitly by

$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \mathbb{E}\left[T\left(\mathbf{x}_N\right) + \sum_{k=0}^{N-1} L\left(\mathbf{x}_k, \mathbf{u}_k\right)\right] \\ \text{s.t.} \quad & \mathbf{x}_{k+1} \sim \rho_{\boldsymbol{\theta}}\left(\mathbf{x}_k, \mathbf{u}_k\right) \\ & \mathbf{x}_0 = \mathbf{s} \end{aligned}$$

use $\pi_{\boldsymbol{\theta}}(\mathrm{s}) = \mathbf{u}_0^\star$ on the system

Relationship π_θ to $\pi^\star$??

- They match if $\rho_{\boldsymbol{\theta}}$ is exact and deterministic
- But it usually cannot be...



Back to MPC - A "Classical" View on Decision Making

Policy $\pi_\theta(\mathrm{s})$ given implicitly by

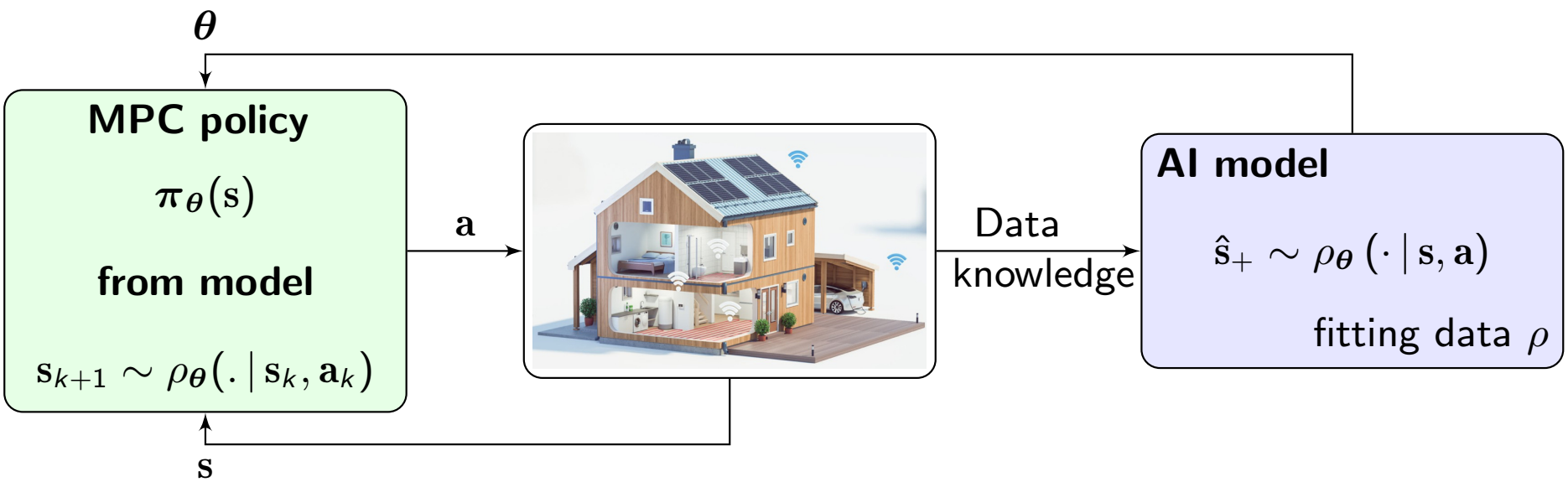
$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \mathbb{E}\left[T\left(\mathbf{x}_N\right) + \sum_{k=0}^{N-1} L\left(\mathbf{x}_k, \mathbf{u}_k\right)\right] \\ \text{s.t.} \quad & \mathbf{x}_{k+1} \sim \rho_\theta\left(\mathbf{x}_k, \mathbf{u}_k\right) \\ & \mathbf{x}_0 = \mathbf{s} \end{aligned}$$

use $\pi_\theta(\mathrm{s}) = \mathbf{u}_0^\star$ on the system

Relationship π_θ to $\pi^\star$??

- They match if ρ_θ is exact and deterministic
- But it usually cannot be...

"Standard methods" for choosing θ in general do not yield the best π_θ.
RL over MPC can fix that, L becomes part of the model



Back to MPC - A "Classical" View on Decision Making

Policy $\pi_\theta(\mathrm{s})$ given implicitly by

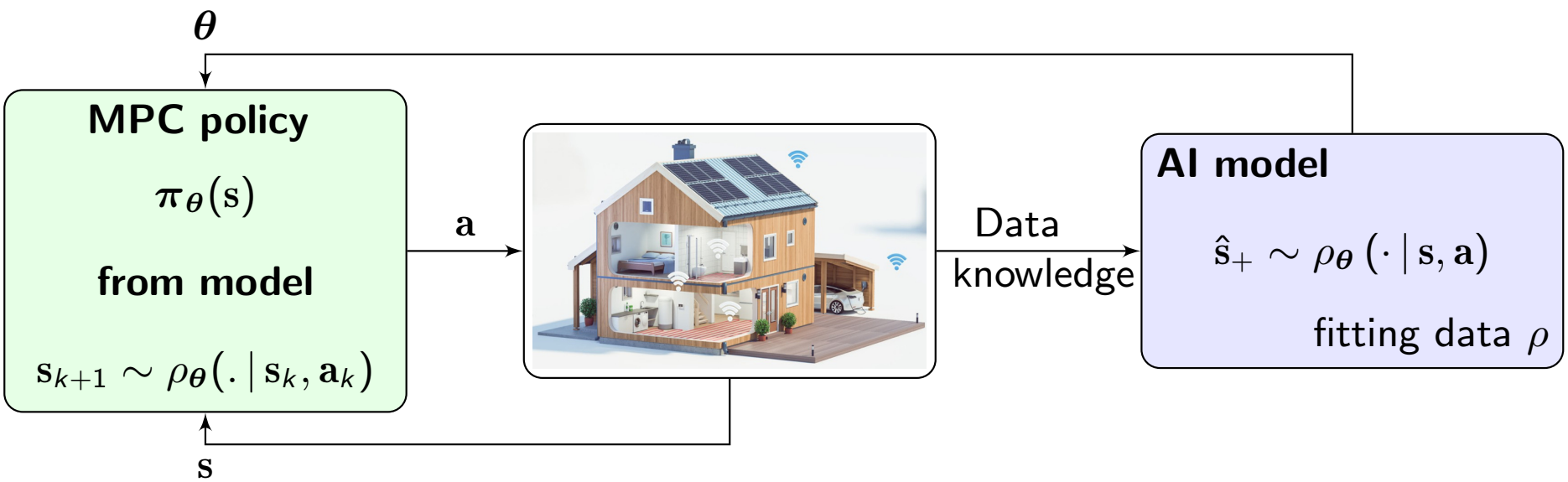
$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \mathbb{E}\left[T\left(\mathbf{x}_N\right)+\sum_{k=0}^{N-1} L\left(\mathbf{x}_k,\mathbf{u}_k\right)\right] \\ \text{s.t.} \quad & \mathbf{x}_{k+1} \sim \rho_\theta\left(\mathbf{x}_k,\mathbf{u}_k\right) \\ & \mathbf{x}_0 = \mathbf{s} \end{aligned}$$

use $\pi_\theta(\mathrm{s}) = \mathbf{u}_0^\star$ on the system

Relationship π_θ to $\pi^\star$??

- They match if ρ_θ is exact and deterministic
- But it usually cannot be...

"Standard methods" for choosing θ in general do not yield the best π_θ. RL over MPC can fix that, L becomes part of the model



This is not the only way of taking decisions from models

Decision policy from AI models (Sim2Real)

AI model

$$\hat{\mathbf{s}}_+ \sim \rho_{\boldsymbol{\theta}}(\cdot \,|\, \mathbf{s}, \mathbf{a})$$

fitting data ρ

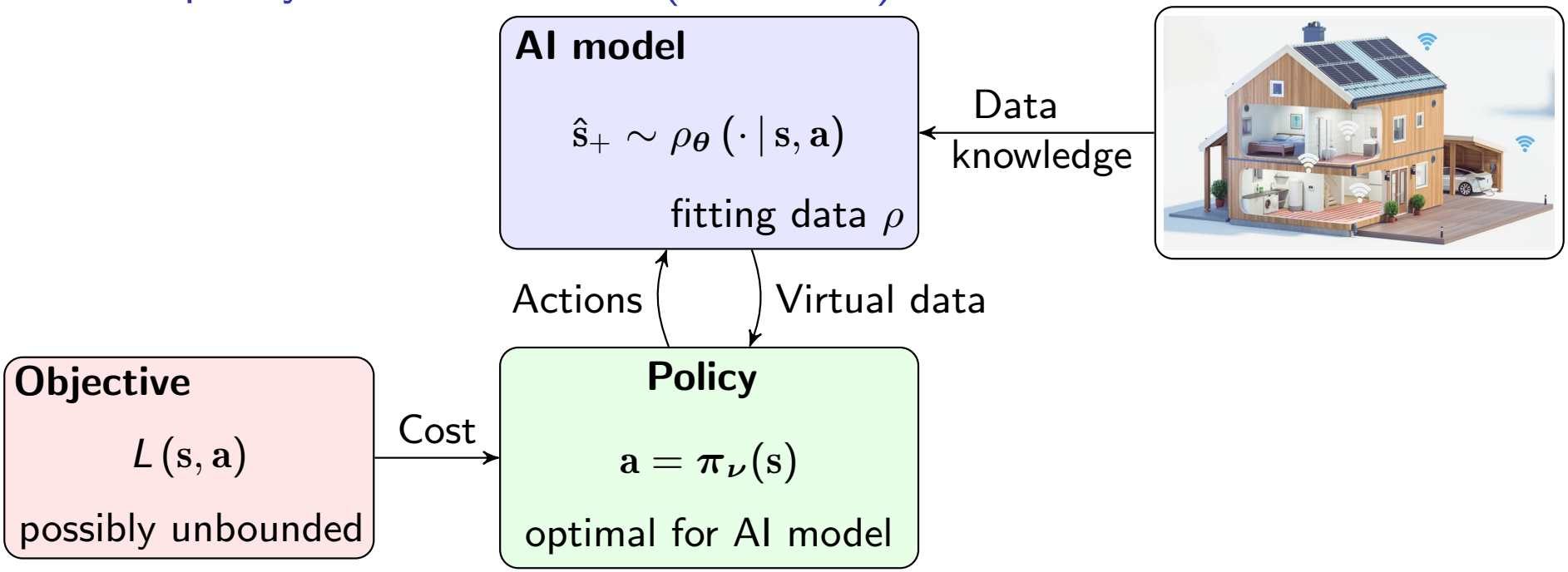
Data knowledge



Classical Process:

- Fit AI model $\rho_{\boldsymbol{\theta}}$ to real data

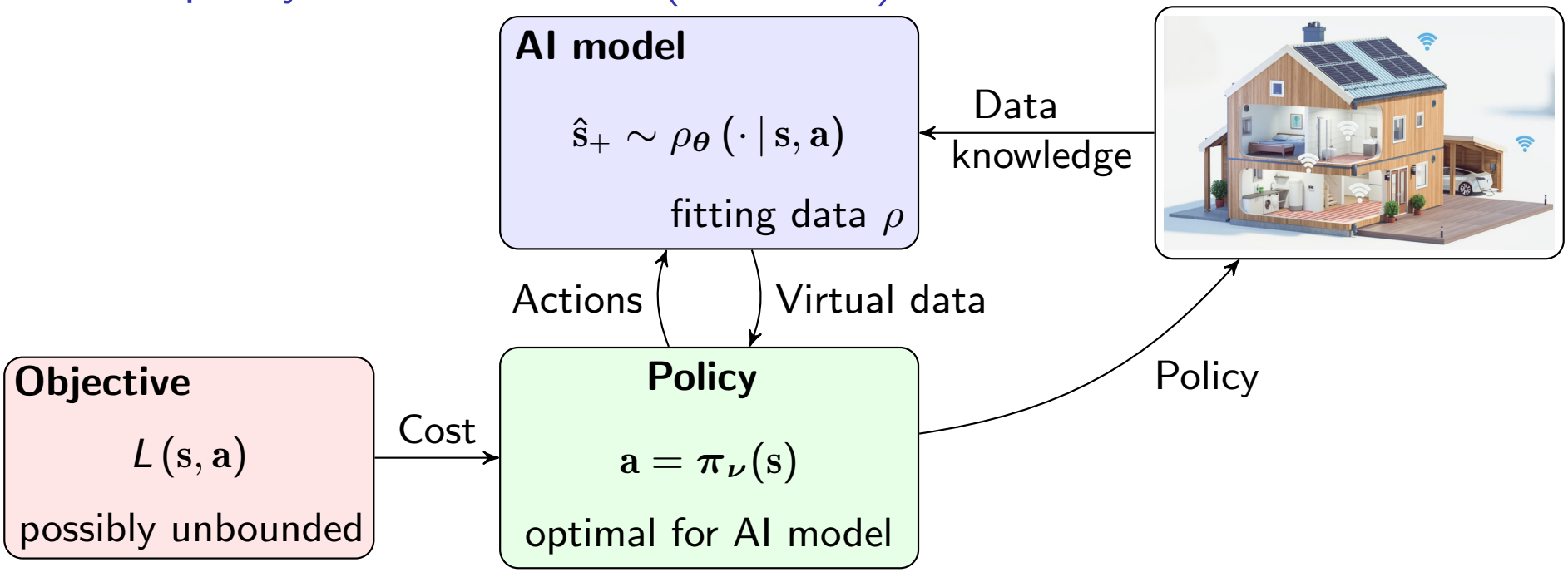
Decision policy from AI models (Sim2Real)



Classical Process:

- Fit AI model $\rho_{\boldsymbol{\theta}}$ to real data
- Develop optimal policy for L and $\rho_{\boldsymbol{\theta}}$ from AI model, e.g. using In-Sim RL
  - Define parametrized policy $\boldsymbol{\pi}_{\boldsymbol{\nu}}$
  - Optimize policy parameters $\boldsymbol{\nu}$ for performance w.r.t. AI model: $\rho_{\boldsymbol{\theta}}, L \rightarrow \boldsymbol{\nu}^\star$

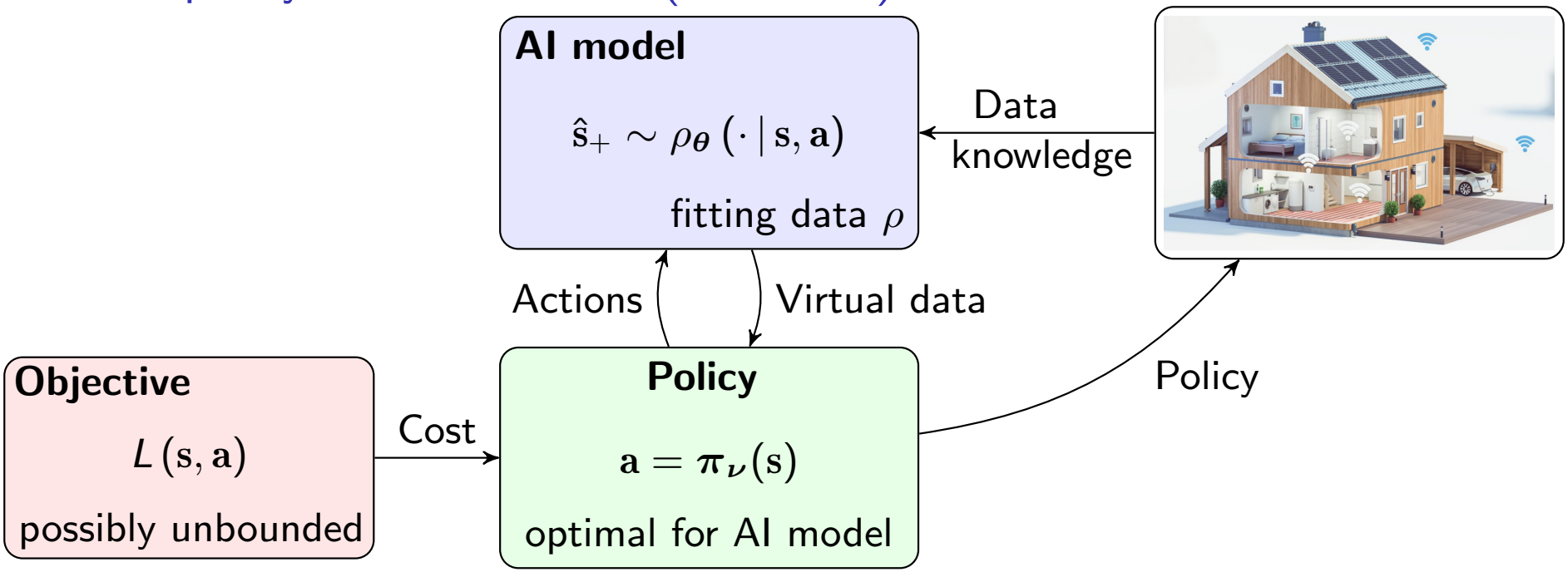
Decision policy from AI models (Sim2Real)



Classical Process:

- Fit AI model $\rho_{\boldsymbol{\theta}}$ to real data
- Develop optimal policy for L and $\rho_{\boldsymbol{\theta}}$ from AI model, e.g. using In-Sim RL
  - Define parametrized policy $\boldsymbol{\pi}_{\boldsymbol{\nu}}$
  - Optimize policy parameters $\boldsymbol{\nu}$ for performance w.r.t. AI model: $\rho_{\boldsymbol{\theta}}, L \rightarrow \boldsymbol{\nu}^\star$
- Transfer policy $\boldsymbol{\pi}_{\boldsymbol{\nu}^\star}$ into the real world

Decision policy from AI models (Sim2Real)

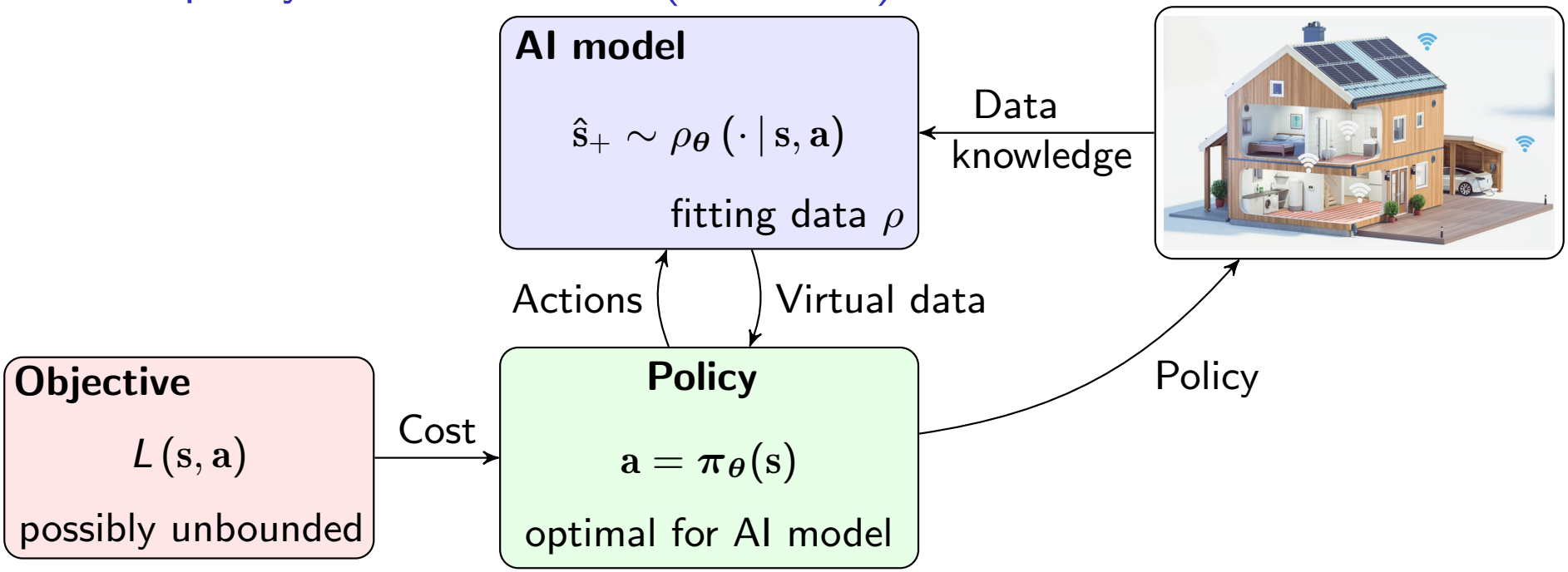


Classical Process:

- Fit AI model $\rho_{\boldsymbol{\theta}}$ to real data
- Develop optimal policy for L and $\rho_{\boldsymbol{\theta}}$ from AI model, e.g. using In-Sim RL
  - Define parametrized policy $\boldsymbol{\pi}_{\boldsymbol{\nu}}$
  - Optimize policy parameters $\boldsymbol{\nu}$ for performance w.r.t. AI model: $\rho_{\boldsymbol{\theta}}, L \rightarrow \boldsymbol{\nu}^\star$
- Transfer policy $\boldsymbol{\pi}_{\boldsymbol{\nu}^\star}$ into the real world

Note: policy parameters $\nu^\star$ optimal for AI model become function of θ
Let's then label $\pi_\theta = \pi_{\nu^\star}$ for simplicity

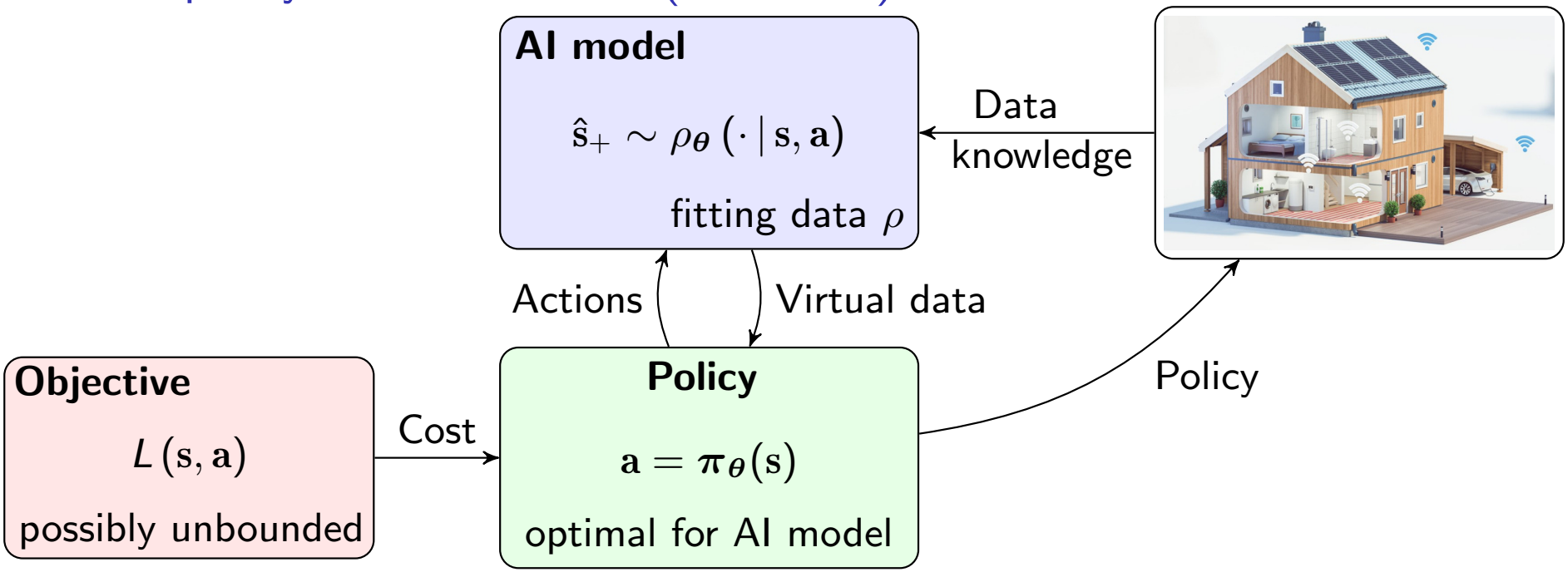
Decision policy from AI models (Sim2Real)



Remarks

- Relationship $\boldsymbol{\pi}_{\boldsymbol{\theta}}$ to $\boldsymbol{\pi}^\star$??
  - They match if $\rho_{\boldsymbol{\theta}} = \rho$, i.e. if model is exact
  - But $\rho_{\boldsymbol{\theta}}$ is (almost always) an approximation

Decision policy from AI models (Sim2Real)



Remarks

- Relationship $\boldsymbol{\pi}_{\boldsymbol{\theta}}$ to $\boldsymbol{\pi}^\star$??
  - They match if $\rho_{\boldsymbol{\theta}} = \rho$, i.e. if model is exact
  - But $\rho_{\boldsymbol{\theta}}$ is (almost always) an approximation

Are "standard methods" for choosing θ resulting in a good policy π_θ??
Empirically the answer is "no". Then how to choose θ to get a good policy?

MPC or Sim2Real - It's the same question...

MPC or Sim2Real - It's the same question...

The theory is about model-based decisions, and equivalences between MDPs

MPC or Sim2Real - It's the same question...

The theory is about model-based decisions, and equivalences between MDPs

World MDP

World MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

MPC or Sim2Real - It's the same question...

The theory is about model-based decisions, and equivalences between MDPs

World MDP

World MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$V^\star(\mathbf{s}) = \min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

$$Q^\star(\mathbf{s}, \mathbf{a}) = L(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \rho}[V^\star(\mathbf{s}_+)|\mathbf{s}, \mathbf{a}]$$

MPC or Sim2Real - It's the same question...

The theory is about model-based decisions, and equivalences between MDPs

World MDP

World MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$V^\star(\mathbf{s}) = \min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

$$Q^\star(\mathbf{s}, \mathbf{a}) = L(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \rho}[V^\star(\mathbf{s}_+)|\mathbf{s}, \mathbf{a}]$$

Optimal policy

$$\pi^\star(\mathbf{s}) = \arg\min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

MPC or Sim2Real - It's the same question...

The theory is about model-based decisions, and equivalences between MDPs

World MDP

World MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$V^\star(\mathbf{s}) = \min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

$$Q^\star(\mathbf{s}, \mathbf{a}) = L(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \rho}[V^\star(\mathbf{s}_+)|\mathbf{s}, \mathbf{a}]$$

Optimal policy

$$\pi^\star(\mathbf{s}) = \arg\min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

Model MDP (simulation environment)

Model MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$\hat{V}^\star(\mathbf{s}) = \min_{\mathbf{a}} \hat{Q}^\star(\mathbf{s}, \mathbf{a})$$

$$\hat{Q}^\star(\mathbf{s}, \mathbf{a}) = \hat{L}(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \hat{\rho}}[V^\star(\hat{\mathbf{s}}_+)|\mathbf{s}, \mathbf{a}]$$

Optimal policy

$$\hat{\pi}^\star(\mathbf{s}) = \arg\min_{\mathbf{a}} \hat{Q}^\star(\mathbf{s}, \mathbf{a})$$

MPC or Sim2Real - It's the same question...

The theory is about model-based decisions, and equivalences between MDPs

World MDP

World MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$V^\star(\mathbf{s}) = \min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

$$Q^\star(\mathbf{s}, \mathbf{a}) = L(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \rho}[V^\star(\mathbf{s}_+)|\mathbf{s}, \mathbf{a}]$$

Optimal policy

$$\boldsymbol{\pi}^\star(\mathbf{s}) = \arg\min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

Model MDP (simulation environment)

Model MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$\hat{V}^\star(\mathbf{s}) = \min_{\mathbf{a}} \hat{Q}^\star(\mathbf{s}, \mathbf{a})$$

$$\hat{Q}^\star(\mathbf{s}, \mathbf{a}) = \hat{L}(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \hat{\rho}}[V^\star(\hat{\mathbf{s}}_+)|\mathbf{s}, \mathbf{a}]$$

Optimal policy

$$\hat{\boldsymbol{\pi}}^\star(\mathbf{s}) = \arg\min_{\mathbf{a}} \hat{Q}^\star(\mathbf{s}, \mathbf{a})$$

Theory says that - under some technical conditions - there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$

MPC or Sim2Real - It's the same question...

The theory is about model-based decisions, and equivalences between MDPs

World MDP

World MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$V^\star(\mathbf{s}) = \min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

$$Q^\star(\mathbf{s}, \mathbf{a}) = L(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \rho}[V^\star(\mathbf{s}_+)|\mathbf{s}, \mathbf{a}]$$

Optimal policy

$$\pi^\star(\mathbf{s}) = \arg\min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

Model MDP (simulation environment)

Model MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$\hat{V}^\star(\mathbf{s}) = \min_{\mathbf{a}} \hat{Q}^\star(\mathbf{s}, \mathbf{a})$$

$$\hat{Q}^\star(\mathbf{s}, \mathbf{a}) = \hat{L}(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \hat{\rho}}[V^\star(\hat{\mathbf{s}}_+)|\mathbf{s}, \mathbf{a}]$$

Optimal policy

$$\hat{\pi}^\star(\mathbf{s}) = \arg\min_{\mathbf{a}} \hat{Q}^\star(\mathbf{s}, \mathbf{a})$$

Theory says that - under some technical conditions - there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$

Proof: telescopic sum, some non-trivial assumptions to prevent $\infty - \infty$ cancellations

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)
- World MDP and Model MDP do not need to use the same discount γ

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)
- World MDP and Model MDP do not need to use the same discount γ

In-Sim policy training uses a Model MDP

- Practitioners mostly work on model $\hat{\rho}$, but "unsure" on how to tune it
- In the ML communities, people talk about "value alignment", we are trying to make this "MDP equivalence" understood

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)
- World MDP and Model MDP do not need to use the same discount γ

Remarks

- Non-unique optimal model $\hat{\rho}$ leaves room for aligning it to the real world (classical fitting)

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)
- World MDP and Model MDP do not need to use the same discount γ

Remarks

- Non-unique optimal model $\hat{\rho}$ leaves room for aligning it to the real world (classical fitting)
- If $V^\star$ is continuous and support of ρ is bounded(?) and path-connected, then we can have support $\hat{\rho} \subset$ support of ρ

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

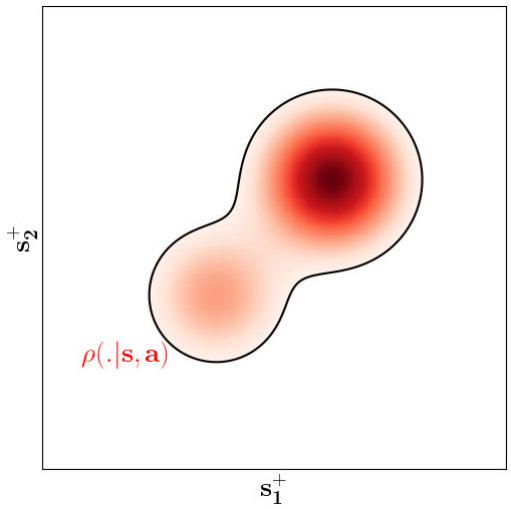
- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)
- World MDP and Model MDP do not need to use the same discount γ

Remarks

- Non-unique optimal model $\hat{\rho}$ leaves room for aligning it to the real world (classical fitting)
- If $V^\star$ is continuous and support of ρ is bounded(?) and path-connected, then we can have support $\hat{\rho} \subset$ support of ρ



More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

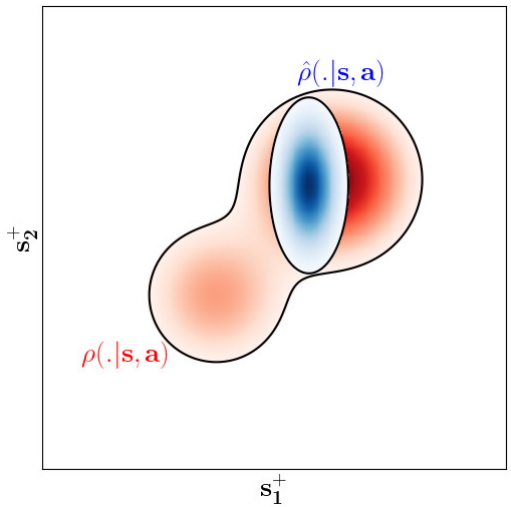
- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)
- World MDP and Model MDP do not need to use the same discount γ

Remarks

- Non-unique optimal model $\hat{\rho}$ leaves room for aligning it to the real world (classical fitting)
- If $V^\star$ is continuous and support of ρ is bounded(?) and path-connected, then we can have support $\hat{\rho} \subset$ support of ρ



More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

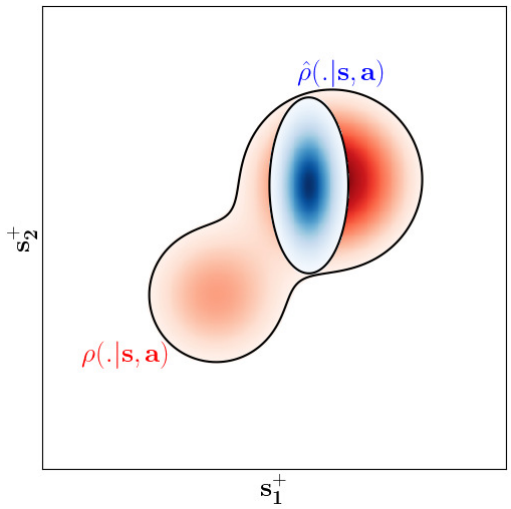
- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)
- World MDP and Model MDP do not need to use the same discount γ

Remarks

- Non-unique optimal model $\hat{\rho}$ leaves room for aligning it to the real world (classical fitting)
- If $V^\star$ is continuous and support of ρ is bounded(?) and path-connected, then we can have support $\hat{\rho} \subset$ support of ρ
- *More simply said*: we can build optimal models $\hat{\rho}$ that make "plausible" predictions about the real world.

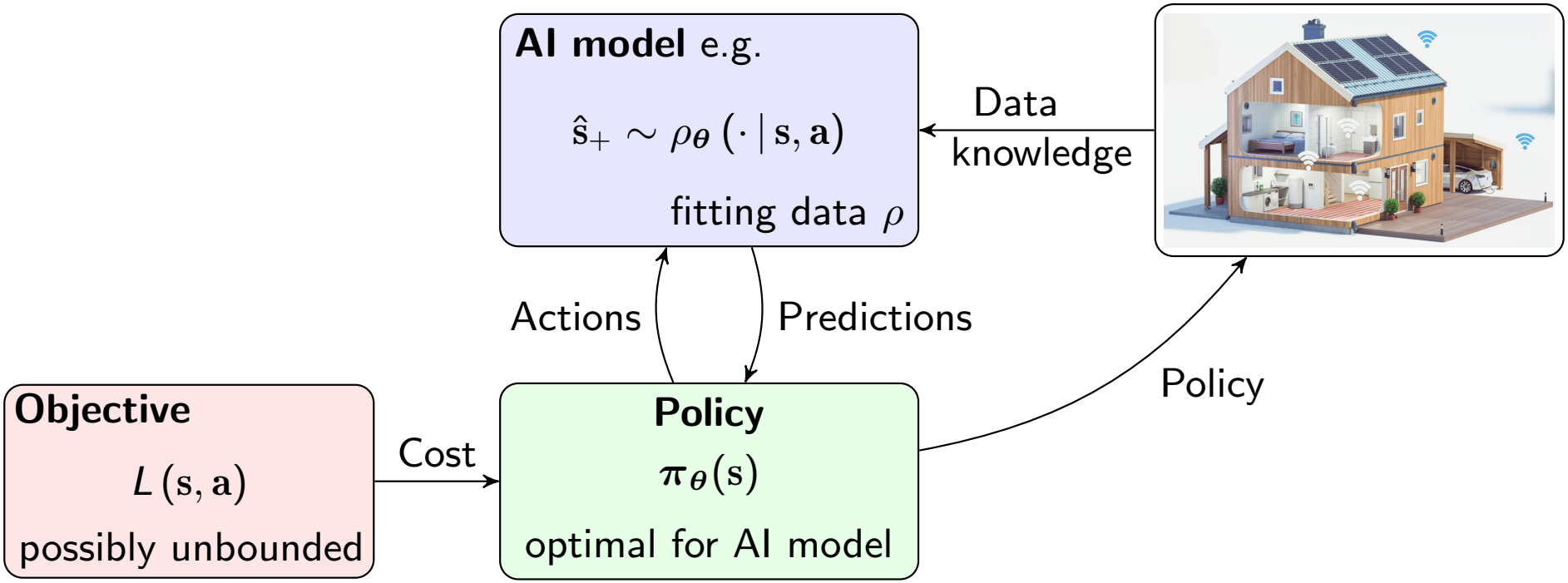


Outline

1. Decisions from data
2. AI models for Decision
3. Differentiating Policy Training
4. Examples



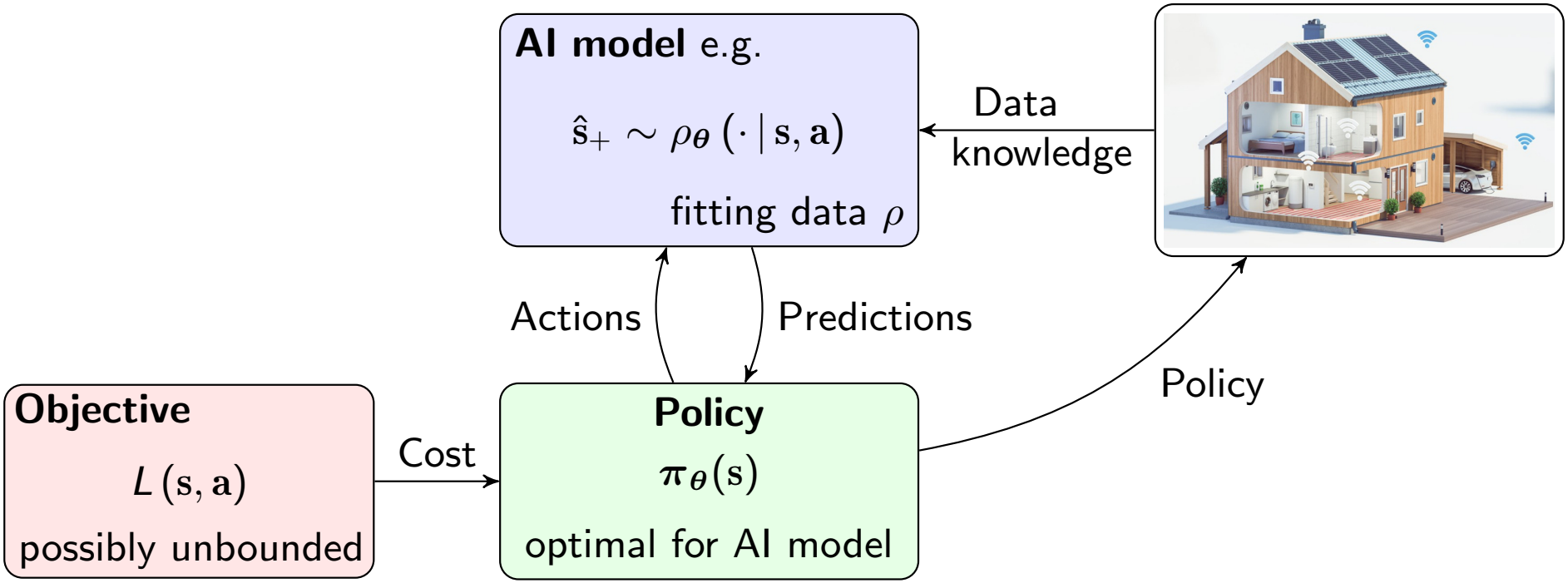
AI models for Decisions?



Learning model ρ_θ should aim at identifying

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\pi}_{\boldsymbol{\theta}}) \tag{1}$$

AI models for Decisions?

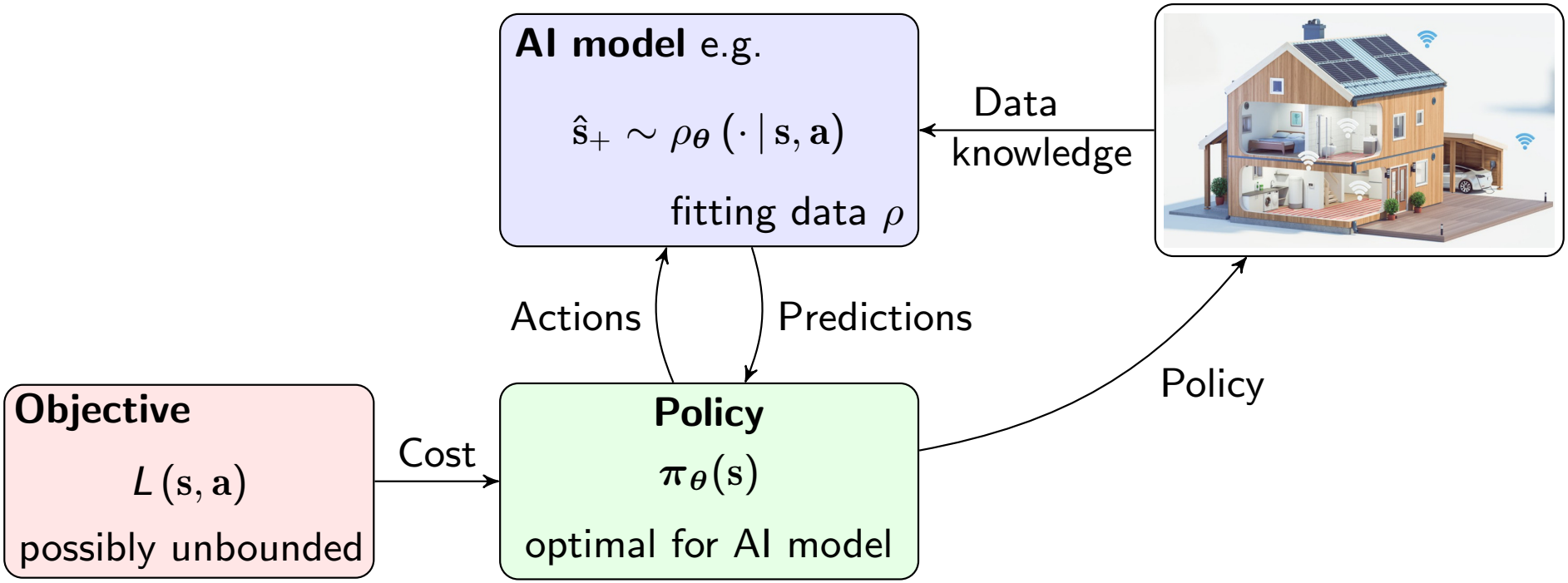


Learning model ρ_θ should aim at identifying

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}} J(\pi_{\boldsymbol{\theta}}) \tag{1}$$

- Closed-loop performance J is intricate ($\boldsymbol{\theta} \rightarrow$ simulations $\rightarrow$ policy $\rightarrow$ real system)

AI models for Decisions?

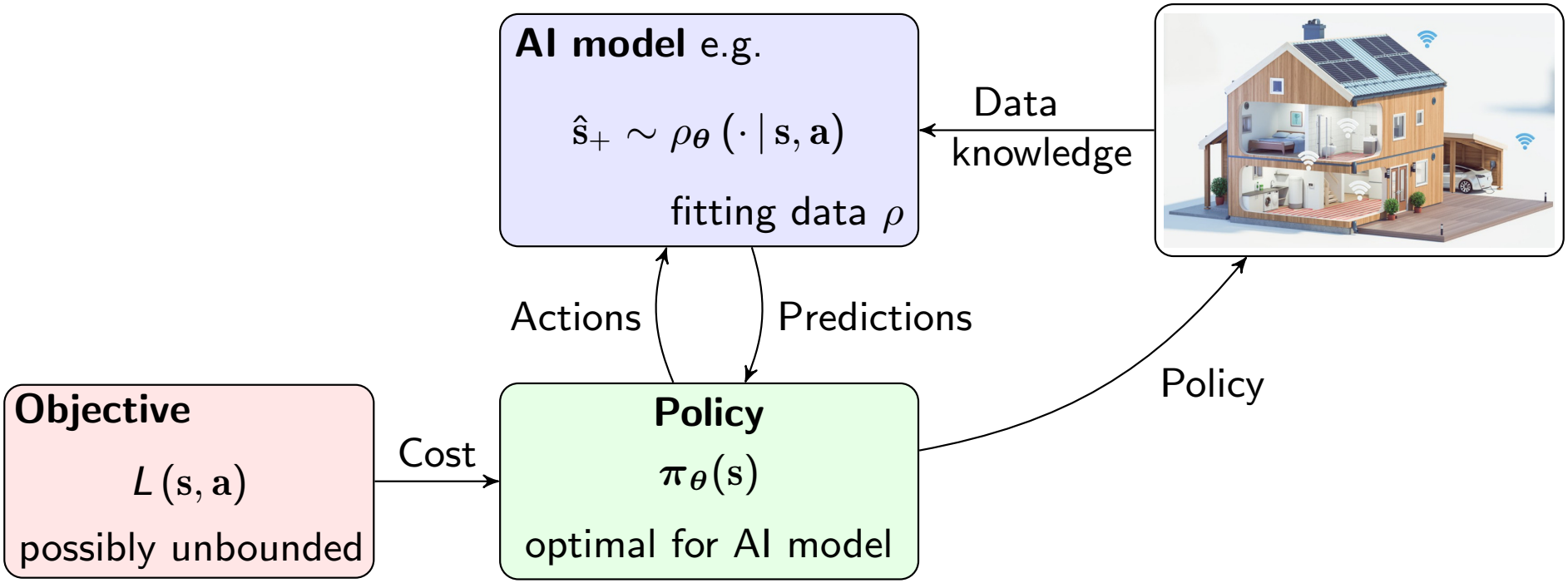


Learning model ρ_θ should aim at identifying

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\pi}_{\boldsymbol{\theta}}) \tag{1}$$

- Closed-loop performance J is intricate ($\boldsymbol{\theta} \rightarrow$ simulations $\rightarrow$ policy $\rightarrow$ real system)
- $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\pi}_{\boldsymbol{\theta}})$ is to be estimated from data, i.e. data hungry, noisy, possible biases

AI models for Decisions?

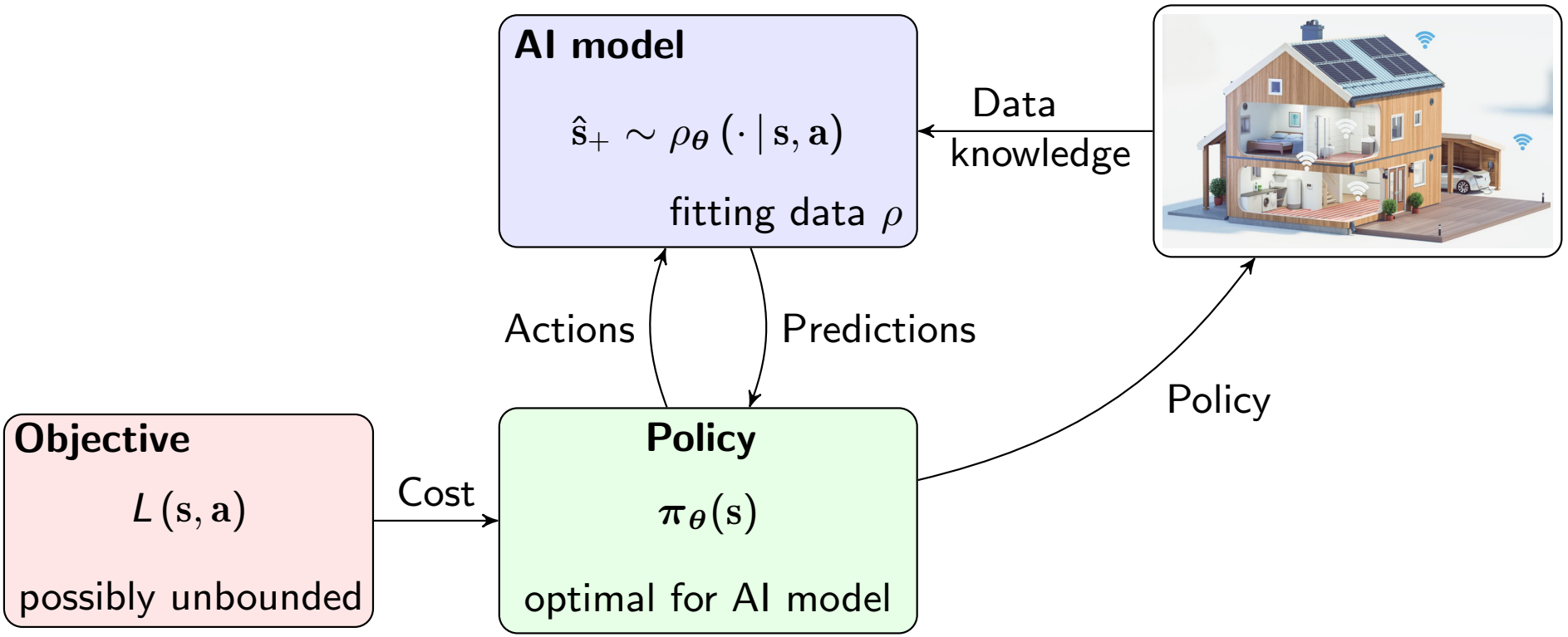


Learning model ρ_θ should aim at identifying

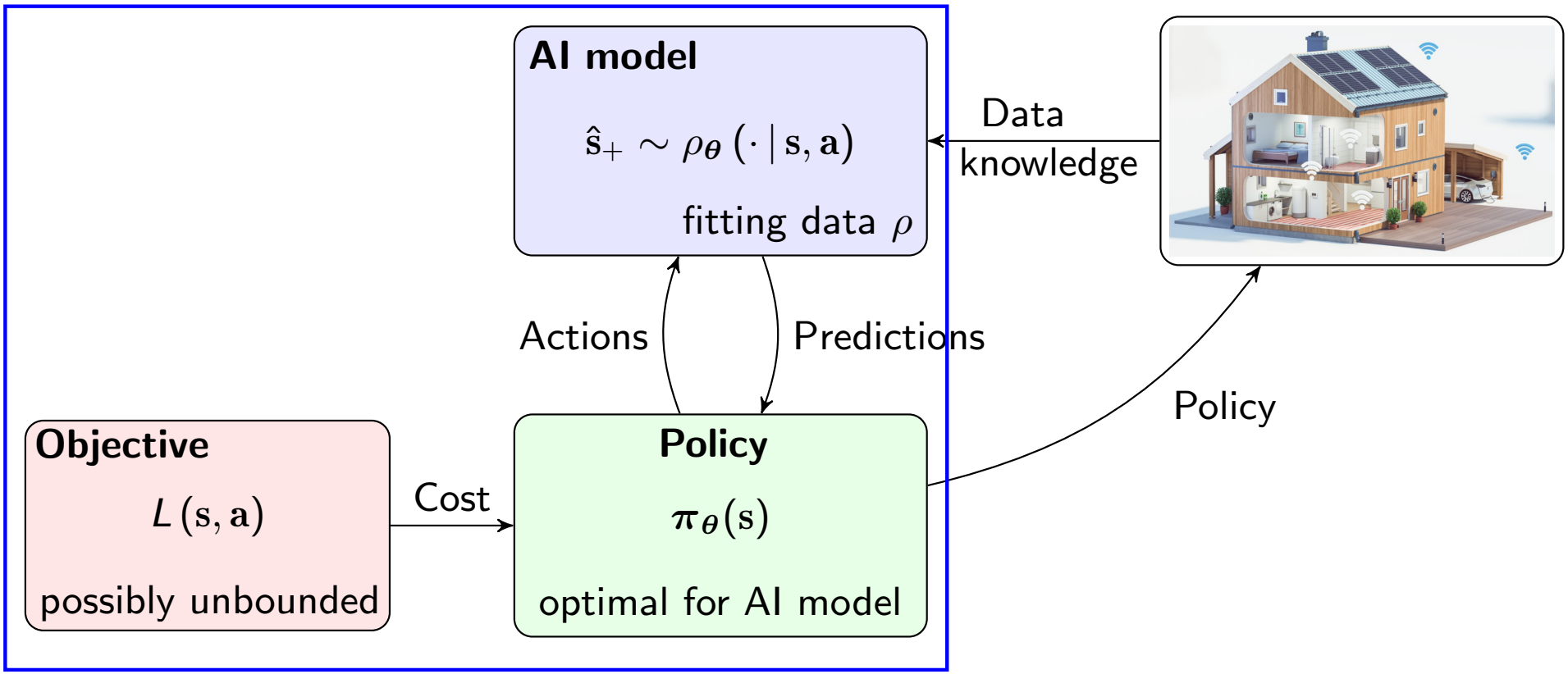
$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\pi}_{\boldsymbol{\theta}}) \tag{1}$$

- Closed-loop performance J is intricate ($\boldsymbol{\theta} \rightarrow$ simulations $\rightarrow$ policy $\rightarrow$ real system)
- $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\pi}_{\boldsymbol{\theta}})$ is to be estimated from data, i.e. data hungry, noisy, possible biases
- (1) is in general different than model fitting, i.e. no loss function does (1)

Optimal AI models for Decision - A Paradigm Shift

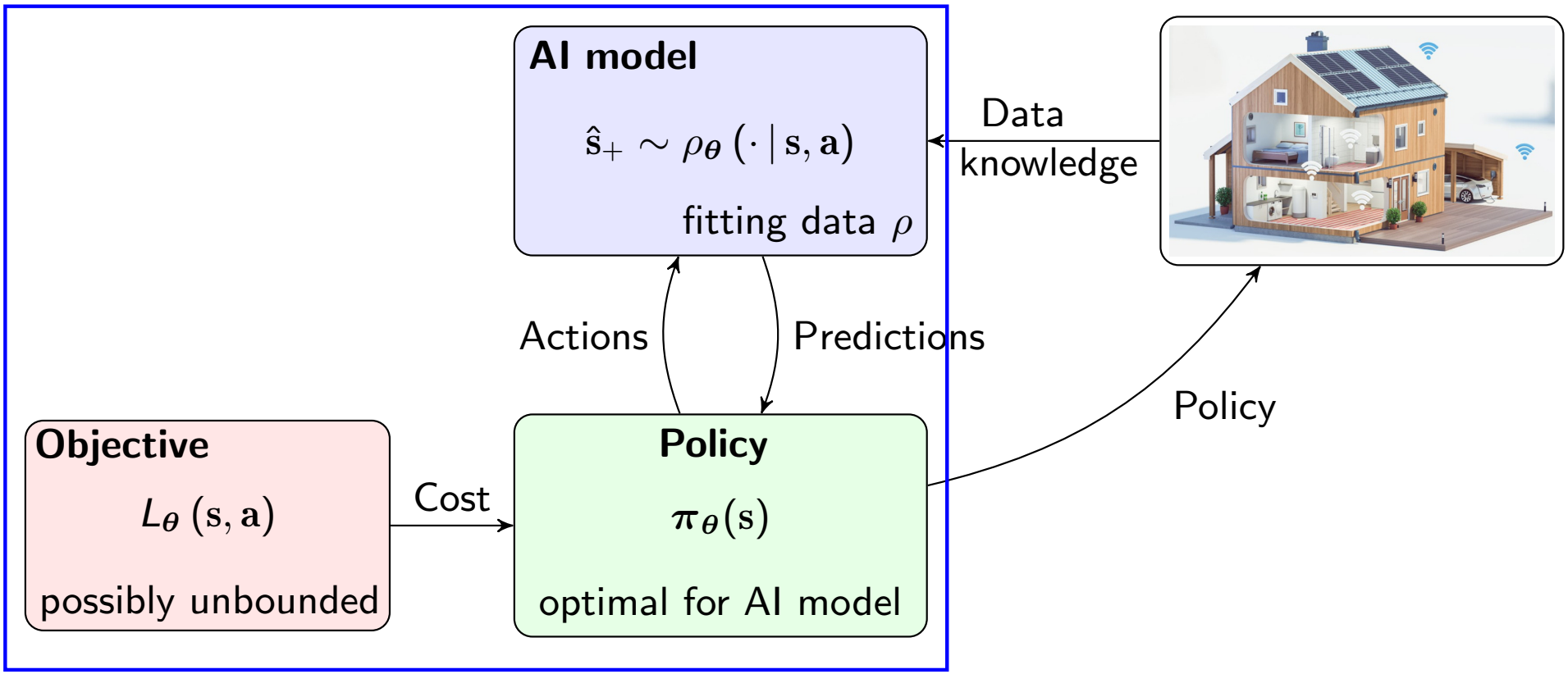


Optimal AI models for Decision - A Paradigm Shift



- The model is not $\rho_{\boldsymbol{\theta}}$, it is the entire "decision-making box"

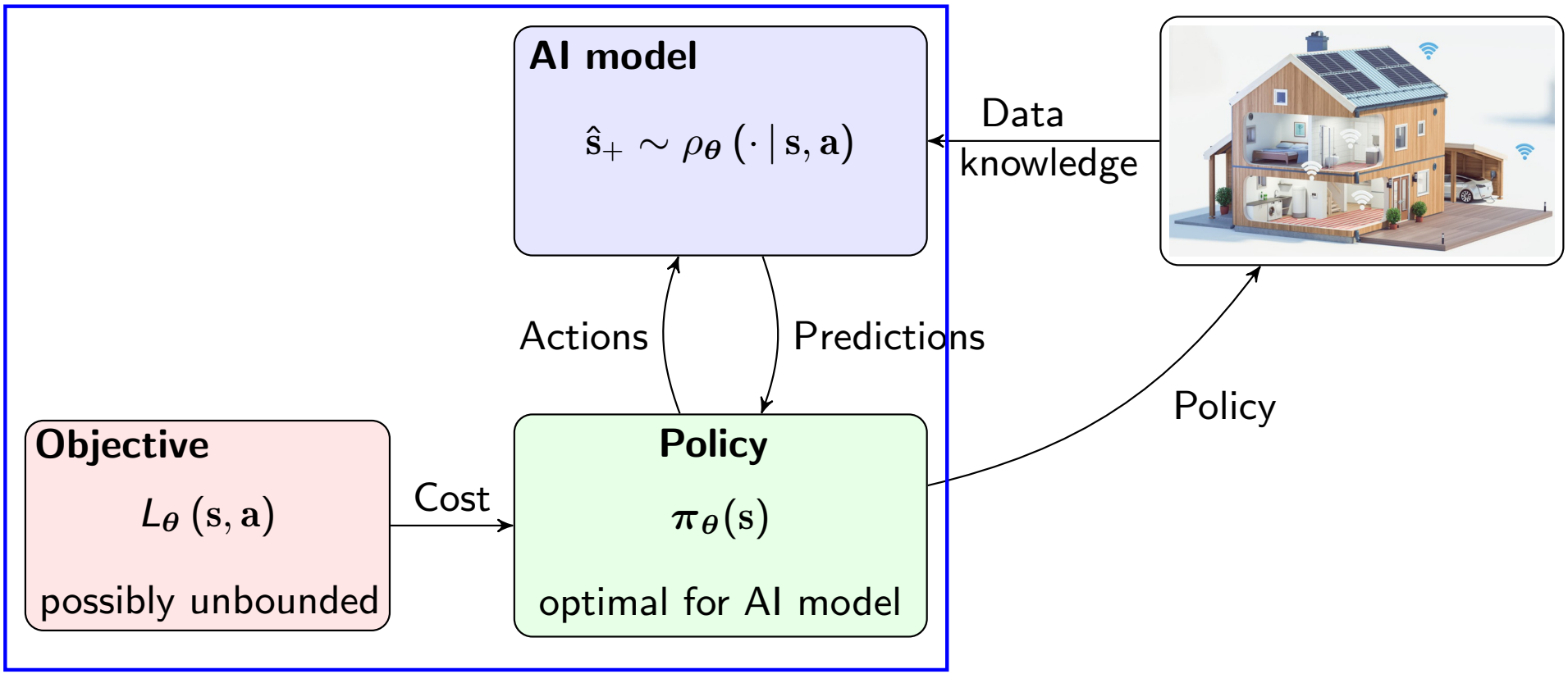
Optimal AI models for Decision - A Paradigm Shift



- The model is not $\rho_{\boldsymbol{\theta}}$, it is the entire "decision-making box"
- The model includes objective $L_{\boldsymbol{\theta}}$ used to build the policy*

* policy performance on real system still assessed via L

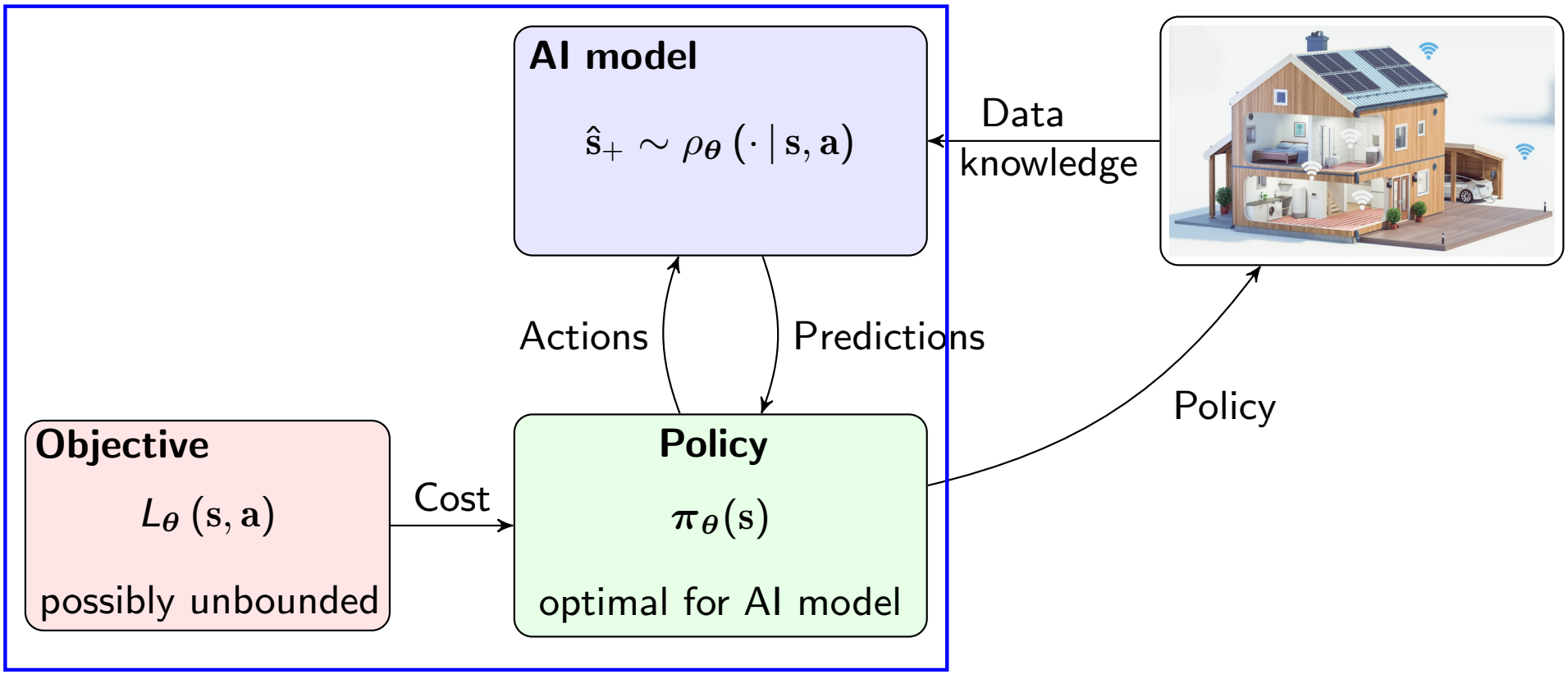
Optimal AI models for Decision - A Paradigm Shift



- The model is not $\rho_{\boldsymbol{\theta}}$, it is the entire "decision-making box"
- The model includes objective $L_{\boldsymbol{\theta}}$ used to build the policy*
- Best model $\rho_{\boldsymbol{\theta}}$ should *not necessarily* represent the data in a "classical sense"

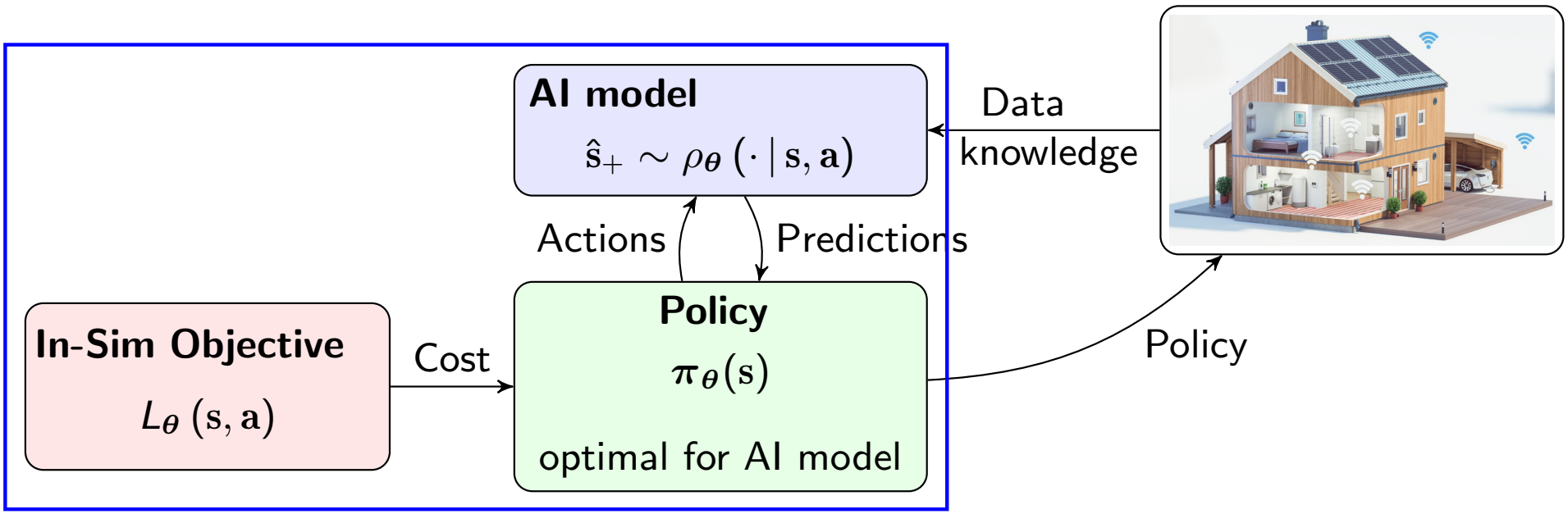
* policy performance on real system still assessed via L

Optimal AI models for Decision - A Paradigm Shift

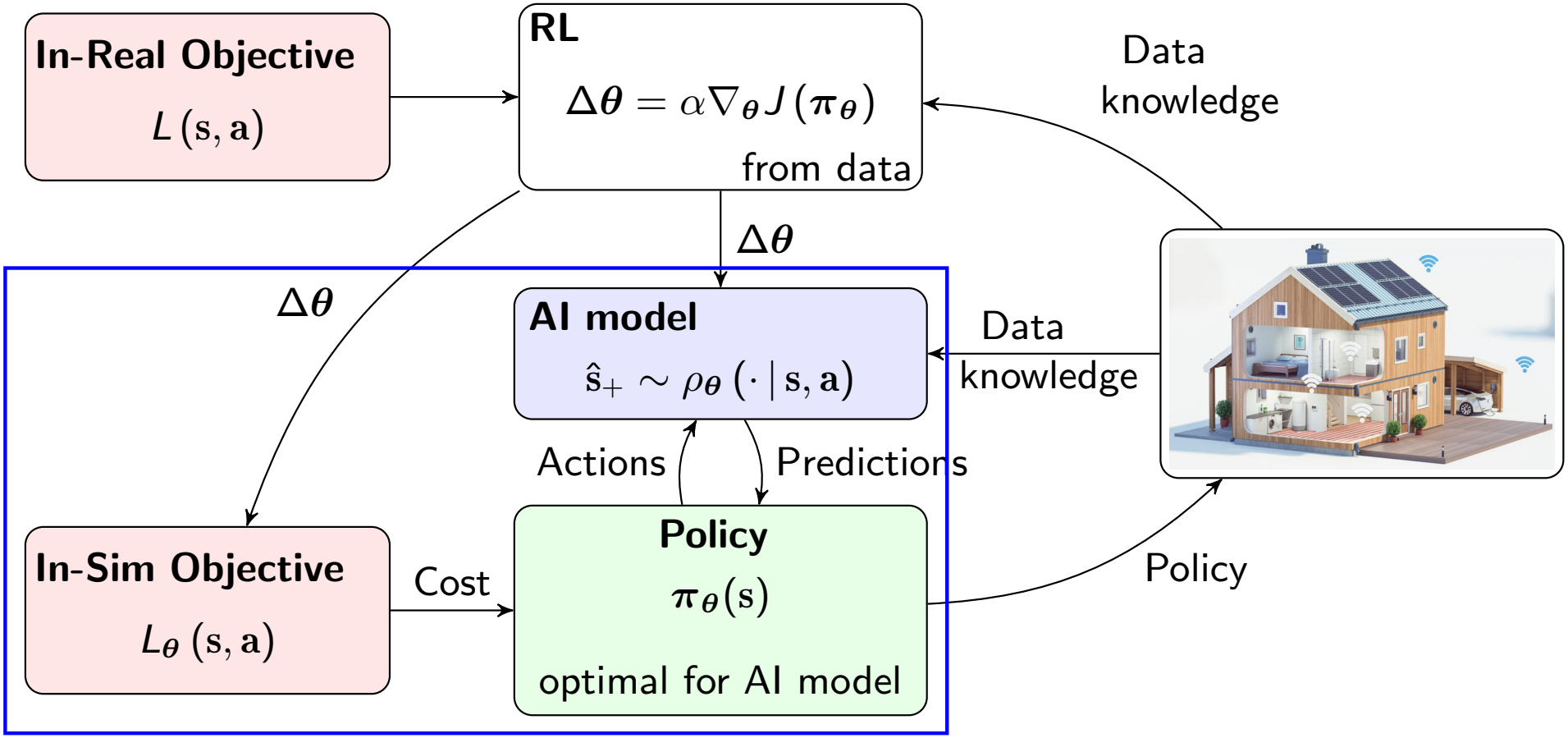


Theorem: under some (technical) assumptions, there is a $L_{\boldsymbol{\theta}}$ such that $\boldsymbol{\pi}_{\boldsymbol{\theta}} = \boldsymbol{\pi}_\star$, even if $\rho_{\boldsymbol{\theta}}$ does not represent real world ρ correctly

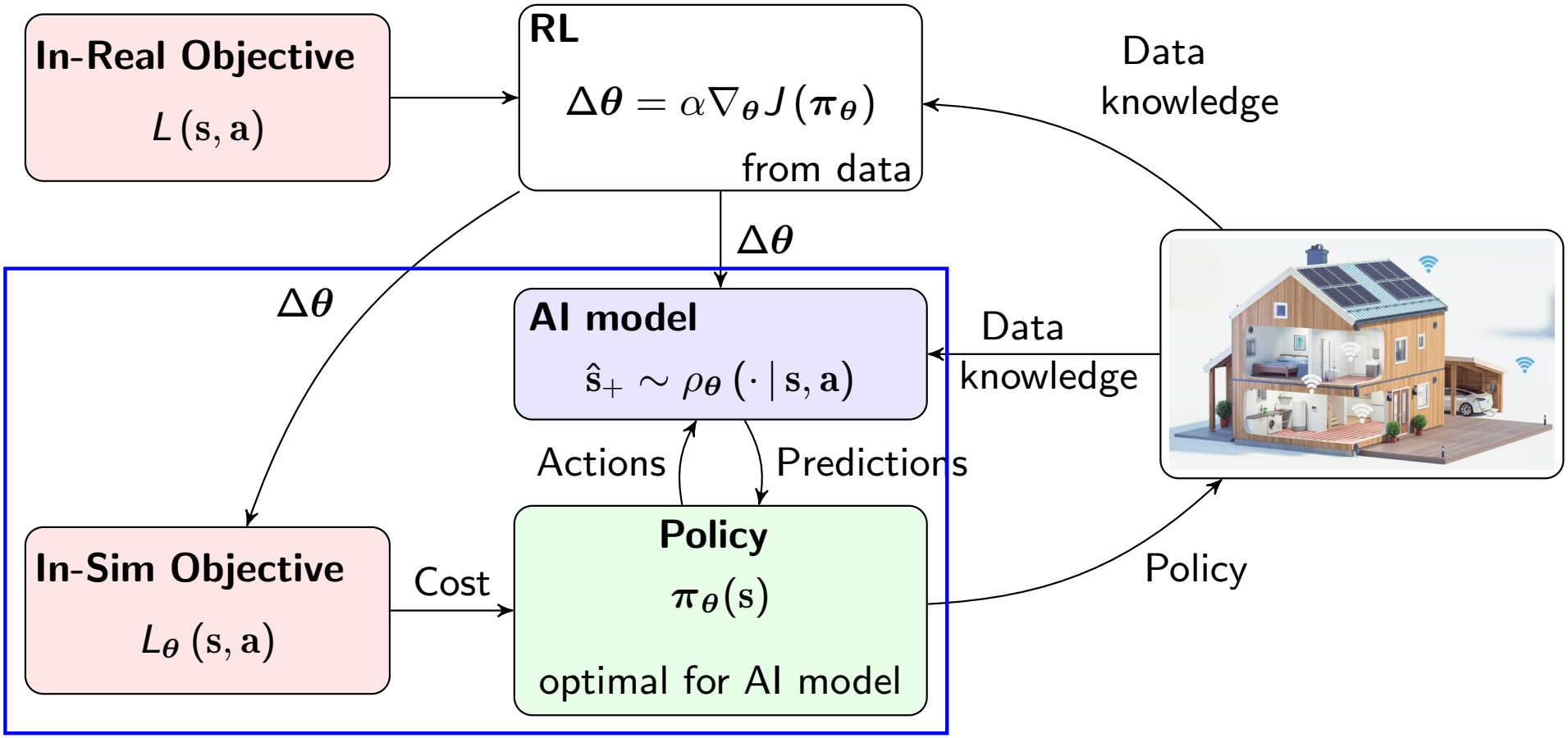
How to use this more generally? RL over Policy Training



How to use this more generally? RL over Policy Training



How to use this more generally? RL over Policy Training

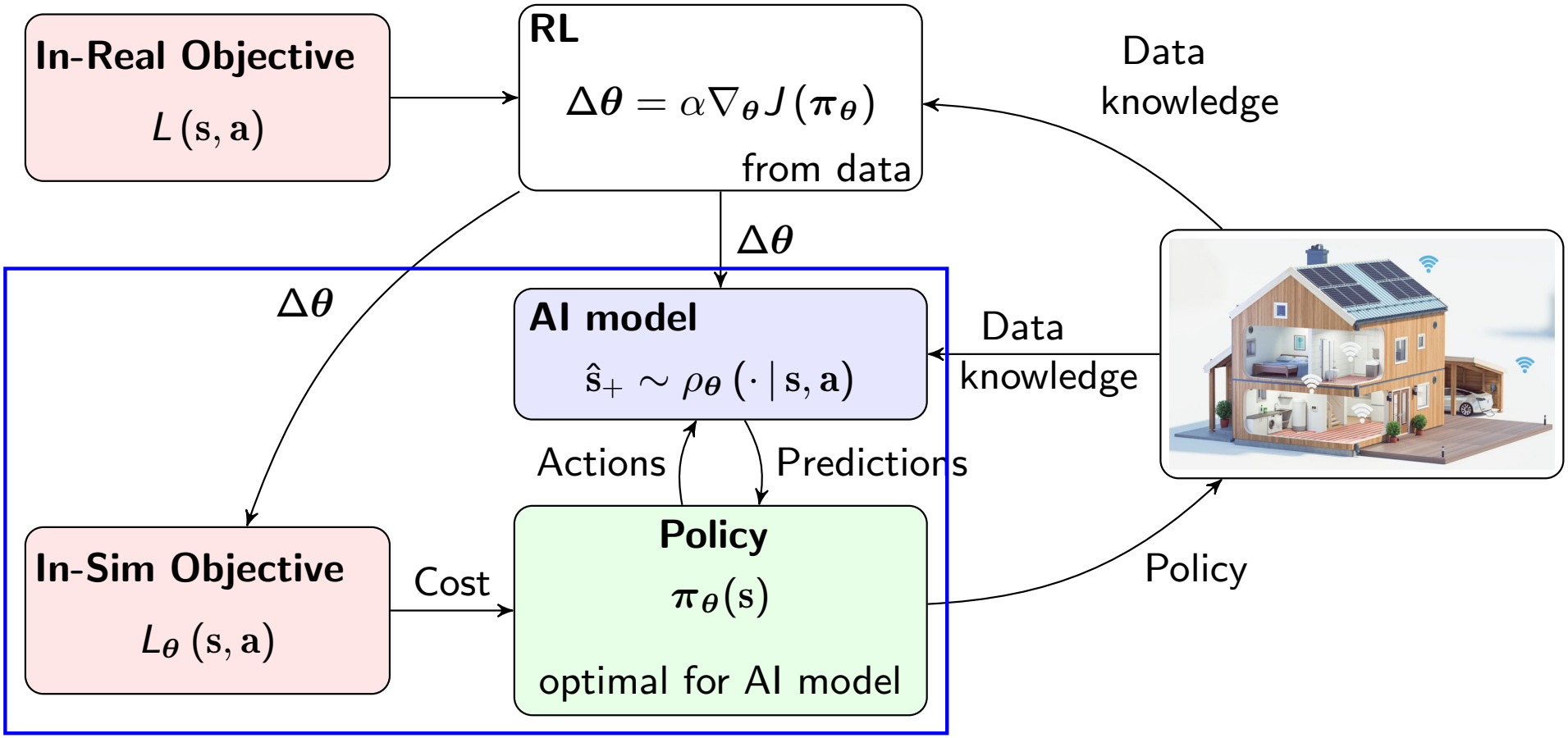


Policy gradient

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\pi}_{\boldsymbol{\theta}}) = \mathbb{E}\left[\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}} \nabla_{\mathbf{a}} Q^{\boldsymbol{\pi}_{\boldsymbol{\theta}}}\right]$$

- $Q^{\boldsymbol{\pi}_{\boldsymbol{\theta}}}$ is the critic, well-established RL tool
- $\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}}$ requires differentiating the closed-loop simulation...

How to use this more generally? RL over Policy Training



Difficulty: computing $\nabla_{\boldsymbol{\theta}}\boldsymbol{\pi}_{\boldsymbol{\theta}}$ requires total differentiation through the simulation-based policy optimization process:

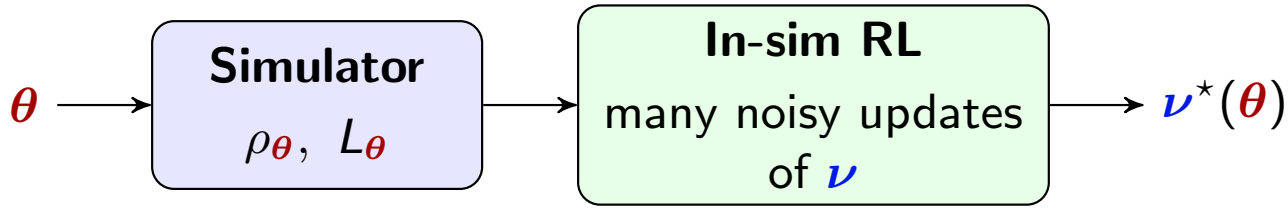
$\boldsymbol{\theta} \rightarrow$ simulations $\rightarrow$ optimal policy for $\rho_{\boldsymbol{\theta}}$

Differentiating through policy optimization can be computationally heavy

Outline

1. Decisions from data
2. AI models for Decision
3. Differentiating Policy Training
4. Examples

What we need: the sensitivity of a trained policy

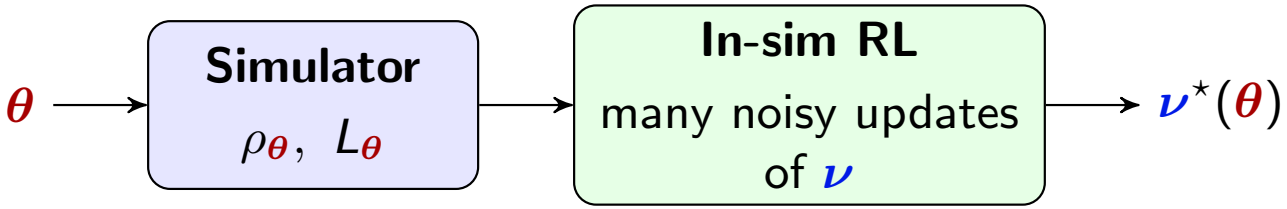


In-sim policy training, for a given $\boldsymbol{\theta}$

$$\boldsymbol{\nu}^\star(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\nu}} \ \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu})$$

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \mathbb{E}_{\rho_{\boldsymbol{\theta}}, \pi_{\boldsymbol{\nu}}}\left[\sum\nolimits_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)\right]$$

What we need: the sensitivity of a trained policy



In-sim policy training, for a given $\boldsymbol{\theta}$

$$\boldsymbol{\nu}^\star(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\nu}} \ \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu})$$

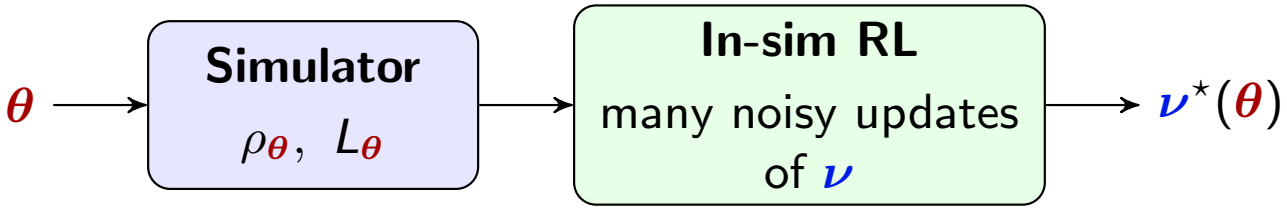
$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \mathbb{E}_{\rho_{\boldsymbol{\theta}}, \pi_{\boldsymbol{\nu}}}\left[\sum\nolimits_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)\right]$$

Policy sensitivity

$$\boldsymbol{\kappa} := \frac{\mathrm{d}\boldsymbol{\nu}^\star}{\mathrm{d}\boldsymbol{\theta}} \in \mathbb{R}^{\dim \boldsymbol{\nu} \times \dim \boldsymbol{\theta}}$$

This is the missing piece of the previous slide:
$\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\big|_{\boldsymbol{\nu}^\star} \boldsymbol{\kappa}$

What we need: the sensitivity of a trained policy



In-sim policy training, for a given $\boldsymbol{\theta}$

$$\boldsymbol{\nu}^\star(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\nu}} \ \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu})$$

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \mathbb{E}_{\rho_{\boldsymbol{\theta}}, \pi_{\boldsymbol{\nu}}}\left[\sum\nolimits_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)\right]$$

Policy sensitivity

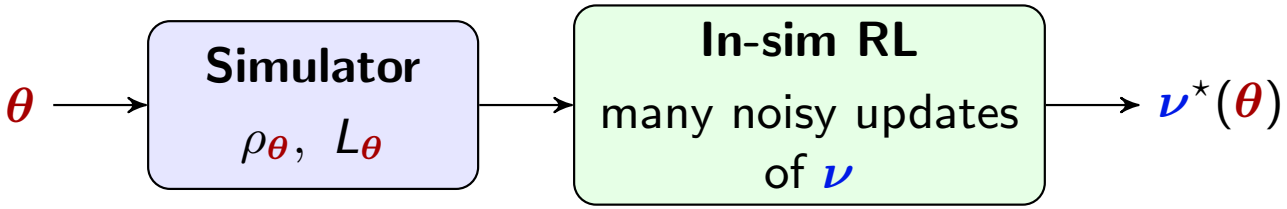
$$\boldsymbol{\kappa} := \frac{\mathrm{d}\boldsymbol{\nu}^\star}{\mathrm{d}\boldsymbol{\theta}} \in \mathbb{R}^{\dim \boldsymbol{\nu} \times \dim \boldsymbol{\theta}}$$

This is the missing piece of the previous slide:
$\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\big|_{\boldsymbol{\nu}^\star} \boldsymbol{\kappa}$

How to get $\boldsymbol{\kappa}$?

1. **Unroll training**: backpropagate through all updates $\boldsymbol{\nu}_{j+1} = \boldsymbol{\nu}_j - \beta\, \hat{\mathbf{g}}_j$
   ✘ cost $\propto$ number of updates

What we need: the sensitivity of a trained policy



In-sim policy training, for a given $\boldsymbol{\theta}$

$$\boldsymbol{\nu}^\star(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\nu}} \ \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu})$$

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \mathbb{E}_{\rho_{\boldsymbol{\theta}},\, \pi_{\boldsymbol{\nu}}}\left[\sum\nolimits_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)\right]$$

Policy sensitivity

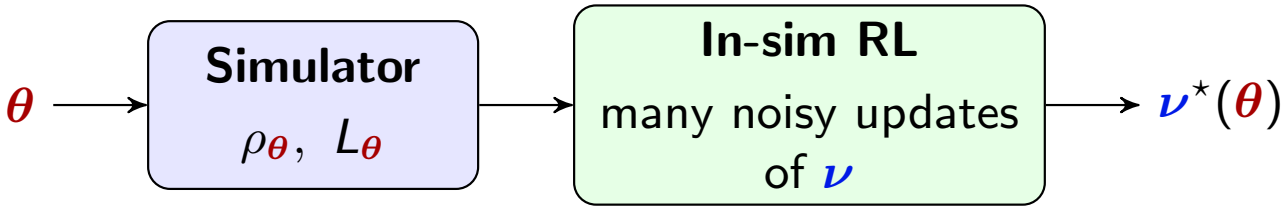
$$\boldsymbol{\kappa} := \frac{\mathrm{d}\boldsymbol{\nu}^\star}{\mathrm{d}\boldsymbol{\theta}} \in \mathbb{R}^{\dim \boldsymbol{\nu} \times \dim \boldsymbol{\theta}}$$

This is the missing piece of the previous slide:
$\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\big|_{\boldsymbol{\nu}^\star} \boldsymbol{\kappa}$

How to get $\boldsymbol{\kappa}$?

1. **Unroll training**: backpropagate through all updates $\boldsymbol{\nu}_{j+1} = \boldsymbol{\nu}_j - \beta\, \hat{\mathbf{g}}_j$
   ✘ cost $\propto$ number of updates
2. **Differentiate the Bellman optimality equations** (Nikishin et al., 2022)
   ✘ needs $\pi = \mathrm{softmax}(Q^\star)$

What we need: the sensitivity of a trained policy



In-sim policy training, for a given $\boldsymbol{\theta}$

$$\boldsymbol{\nu}^\star(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu})$$

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \mathbb{E}_{\rho_{\boldsymbol{\theta}}, \pi_{\boldsymbol{\nu}}}\left[\sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)\right]$$

Policy sensitivity

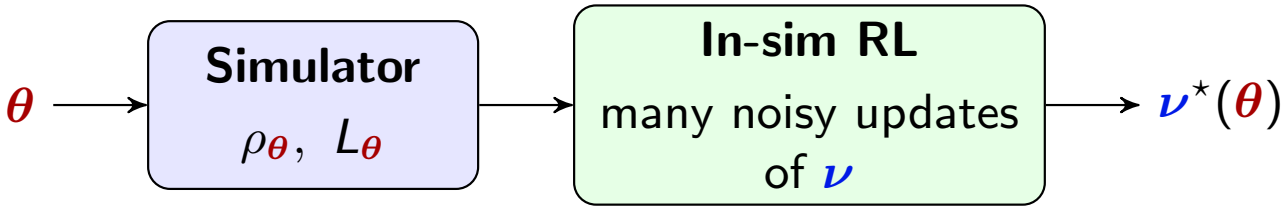
$$\boldsymbol{\kappa} := \frac{\mathrm{d}\boldsymbol{\nu}^\star}{\mathrm{d}\boldsymbol{\theta}} \in \mathbb{R}^{\dim \boldsymbol{\nu} \times \dim \boldsymbol{\theta}}$$

This is the missing piece of the previous slide:
$\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\big|_{\boldsymbol{\nu}^\star} \boldsymbol{\kappa}$

How to get $\boldsymbol{\kappa}$?

1. **Unroll training**: backpropagate through all updates $\boldsymbol{\nu}_{j+1} = \boldsymbol{\nu}_j - \beta\, \hat{\mathbf{g}}_j$
   ✘ cost $\propto$ number of updates
2. **Differentiate the Bellman optimality equations** (Nikishin et al., 2022)
   ✘ needs $\pi = \mathrm{softmax}(Q^\star)$
3. **Implicitly differentiate the end point of training**
   ✔ any policy class, any policy gradient

What we need: the sensitivity of a trained policy



In-sim policy training, for a given $\boldsymbol{\theta}$

$$\boldsymbol{\nu}^\star(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\nu}} \ \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu})$$

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \mathbb{E}_{\rho_{\boldsymbol{\theta}},\, \pi_{\boldsymbol{\nu}}}\left[\sum\nolimits_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)\right]$$

Policy sensitivity

$$\boldsymbol{\kappa} := \frac{\mathrm{d}\boldsymbol{\nu}^\star}{\mathrm{d}\boldsymbol{\theta}} \in \mathbb{R}^{\dim \boldsymbol{\nu} \times \dim \boldsymbol{\theta}}$$

This is the missing piece of the previous slide:
$\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\big|_{\boldsymbol{\nu}^\star} \boldsymbol{\kappa}$

How to get $\boldsymbol{\kappa}$?

1. **Unroll training**: backpropagate through all updates $\boldsymbol{\nu}_{j+1} = \boldsymbol{\nu}_j - \beta\, \hat{\mathbf{g}}_j$
   ✘ cost $\propto$ number of updates
2. **Differentiate the Bellman optimality equations** (Nikishin et al., 2022)
   ✘ needs $\pi = \mathrm{softmax}(Q^\star)$
3. **Implicitly differentiate the end point of training**
   ✔ any policy class, any policy gradient

So first: How do we differentiate through an MDP at all?

Two ways to differentiate an expectation

- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\left[f(x)\right]$ from samples $x_1, \ldots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)

Two ways to differentiate an expectation

- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\left[f(x)\right]$ from samples $x_1, \ldots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)
- **Idea**: a gradient is the change of the sample average when ψ moves a little

$$\nabla_\psi \mathbb{E}\left[f(x)\right] \approx \frac{\mathbb{E}_{\psi+\mathrm{d}\psi}[f] - \mathbb{E}_\psi[f]}{\mathrm{d}\psi}, \qquad \mathbb{E}_\psi[f] \approx \frac{1}{n}\sum_i f(x_i)$$

- **Question**: how to get the *new* average $\mathbb{E}_{\psi+\mathrm{d}\psi}[f]$ from the *same* samples?

Two ways to differentiate an expectation

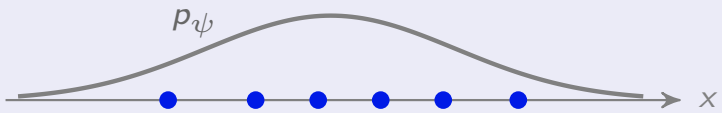
- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\big[f(x)\big]$ from samples $x_1, \ldots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)
- **Idea**: a gradient is the change of the sample average when ψ moves a little

$$\nabla_\psi \mathbb{E}\big[f(x)\big] \approx \frac{\mathbb{E}_{\psi+\mathrm{d}\psi}[f] - \mathbb{E}_\psi[f]}{\mathrm{d}\psi}, \qquad \mathbb{E}_\psi[f] \approx \frac{1}{n}\sum_i f(x_i)$$

- **Question**: how to get the *new* average $\mathbb{E}_{\psi+\mathrm{d}\psi}[f]$ from the *same* samples?

LR (likelihood ratio): **re-weight** samples

$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}\Big[f(x)\,\nabla_\psi \log p_\psi(x)\Big]$$



- samples $x_i \sim p_\psi$ stay where they are

Pictures: $\psi = \mu$, $x = \mu + \varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$; gray $= p_\psi$, dashed $= p_{\psi+\mathrm{d}\psi}$

Two ways to differentiate an expectation

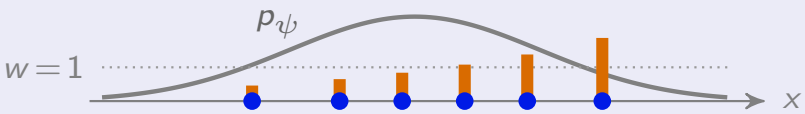
- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\big[f(x)\big]$ from samples $x_1, \dots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)
- **Idea**: a gradient is the change of the sample average when ψ moves a little

$$\nabla_\psi \mathbb{E}\big[f(x)\big] \approx \frac{\mathbb{E}_{\psi+\mathrm{d}\psi}[f] - \mathbb{E}_\psi[f]}{\mathrm{d}\psi}, \qquad \mathbb{E}_\psi[f] \approx \frac{1}{n}\sum_i f(x_i)$$

- **Question**: how to get the *new* average $\mathbb{E}_{\psi+\mathrm{d}\psi}[f]$ from the *same* samples?

LR (likelihood ratio): **re-weight** samples

$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}\Big[f(x)\,\nabla_\psi \log p_\psi(x)\Big]$$



- samples $x_i \sim p_\psi$ stay where they are
- $w_i = \frac{p_{\psi+\mathrm{d}\psi}}{p_\psi}(x_i) \approx 1 + \nabla_\psi \log p_\psi(x_i)\,\mathrm{d}\psi$

Pictures: $\psi = \mu$, $x = \mu + \varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$; gray $= p_\psi$, dashed $= p_{\psi+\mathrm{d}\psi}$

Two ways to differentiate an expectation

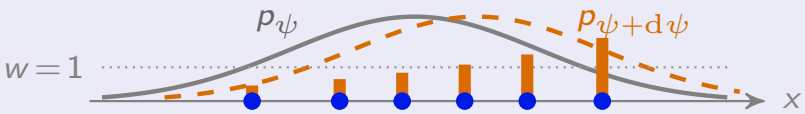
- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\big[f(x)\big]$ from samples $x_1, \dots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)
- **Idea**: a gradient is the change of the sample average when ψ moves a little

$$\nabla_\psi \mathbb{E}\big[f(x)\big] \approx \frac{\mathbb{E}_{\psi+\mathrm{d}\psi}[f] - \mathbb{E}_\psi[f]}{\mathrm{d}\psi}, \qquad \mathbb{E}_\psi[f] \approx \frac{1}{n}\sum_i f(x_i)$$

- **Question**: how to get the *new* average $\mathbb{E}_{\psi+\mathrm{d}\psi}[f]$ from the *same* samples?

LR (likelihood ratio): **re-weight** samples

$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}\Big[f(x)\,\nabla_\psi \log p_\psi(x)\Big]$$



- samples $x_i \sim p_\psi$ stay where they are
- $w_i = \frac{p_{\psi+\mathrm{d}\psi}}{p_\psi}(x_i) \approx 1 + \nabla_\psi \log p_\psi(x_i)\,\mathrm{d}\psi$
- new average $\approx \frac{1}{n}\sum_i w_i\, f(x_i)$

Pictures: $\psi = \mu$, $x = \mu + \varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$; gray $= p_\psi$, dashed $= p_{\psi+\mathrm{d}\psi}$

Two ways to differentiate an expectation

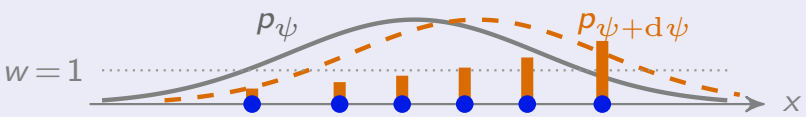
- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\big[f(x)\big]$ from samples $x_1, \dots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)
- **Idea**: a gradient is the change of the sample average when ψ moves a little

$$\nabla_\psi \mathbb{E}\big[f(x)\big] \approx \frac{\mathbb{E}_{\psi+\mathrm{d}\psi}[f] - \mathbb{E}_\psi[f]}{\mathrm{d}\psi}, \qquad \mathbb{E}_\psi[f] \approx \frac{1}{n}\sum_i f(x_i)$$

- **Question**: how to get the *new* average $\mathbb{E}_{\psi+\mathrm{d}\psi}[f]$ from the *same* samples?

LR (likelihood ratio): **re-weight** samples

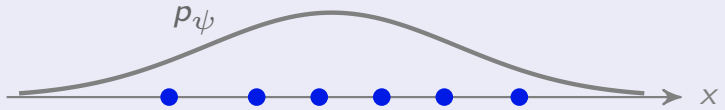
$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}\Big[f(x)\,\nabla_\psi \log p_\psi(x)\Big]$$



- samples $x_i \sim p_\psi$ stay where they are
- $w_i = \frac{p_{\psi+\mathrm{d}\psi}}{p_\psi}(x_i) \approx 1 + \nabla_\psi \log p_\psi(x_i)\,\mathrm{d}\psi$
- new average $\approx \frac{1}{n}\sum_i w_i\, f(x_i)$

PD (reparameterization): **move** samples

$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}_\varepsilon\Big[\nabla_x f(x)\,\nabla_\psi g_\psi(\varepsilon)\Big]$$



- samples $x_i = g_\psi(\varepsilon_i)$, noise $\varepsilon_i \sim p(\varepsilon)$

Pictures: $\psi = \mu$, $x = \mu + \varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$; gray $= p_\psi$, dashed $= p_{\psi+\mathrm{d}\psi}$

Two ways to differentiate an expectation

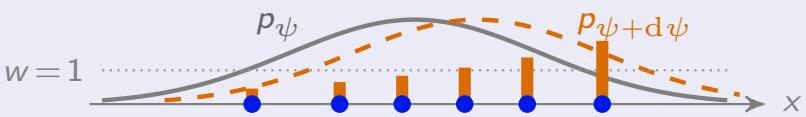
- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\big[f(x)\big]$ from samples $x_1, \dots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)
- **Idea**: a gradient is the change of the sample average when ψ moves a little

$$\nabla_\psi \mathbb{E}\big[f(x)\big] \approx \frac{\mathbb{E}_{\psi+\mathrm{d}\psi}[f] - \mathbb{E}_\psi[f]}{\mathrm{d}\psi}, \qquad \mathbb{E}_\psi[f] \approx \frac{1}{n}\sum_i f(x_i)$$

- **Question**: how to get the *new* average $\mathbb{E}_{\psi+\mathrm{d}\psi}[f]$ from the *same* samples?

LR (likelihood ratio): **re-weight** samples

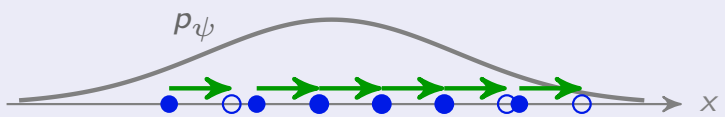
$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}\Big[f(x)\,\nabla_\psi \log p_\psi(x)\Big]$$



- samples $x_i \sim p_\psi$ stay where they are
- $w_i = \frac{p_{\psi+\mathrm{d}\psi}}{p_\psi}(x_i) \approx 1 + \nabla_\psi \log p_\psi(x_i)\,\mathrm{d}\psi$
- new average $\approx \frac{1}{n}\sum_i w_i\, f(x_i)$

PD (reparameterization): **move** samples

$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}_\varepsilon\Big[\nabla_x f(x)\,\nabla_\psi g_\psi(\varepsilon)\Big]$$



- samples $x_i = g_\psi(\varepsilon_i)$, noise $\varepsilon_i \sim p(\varepsilon)$
- keep ε_i, move: $x_i' \approx x_i + \nabla_\psi g_\psi(\varepsilon_i)\,\mathrm{d}\psi$

Pictures: $\psi = \mu$, $x = \mu + \varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$; gray $= p_\psi$, dashed $= p_{\psi+\mathrm{d}\psi}$

Two ways to differentiate an expectation

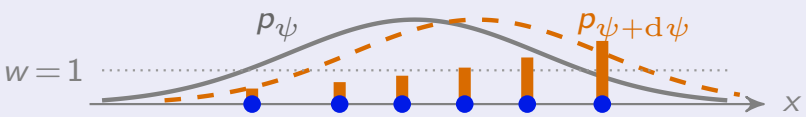
- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\big[f(x)\big]$ from samples $x_1, \dots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)
- **Idea**: a gradient is the change of the sample average when ψ moves a little

$$\nabla_\psi \mathbb{E}\big[f(x)\big] \approx \frac{\mathbb{E}_{\psi+\mathrm{d}\psi}[f] - \mathbb{E}_\psi[f]}{\mathrm{d}\psi}, \qquad \mathbb{E}_\psi[f] \approx \frac{1}{n}\sum_i f(x_i)$$

- **Question**: how to get the *new* average $\mathbb{E}_{\psi+\mathrm{d}\psi}[f]$ from the *same* samples?

LR (likelihood ratio): **re-weight** samples

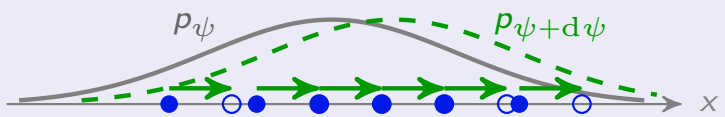
$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}\Big[f(x)\,\nabla_\psi \log p_\psi(x)\Big]$$



- samples $x_i \sim p_\psi$ stay where they are
- $w_i = \frac{p_{\psi+\mathrm{d}\psi}}{p_\psi}(x_i) \approx 1 + \nabla_\psi \log p_\psi(x_i)\,\mathrm{d}\psi$
- new average $\approx \frac{1}{n}\sum_i w_i\, f(x_i)$

PD (reparameterization): **move** samples

$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}_\varepsilon\Big[\nabla_x f(x)\,\nabla_\psi g_\psi(\varepsilon)\Big]$$



- samples $x_i = g_\psi(\varepsilon_i)$, noise $\varepsilon_i \sim p(\varepsilon)$
- keep ε_i, move: $x_i' \approx x_i + \nabla_\psi g_\psi(\varepsilon_i)\,\mathrm{d}\psi$
- new average $\approx \frac{1}{n}\sum_i f(x_i')$

Pictures: $\psi = \mu$, $x = \mu + \varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$; gray $= p_\psi$, dashed $= p_{\psi+\mathrm{d}\psi}$

Two families of policy gradients

RL: $x \rightarrow$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \rightarrow \boldsymbol{\nu}$, $f \rightarrow \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$
  - the model $\rho_{\boldsymbol{\theta}}$ drops out

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$
  - the model $\rho_{\boldsymbol{\theta}}$ drops out

PD (pathwise): **move** trajectories

- τ is a **function** of fixed noise (ε_k, ξ_k):
  $\mathbf{a}_k = g_{\boldsymbol{\nu}}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$
  - the model $\rho_{\boldsymbol{\theta}}$ drops out

PD (pathwise): **move** trajectories

- τ is a **function** of fixed noise (ε_k, ξ_k):
  $\mathbf{a}_k = g_{\boldsymbol{\nu}}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$
- gradients: $\frac{\mathrm{d}\mathbf{s}_{k+1}}{\mathrm{d}\boldsymbol{\nu}} = \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}$

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$
  - the model $\rho_{\boldsymbol{\theta}}$ drops out

PD (pathwise): **move** trajectories

- τ is a **function** of fixed noise (ε_k, ξ_k):
  $\mathbf{a}_k = g_{\boldsymbol{\nu}}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$
- gradients: $\frac{\mathrm{d}\mathbf{s}_{k+1}}{\mathrm{d}\boldsymbol{\nu}} = \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}$
  - the model $f_{\boldsymbol{\theta}}$ stays in

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$
  - the model $\rho_{\boldsymbol{\theta}}$ drops out

PD (pathwise): **move** trajectories

- τ is a **function** of fixed noise (ε_k, ξ_k):
  $\mathbf{a}_k = g_{\boldsymbol{\nu}}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$
- gradients: $\frac{\mathrm{d}\mathbf{s}_{k+1}}{\mathrm{d}\boldsymbol{\nu}} = \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}$
  - the model $f_{\boldsymbol{\theta}}$ stays in

The policy gradient in the two views

LR : $\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}\Big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)\Big]$ REINFORCE, A2C, PPO

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$
  - the model $\rho_{\boldsymbol{\theta}}$ drops out

PD (pathwise): **move** trajectories

- τ is a **function** of fixed noise (ε_k, ξ_k):
  $\mathbf{a}_k = g_{\boldsymbol{\nu}}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$
- gradients: $\frac{\mathrm{d}\mathbf{s}_{k+1}}{\mathrm{d}\boldsymbol{\nu}} = \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}$
  - the model $f_{\boldsymbol{\theta}}$ stays in

The policy gradient in the two views

LR: $\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}\Big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)\Big]$ REINFORCE, A2C, PPO

PD: $\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}_\omega\Big[\sum_k \gamma^k \big(\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}\big)\Big]$ BPTT, SVG, SVG

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**: $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$
  - the model $\rho_{\boldsymbol{\theta}}$ drops out

PD (pathwise): **move** trajectories

- τ is a **function** of fixed noise (ε_k, ξ_k): $\mathbf{a}_k = g_{\boldsymbol{\nu}}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$
- gradients: $\frac{\mathrm{d}\mathbf{s}_{k+1}}{\mathrm{d}\boldsymbol{\nu}} = \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}$
  - the model $f_{\boldsymbol{\theta}}$ stays in

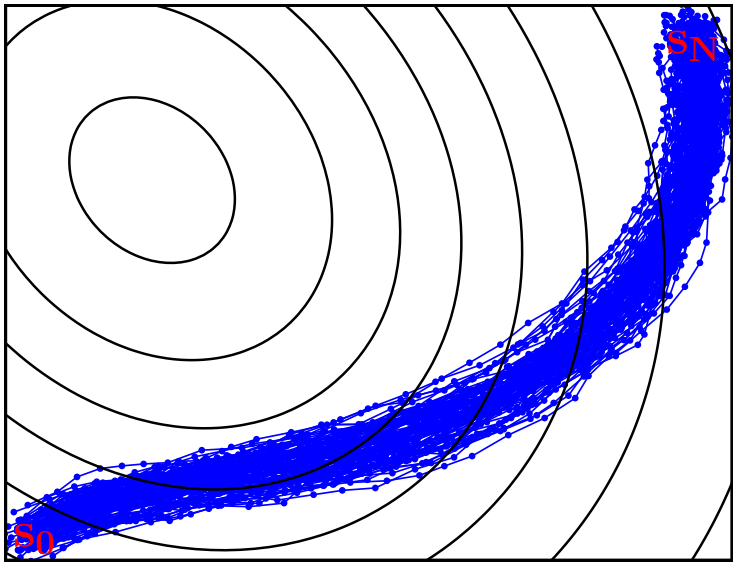
The policy gradient in the two views

LR : $\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}\Big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)\Big]$ REINFORCE, A2C, PPO

PD : $\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}_{\omega}\Big[\sum_k \gamma^k \big(\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}\big)\Big]$ BPTT, SVG, SVG

Two estimates of the same gradient, but the simulator $\boldsymbol{\theta}$ sits in different places:
LR: only in the **sampling** ($\rho_{\boldsymbol{\theta}}$) and the critic $Q_{\boldsymbol{\theta}}$; **PD**: inside each path, through the **Jacobians** of $f_{\boldsymbol{\theta}}$.
This decides how we will differentiate training with respect to $\boldsymbol{\theta}$ later.

Stochastic trajectories: sampling the simulator

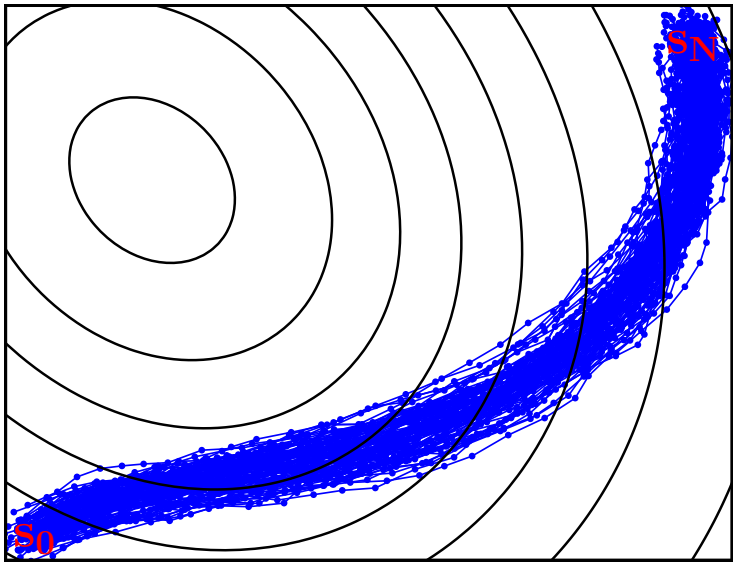


There are $(|\mathcal{A}||\mathcal{S}|)^N$ possible histories: we never enumerate them, we **sample** them.

n simulated closed-loop trajectories from s_0 to s_N:

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) \approx \frac{1}{n} \sum_{i=1}^{n} \sum_{k=0}^{N} \gamma^k L_{\boldsymbol{\theta}}(\mathrm{s}_k^i, \mathrm{a}_k^i)$$

Stochastic trajectories: sampling the simulator



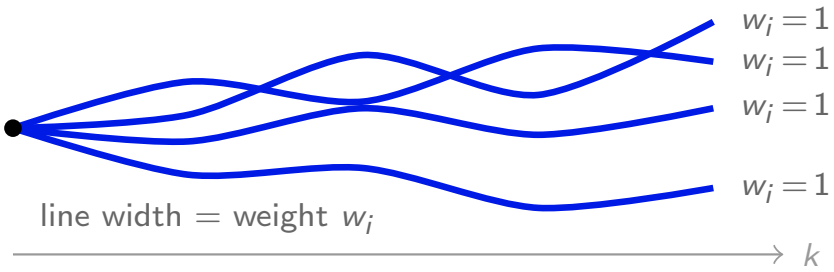
There are $(|\mathcal{A}||\mathcal{S}|)^N$ possible histories: we never enumerate them, we **sample** them.

n simulated closed-loop trajectories from s_0 to s_N:

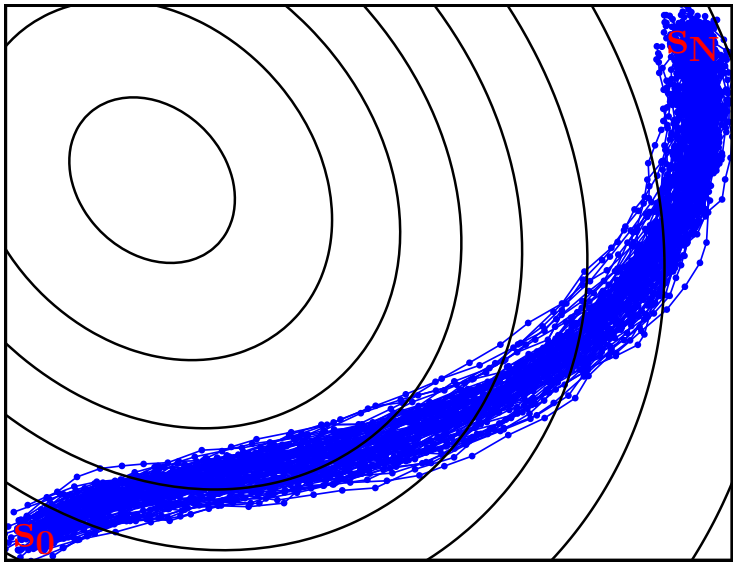
$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) \approx \frac{1}{n} \sum_{i=1}^{n} \sum_{k=0}^{N} \gamma^k L_{\boldsymbol{\theta}}(\mathrm{s}_k^i, \mathrm{a}_k^i)$$

Perturb $\boldsymbol{\theta} \to \boldsymbol{\theta} + \mathrm{d}\boldsymbol{\theta}$: what happens to the samples?

LR: same paths, **new weights**



Stochastic trajectories: sampling the simulator



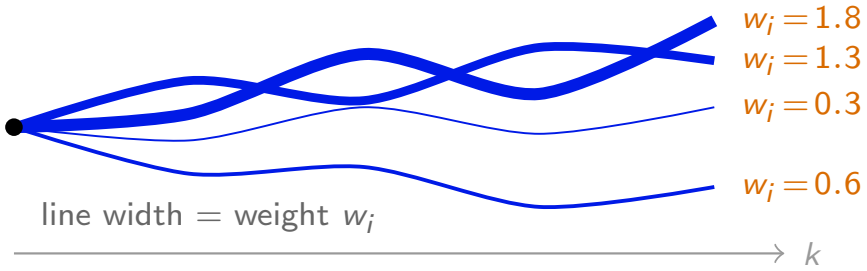
There are $(|\mathcal{A}||\mathcal{S}|)^N$ possible histories: we never enumerate them, we **sample** them.

n simulated closed-loop trajectories from s_0 to s_N:

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) \approx \frac{1}{n} \sum_{i=1}^{n} \sum_{k=0}^{N} \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k^i, \mathbf{a}_k^i)$$

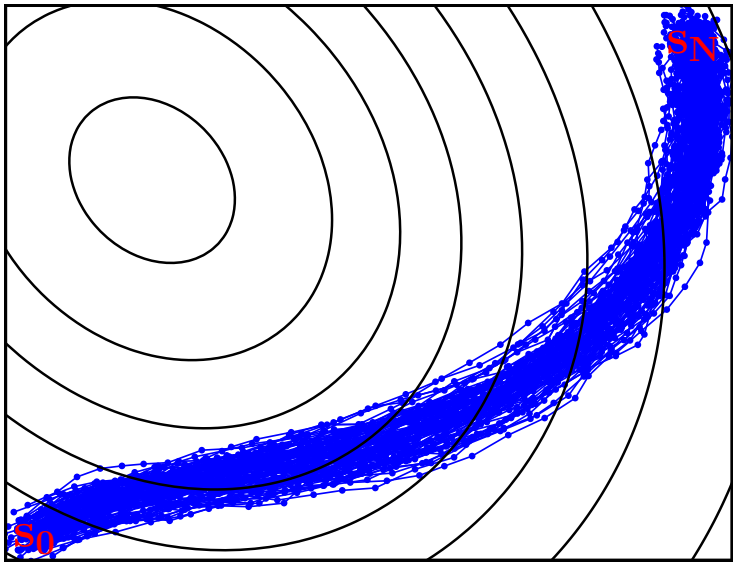
Perturb $\boldsymbol{\theta} \to \boldsymbol{\theta} + \mathrm{d}\boldsymbol{\theta}$: what happens to the samples?

LR: same paths, **new weights**



$$w_i = \frac{p_{\boldsymbol{\theta}+\mathrm{d}\boldsymbol{\theta}}(\tau_i)}{p_{\boldsymbol{\theta}}(\tau_i)} \approx 1 + \sum_k \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}^i | \mathbf{s}_k^i, \mathbf{a}_k^i)\, \mathrm{d}\boldsymbol{\theta}$$

Stochastic trajectories: sampling the simulator



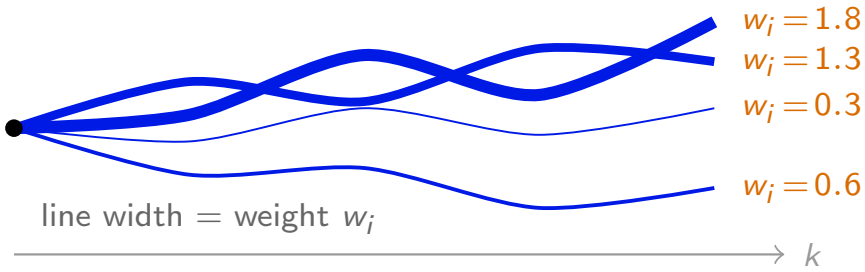
There are $(|\mathcal{A}||\mathcal{S}|)^N$ possible histories: we never enumerate them, we **sample** them.

n simulated closed-loop trajectories from s_0 to s_N:

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) \approx \frac{1}{n} \sum_{i=1}^{n} \sum_{k=0}^{N} \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k^i, \mathbf{a}_k^i)$$

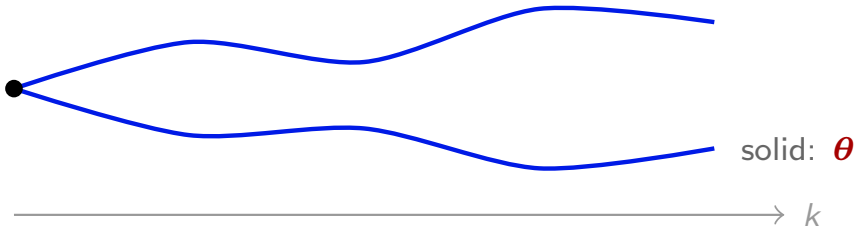
Perturb $\boldsymbol{\theta} \to \boldsymbol{\theta} + \mathrm{d}\boldsymbol{\theta}$: what happens to the samples?

LR: same paths, **new weights**

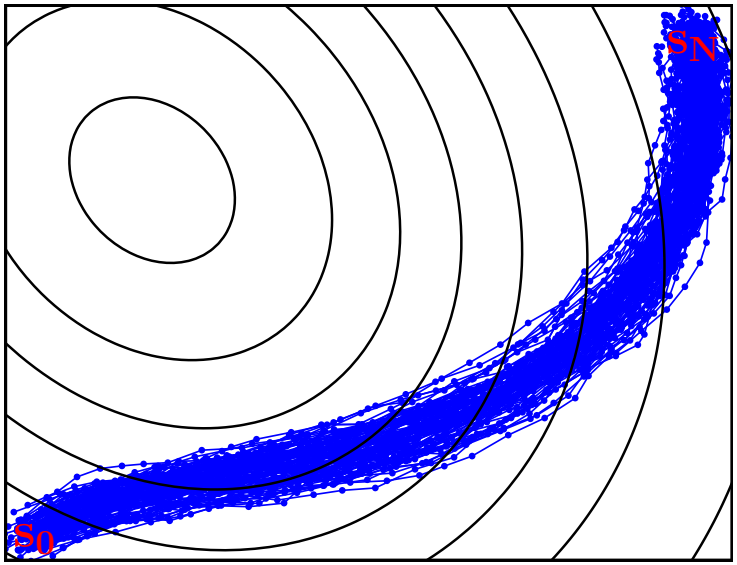


$$w_i = \frac{p_{\boldsymbol{\theta}+\mathrm{d}\boldsymbol{\theta}}(\tau_i)}{p_{\boldsymbol{\theta}}(\tau_i)} \approx 1 + \sum_k \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}^i | \mathbf{s}_k^i, \mathbf{a}_k^i)\, \mathrm{d}\boldsymbol{\theta}$$

PD: same noise ω_i, **moved paths**



Stochastic trajectories: sampling the simulator



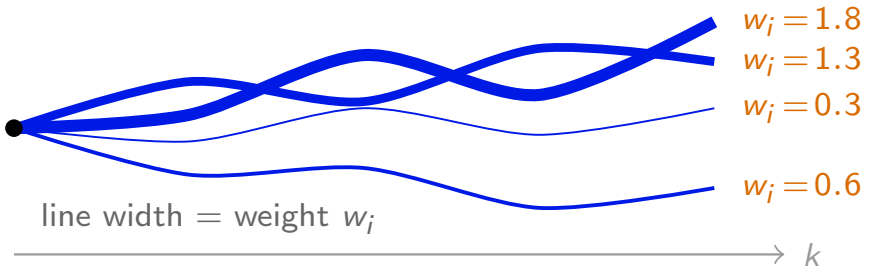
There are $(|\mathcal{A}||\mathcal{S}|)^N$ possible histories: we never enumerate them, we **sample** them.

n simulated closed-loop trajectories from s_0 to s_N:

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) \approx \frac{1}{n} \sum_{i=1}^{n} \sum_{k=0}^{N} \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k^i, \mathbf{a}_k^i)$$

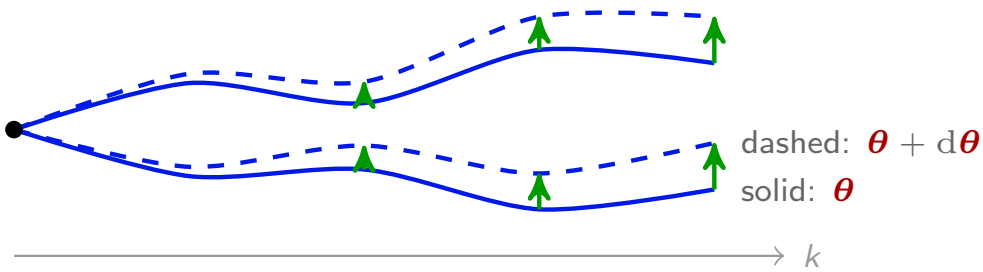
Perturb $\boldsymbol{\theta} \to \boldsymbol{\theta} + \mathrm{d}\boldsymbol{\theta}$: what happens to the samples?

LR: same paths, **new weights**



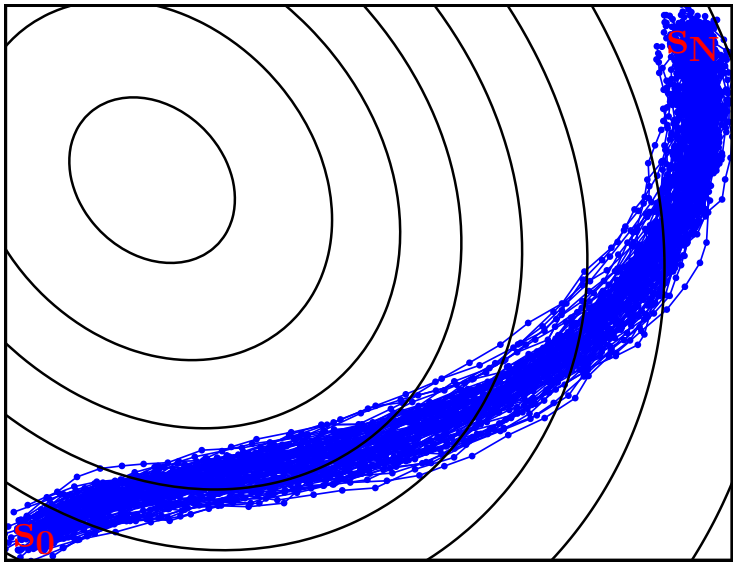
$$w_i = \frac{p_{\boldsymbol{\theta}+\mathrm{d}\boldsymbol{\theta}}(\tau_i)}{p_{\boldsymbol{\theta}}(\tau_i)} \approx 1 + \sum_k \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}^i | \mathbf{s}_k^i, \mathbf{a}_k^i)\, \mathrm{d}\boldsymbol{\theta}$$

PD: same noise ω_i, **moved paths**



$\mathbf{s}_k^i(\boldsymbol{\theta} + \mathrm{d}\boldsymbol{\theta}) \approx \mathbf{s}_k^i(\boldsymbol{\theta}) + S_k^i\, \mathrm{d}\boldsymbol{\theta}$, with $S_k = \mathrm{d}\mathbf{s}_k / \mathrm{d}\boldsymbol{\theta}$ (chain rule along the path)

Stochastic trajectories: sampling the simulator



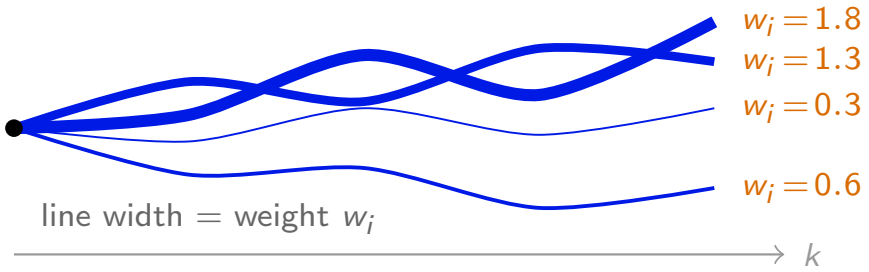
There are $(|\mathcal{A}||\mathcal{S}|)^N$ possible histories: we never enumerate them, we **sample** them.

n simulated closed-loop trajectories from s_0 to s_N:

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) \approx \frac{1}{n} \sum_{i=1}^{n} \sum_{k=0}^{N} \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k^i, \mathbf{a}_k^i)$$

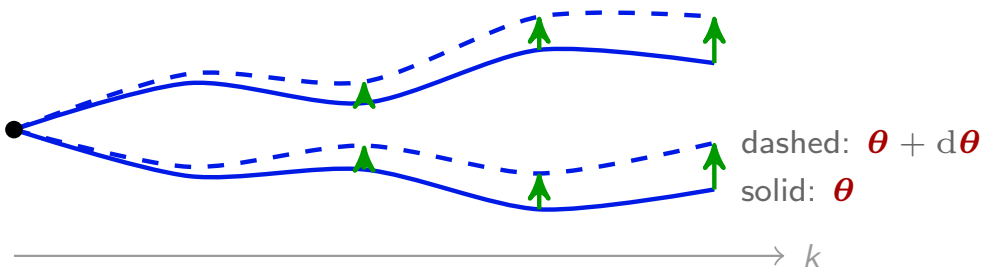
Perturb $\boldsymbol{\theta} \to \boldsymbol{\theta} + \mathrm{d}\boldsymbol{\theta}$: what happens to the samples?

LR: same paths, **new weights**



$$w_i = \frac{p_{\boldsymbol{\theta}+\mathrm{d}\boldsymbol{\theta}}(\tau_i)}{p_{\boldsymbol{\theta}}(\tau_i)} \approx 1 + \sum_k \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}^i | \mathbf{s}_k^i, \mathbf{a}_k^i)\, \mathrm{d}\boldsymbol{\theta}$$

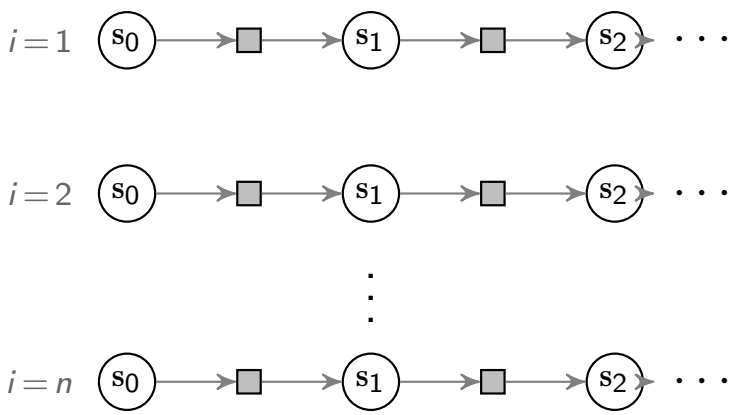
PD: same noise ω_i, **moved paths**



$\mathbf{s}_k^i(\boldsymbol{\theta} + \mathrm{d}\boldsymbol{\theta}) \approx \mathbf{s}_k^i(\boldsymbol{\theta}) + S_k^i\, \mathrm{d}\boldsymbol{\theta}$, with $S_k = \mathrm{d}\mathbf{s}_k / \mathrm{d}\boldsymbol{\theta}$ (chain rule along the path)

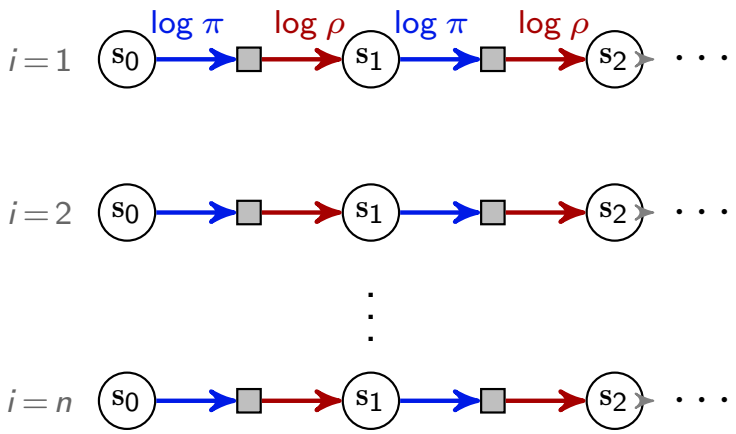
LR changes **how likely** each history is; **PD** changes **where** it goes

LR gradients: sample, score, average



1. Simulate n episodes with $\pi_{\boldsymbol{\nu}}$, keep the sampled $(\mathbf{s}_k, \mathbf{a}_k)$; no gradient flows through the simulator

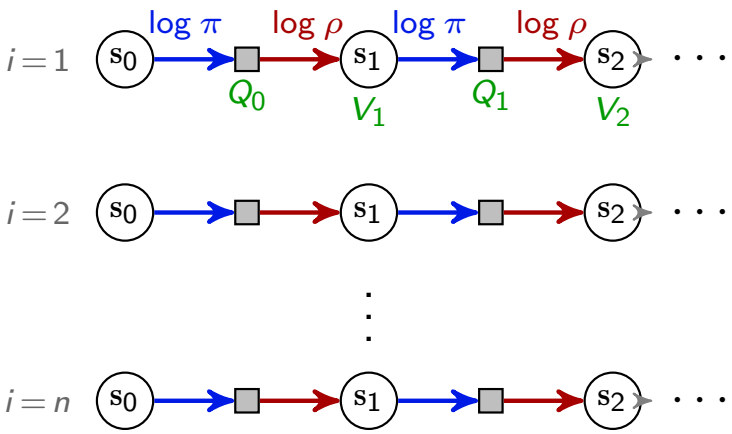
LR gradients: sample, score, average



1. **Simulate** n episodes with $\pi_{\boldsymbol{\nu}}$, keep the sampled $(\mathbf{s}_k, \mathbf{a}_k)$; no gradient flows through the simulator

2. **Score**: the log-probability of every sampled step, $\log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$ (holds $\boldsymbol{\nu}$) and $\log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$ (holds $\boldsymbol{\theta}$)

LR gradients: sample, score, average

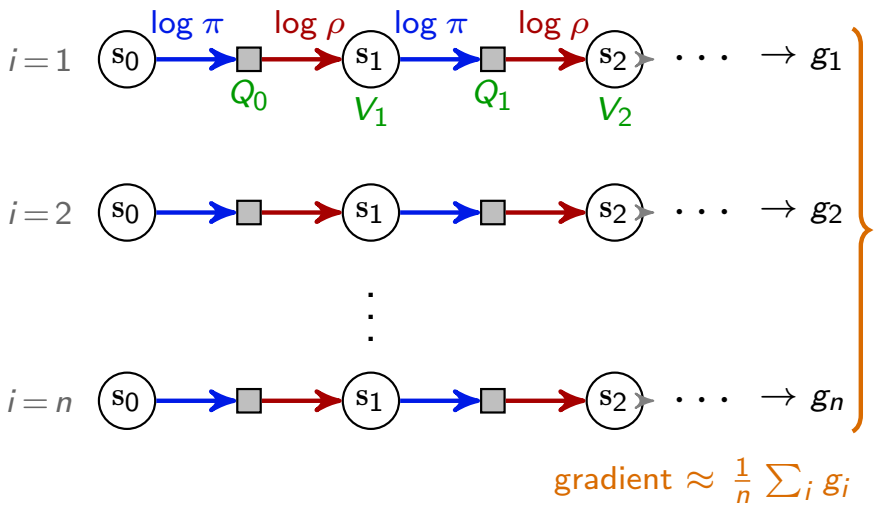


1. **Simulate** n episodes with $\pi_{\boldsymbol{\nu}}$, keep the sampled $(\mathbf{s}_k, \mathbf{a}_k)$; no gradient flows through the simulator

2. **Score**: the log-probability of every sampled step, $\log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$ (holds $\boldsymbol{\nu}$) and $\log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$ (holds $\boldsymbol{\theta}$)

3. **Weight** each log-probability by the cost that **follows** it: Q_k or V_{k+1} (a critic or the sampled return)

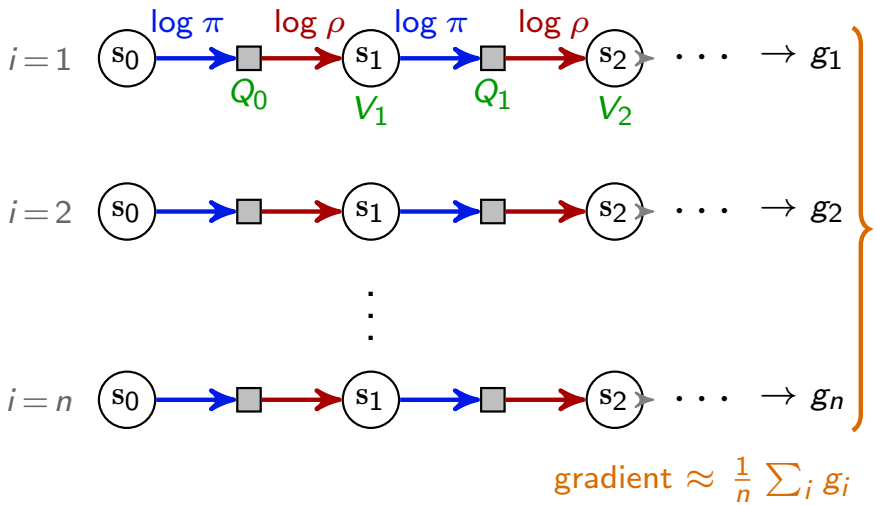
LR gradients: sample, score, average



1. **Simulate** n episodes with $\pi_{\boldsymbol{\nu}}$, keep the sampled $(\mathbf{s}_k, \mathbf{a}_k)$; no gradient flows through the simulator
2. **Score**: the log-probability of every sampled step, $\log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$ (holds $\boldsymbol{\nu}$) and $\log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$ (holds $\boldsymbol{\theta}$)
3. **Weight** each log-probability by the cost that **follows** it: Q_k or V_{k+1} (a critic or the sampled return)
4. **Average** over the episodes:

$$\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} \approx \frac{1}{n} \sum_{\text{episodes}} \sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, Q_k$$

LR gradients: sample, score, average

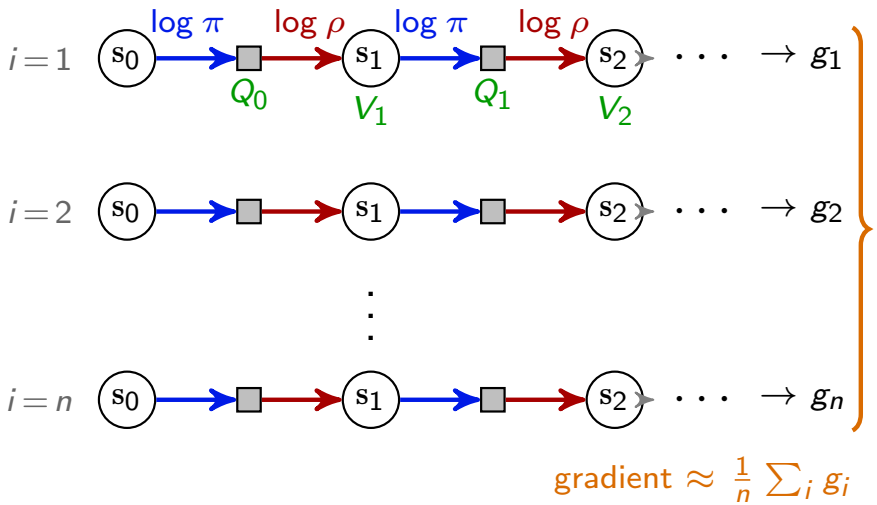


1. **Simulate** n episodes with $\pi_{\boldsymbol{\nu}}$, keep the sampled $(\mathbf{s}_k, \mathbf{a}_k)$; no gradient flows through the simulator
2. **Score**: the log-probability of every sampled step, $\log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$ (holds $\boldsymbol{\nu}$) and $\log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$ (holds $\boldsymbol{\theta}$)
3. **Weight** each log-probability by the cost that **follows** it: Q_k or V_{k+1} (a critic or the sampled return)
4. **Average** over the episodes:

$$\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} \approx \frac{1}{n} \sum_{\text{episodes}} \sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, Q_k$$

$$\nabla_{\boldsymbol{\theta}} \hat{J}_{\boldsymbol{\theta}} \approx \frac{1}{n} \sum_{\text{episodes}} \sum_k \gamma^k \left(\nabla_{\boldsymbol{\theta}} L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k) + \gamma\, \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)\, V_{k+1}\right)$$

LR gradients: sample, score, average



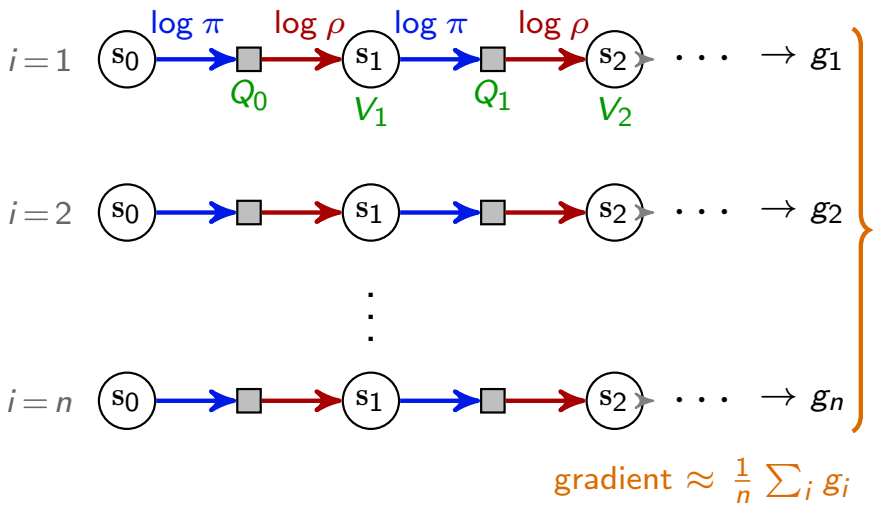
1. **Simulate** n episodes with $\pi_{\boldsymbol{\nu}}$, keep the sampled $(\mathbf{s}_k, \mathbf{a}_k)$; no gradient flows through the simulator
2. **Score**: the log-probability of every sampled step, $\log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$ (holds $\boldsymbol{\nu}$) and $\log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$ (holds $\boldsymbol{\theta}$)
3. **Weight** each log-probability by the cost that **follows** it: Q_k or V_{k+1} (a critic or the sampled return)
4. **Average** over the episodes:

$$\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} \approx \frac{1}{n} \sum_{\text{episodes}} \sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, Q_k$$

$$\nabla_{\boldsymbol{\theta}} \hat{J}_{\boldsymbol{\theta}} \approx \frac{1}{n} \sum_{\text{episodes}} \sum_k \gamma^k \left(\nabla_{\boldsymbol{\theta}} L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k) + \gamma\, \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)\, V_{k+1}\right)$$

In code (A2C, PPO): one backward pass on these sums without ∇, with Q_k, V_{k+1} `detached`

LR gradients: sample, score, average



1. **Simulate** n episodes with $\pi_{\boldsymbol{\nu}}$, keep the sampled $(\mathbf{s}_k, \mathbf{a}_k)$; no gradient flows through the simulator
2. **Score**: the log-probability of every sampled step, $\log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$ (holds $\boldsymbol{\nu}$) and $\log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$ (holds $\boldsymbol{\theta}$)
3. **Weight** each log-probability by the cost that **follows** it: Q_k or V_{k+1} (a critic or the sampled return)
4. **Average** over the episodes:

$$\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} \approx \frac{1}{n} \sum_{\text{episodes}} \sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, Q_k$$

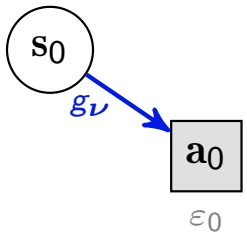
$$\nabla_{\boldsymbol{\theta}} \hat{J}_{\boldsymbol{\theta}} \approx \frac{1}{n} \sum_{\text{episodes}} \sum_k \gamma^k \left(\nabla_{\boldsymbol{\theta}} L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k) + \gamma\, \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)\, V_{k+1}\right)$$

In code (A2C, PPO): one backward pass on these sums without ∇, with Q_k, V_{k+1} `detached`

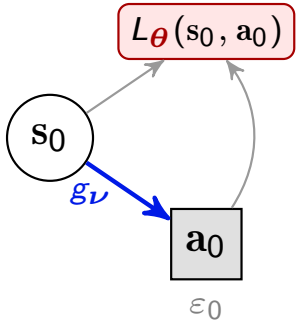
✓ no derivatives in $(\mathbf{s}, \mathbf{a})$: black-box simulator for $\nabla_{\boldsymbol{\nu}}$, non-smooth costs, discrete $\mathbf{s}, \mathbf{a}$

✘ $\nabla_{\boldsymbol{\theta}}$ needs the log-density $\log \rho_{\boldsymbol{\theta}}$ of every simulated step ✘ variance grows with k, Q, V

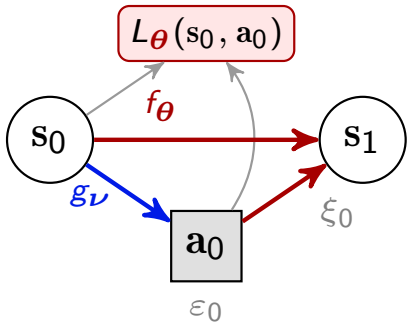
PD gradients: backpropagate through the simulator



PD gradients: backpropagate through the simulator

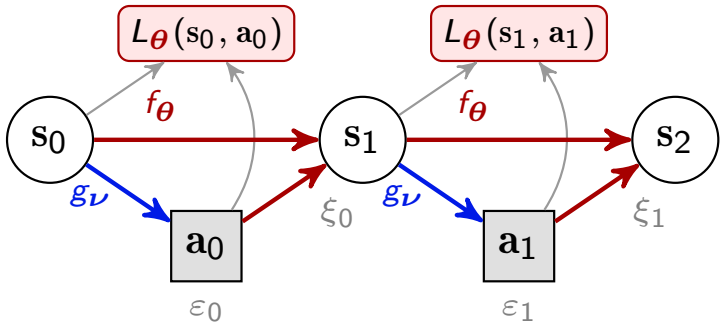


PD gradients: backpropagate through the simulator



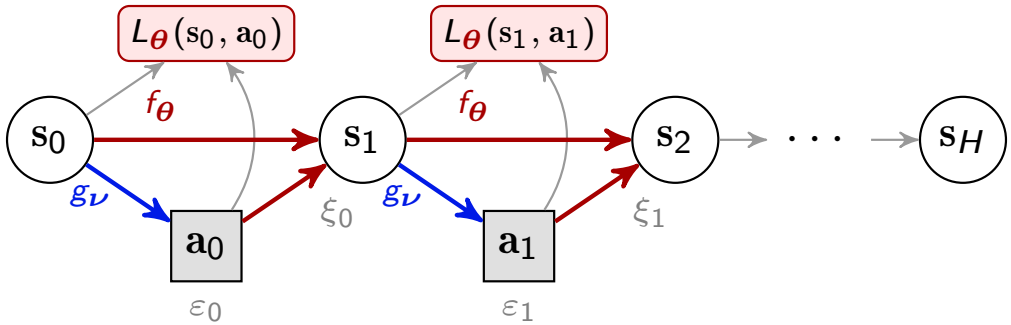
noise ε_k, ξ_k drawn once, then kept fixed: the path is a deterministic function of $(\boldsymbol{\nu}, \boldsymbol{\theta})$

PD gradients: backpropagate through the simulator



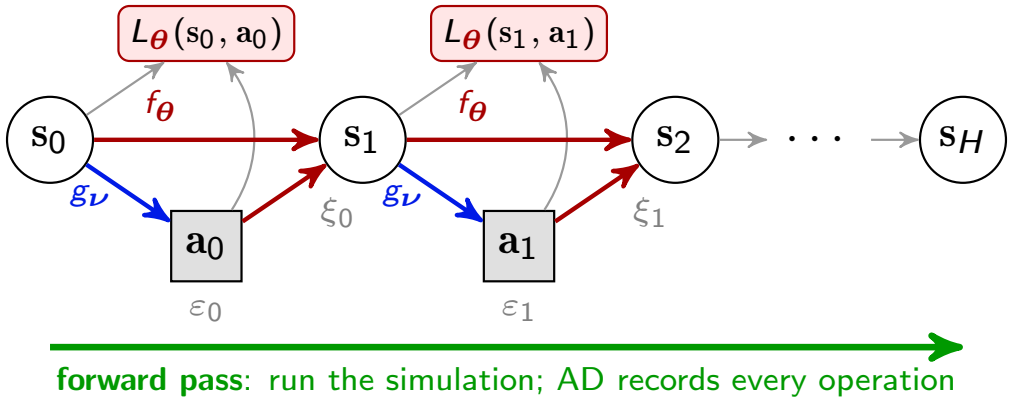
noise ε_k, ξ_k drawn once, then kept fixed: the path is a deterministic function of $(\boldsymbol{\nu}, \boldsymbol{\theta})$

PD gradients: backpropagate through the simulator



noise ε_k, ξ_k drawn once, then kept fixed: the path is a deterministic function of $(\boldsymbol{\nu}, \boldsymbol{\theta})$

PD gradients: backpropagate through the simulator



In practice (PyTorch, JAX)

draw the noise $\omega = (\varepsilon_k, \xi_k)_{k<H}$ once

forward: cost $J_\omega(\nu, \theta)$ of one rollout:

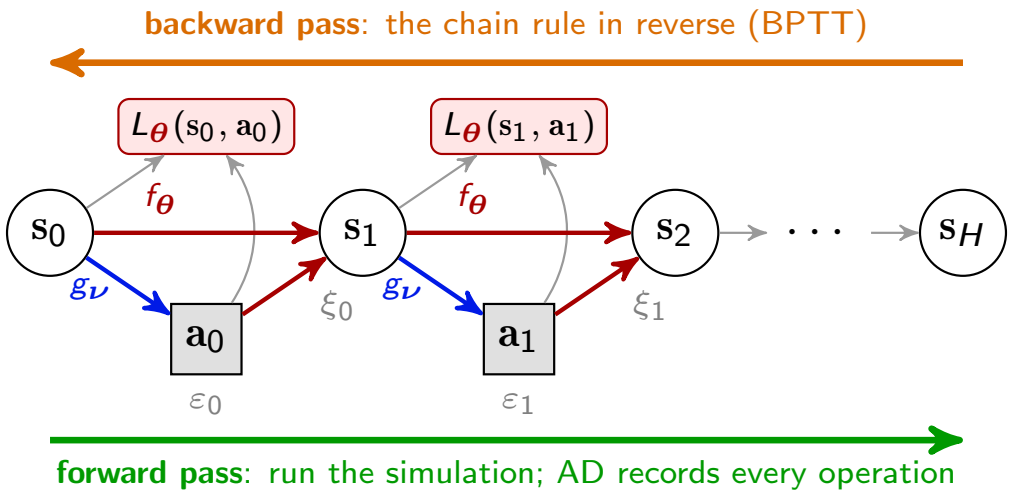
$\mathbf{s} \leftarrow \mathbf{s}_0$; for $k < H$:

$\mathbf{a} \leftarrow g_\nu(\mathbf{s}, \varepsilon_k)$, add $\gamma^k L_\theta(\mathbf{s}, \mathbf{a})$,

$\mathbf{s} \leftarrow f_\theta(\mathbf{s}, \mathbf{a}, \xi_k)$

- **forward** = the rollout itself; every call of g_ν, f_θ, L_θ is recorded

PD gradients: backpropagate through the simulator



In practice (PyTorch, JAX)

draw the noise $\omega = (\varepsilon_k, \xi_k)_{k<H}$ once

forward: cost $J_\omega(\nu, \theta)$ of one rollout:

$\mathbf{s} \leftarrow \mathbf{s}_0$; for $k < H$:

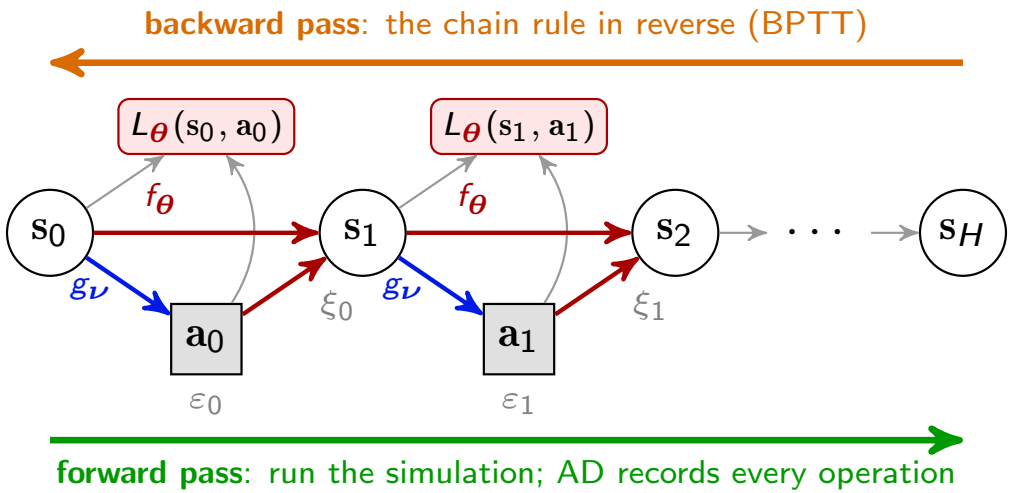
$\mathbf{a} \leftarrow g_\nu(\mathbf{s}, \varepsilon_k)$, add $\gamma^k L_\theta(\mathbf{s}, \mathbf{a})$,

$\mathbf{s} \leftarrow f_\theta(\mathbf{s}, \mathbf{a}, \xi_k)$

backward: $\left(\nabla_\nu J_\omega,\ \nabla_\theta J_\omega\right) = \texttt{grad}(J_\omega)$

- **forward** = the rollout itself; every call of g_ν, f_θ, L_θ is recorded
- **backward** = one reverse sweep through the recorded steps: ∇_ν and ∇_θ together, for any $\dim \nu, \dim \theta$, at the cost of about one more rollout

PD gradients: backpropagate through the simulator



In practice (PyTorch, JAX)

draw the noise $\omega = (\varepsilon_k, \xi_k)_{k<H}$ once

forward: cost $J_\omega(\nu, \theta)$ of one rollout:

$\mathbf{s} \leftarrow \mathbf{s}_0$; for $k < H$:

$\mathbf{a} \leftarrow g_\nu(\mathbf{s}, \varepsilon_k)$, add $\gamma^k L_\theta(\mathbf{s}, \mathbf{a})$,

$\mathbf{s} \leftarrow f_\theta(\mathbf{s}, \mathbf{a}, \xi_k)$

backward: $(\nabla_\nu J_\omega,\ \nabla_\theta J_\omega) = \mathtt{grad}(J_\omega)$

average over n noise draws ω_i $\Rightarrow$ $\nabla_\nu \hat{J}_\theta$, $\nabla_\theta \hat{J}_\theta$

- **forward** = the rollout itself; every call of g_ν, f_θ, L_θ is recorded
- **backward** = one reverse sweep through the recorded steps: ∇_ν and ∇_θ together, for any dim ν, dim θ, at the cost of about one more rollout
- it evaluates the **PD** fequation path by path with AD.

PD gradients: exploding variance and the terminal critic

What the backward pass multiplies: the effect of $\mathbf{a}_0$ on the state k steps later

$$\frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\mathbf{a}_0} = \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\mathbf{s}_{k-1}} \cdots \frac{\mathrm{d}\mathbf{s}_2}{\mathrm{d}\mathbf{s}_1} \frac{\partial \mathbf{s}_1}{\partial \mathbf{a}_0}$$

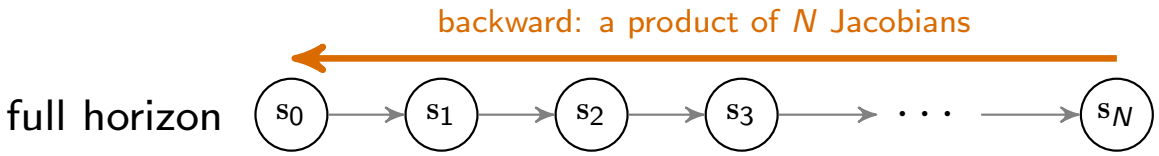
each factor = one closed-loop step through $f_{\boldsymbol{\theta}}$ and $g_{\boldsymbol{\nu}}$

PD gradients: exploding variance and the terminal critic

What the backward pass multiplies: the effect of $\mathbf{a}_0$ on the state k steps later

$$\frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\mathbf{a}_0} = \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\mathbf{s}_{k-1}} \cdots \frac{\mathrm{d}\mathbf{s}_2}{\mathrm{d}\mathbf{s}_1} \frac{\partial \mathbf{s}_1}{\partial \mathbf{a}_0}$$

each factor = one closed-loop step through $f_{\boldsymbol{\theta}}$ and $g_{\boldsymbol{\nu}}$

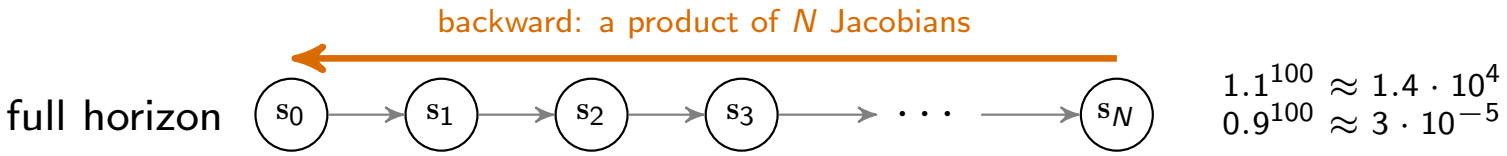


PD gradients: exploding variance and the terminal critic

What the backward pass multiplies: the effect of $\mathbf{a}_0$ on the state k steps later

$$\frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\mathbf{a}_0} = \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\mathbf{s}_{k-1}} \cdots \frac{\mathrm{d}\mathbf{s}_2}{\mathrm{d}\mathbf{s}_1} \frac{\partial \mathbf{s}_1}{\partial \mathbf{a}_0}$$

each factor = one closed-loop step through $f_{\boldsymbol{\theta}}$ and $g_{\boldsymbol{\nu}}$



$1.1^{100} \approx 1.4 \cdot 10^{4}$
$0.9^{100} \approx 3 \cdot 10^{-5}$

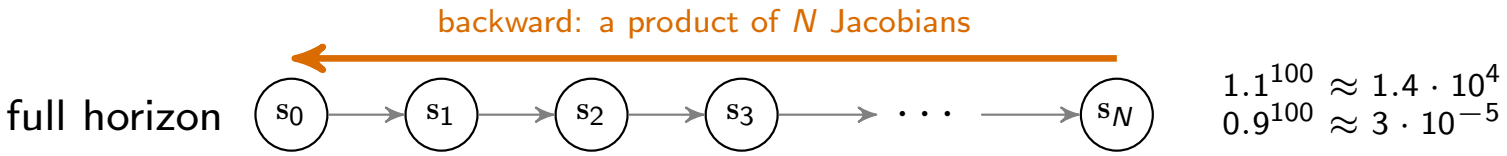
- **Catch**: for long horizons, contacts or chaotic dynamics the product **explodes or vanishes** and differs a lot between noise draws $\Rightarrow$ **high variance**

PD gradients: exploding variance and the terminal critic

What the backward pass multiplies: the effect of $\mathbf{a}_0$ on the state k steps later

$$\frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\mathbf{a}_0} = \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\mathbf{s}_{k-1}} \cdots \frac{\mathrm{d}\mathbf{s}_2}{\mathrm{d}\mathbf{s}_1} \frac{\partial \mathbf{s}_1}{\partial \mathbf{a}_0}$$

each factor = one closed-loop step through $f_{\boldsymbol{\theta}}$ and $g_{\boldsymbol{\nu}}$



$1.1^{100} \approx 1.4 \cdot 10^{4}$
$0.9^{100} \approx 3 \cdot 10^{-5}$

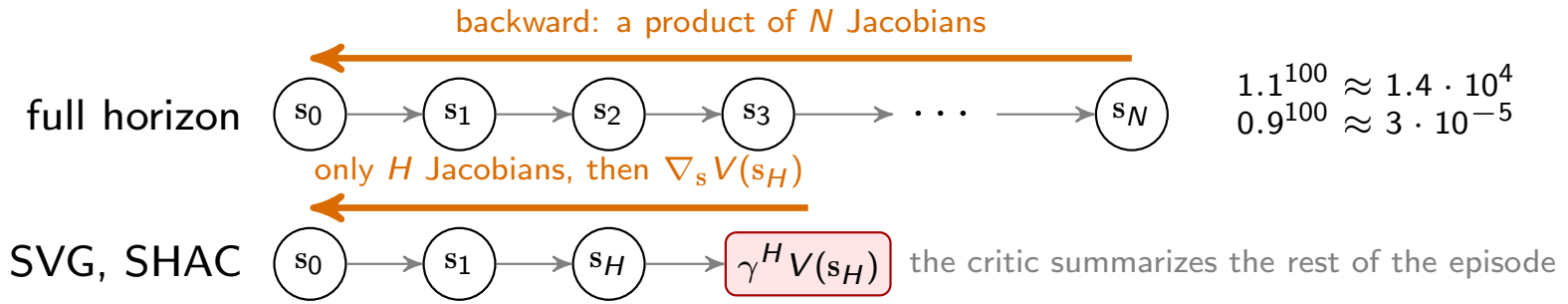
- **Catch**: for long horizons, contacts or chaotic dynamics the product **explodes or vanishes** and differs a lot between noise draws $\Rightarrow$ **high variance**
- **LR or PD?** Neither always has lower variance: **PD** wins on smooth, stable systems, **LR** on chaotic, stiff or contact-rich ones (Parmas et al., 2018; Suh et al., 2022)

PD gradients: exploding variance and the terminal critic

What the backward pass multiplies: the effect of $\mathbf{a}_0$ on the state k steps later

$$\frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\mathbf{a}_0} = \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\mathbf{s}_{k-1}} \cdots \frac{\mathrm{d}\mathbf{s}_2}{\mathrm{d}\mathbf{s}_1} \frac{\partial \mathbf{s}_1}{\partial \mathbf{a}_0}$$

each factor = one closed-loop step through $f_{\boldsymbol{\theta}}$ and $g_{\boldsymbol{\nu}}$



$1.1^{100} \approx 1.4 \cdot 10^{4}$
$0.9^{100} \approx 3 \cdot 10^{-5}$

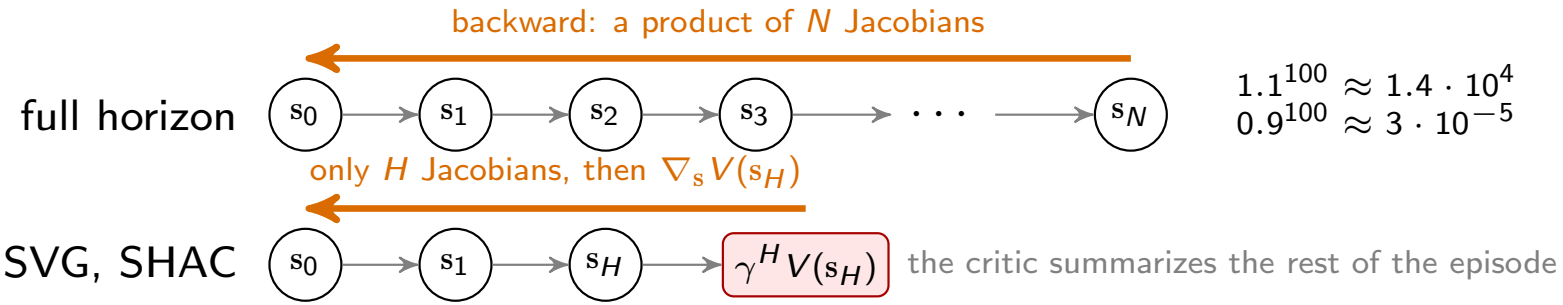
- **Catch**: for long horizons, contacts or chaotic dynamics the product **explodes or vanishes** and differs a lot between noise draws $\Rightarrow$ **high variance**
- **LR or PD?** Neither always has lower variance: **PD** wins on smooth, stable systems, **LR** on chaotic, stiff or contact-rich ones (Parmas et al., 2018; Suh et al., 2022)
- **Fix**: unroll only H steps and add a **terminal critic**: $J_{\omega,H} = \sum_{k<H} \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k) + \gamma^H V(\mathbf{s}_H)$

PD gradients: exploding variance and the terminal critic

What the backward pass multiplies: the effect of $\mathbf{a}_0$ on the state k steps later

$$\frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\mathbf{a}_0} = \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\mathbf{s}_{k-1}} \cdots \frac{\mathrm{d}\mathbf{s}_2}{\mathrm{d}\mathbf{s}_1} \frac{\partial \mathbf{s}_1}{\partial \mathbf{a}_0}$$

each factor = one closed-loop step through $f_{\boldsymbol{\theta}}$ and $g_{\boldsymbol{\nu}}$



$1.1^{100} \approx 1.4 \cdot 10^4$
$0.9^{100} \approx 3 \cdot 10^{-5}$

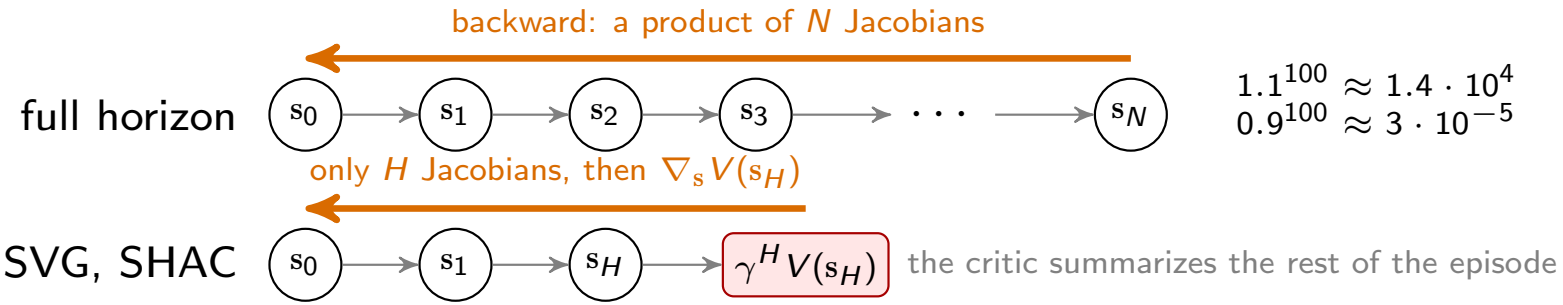
- **Catch**: for long horizons, contacts or chaotic dynamics the product **explodes or vanishes** and differs a lot between noise draws $\Rightarrow$ **high variance**
- **LR or PD?** Neither always has lower variance: **PD** wins on smooth, stable systems, **LR** on chaotic, stiff or contact-rich ones (Parmas et al., 2018; Suh et al., 2022)
- **Fix**: unroll only H steps and add a **terminal critic**: $J_{\omega,H} = \sum_{k<H} \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k) + \gamma^H V(\mathbf{s}_H)$
- short H: low variance but critic bias; SAC is case with no unroll step at all.

PD gradients: exploding variance and the terminal critic

What the backward pass multiplies: the effect of $\mathbf{a}_0$ on the state k steps later

$$\frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\mathbf{a}_0} = \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\mathbf{s}_{k-1}} \cdots \frac{\mathrm{d}\mathbf{s}_2}{\mathrm{d}\mathbf{s}_1} \frac{\partial \mathbf{s}_1}{\partial \mathbf{a}_0}$$

each factor = one closed-loop step through $f_{\boldsymbol{\theta}}$ and $g_{\boldsymbol{\nu}}$



$1.1^{100} \approx 1.4 \cdot 10^4$
$0.9^{100} \approx 3 \cdot 10^{-5}$

- **Catch**: for long horizons, contacts or chaotic dynamics the product **explodes or vanishes** and differs a lot between noise draws $\Rightarrow$ **high variance**
- **LR or PD?** Neither always has lower variance: **PD** wins on smooth, stable systems, **LR** on chaotic, stiff or contact-rich ones (Parmas et al., 2018; Suh et al., 2022)
- **Fix**: unroll only H steps and add a **terminal critic**: $J_{\omega,H} = \sum_{k<H} \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k) + \gamma^H V(\mathbf{s}_H)$
- short H: low variance but critic bias; SAC is case with no unroll step at all.

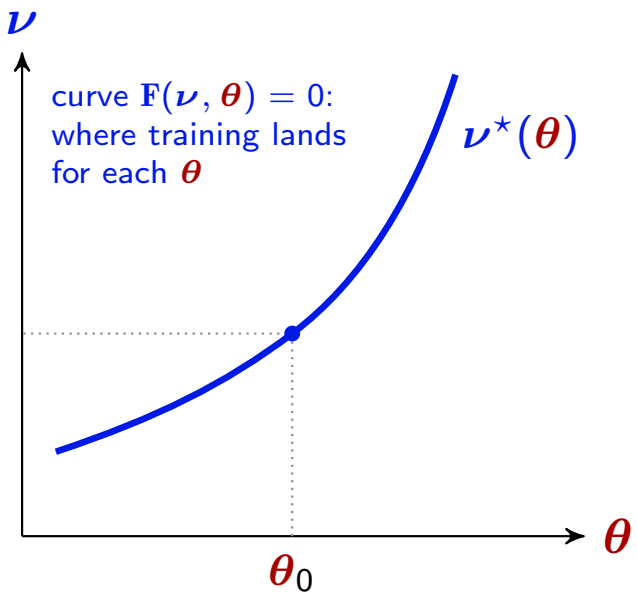
✓ exact per-path gradients; one backward pass for any dim $\boldsymbol{\nu}$, dim $\boldsymbol{\theta}$

✘ needs differentiable $f_{\boldsymbol{\theta}}, g_{\boldsymbol{\nu}}, L_{\boldsymbol{\theta}}$ (smoothed contact, min, max) and continuous $\mathbf{a}$, $\boldsymbol{\theta}$

Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$



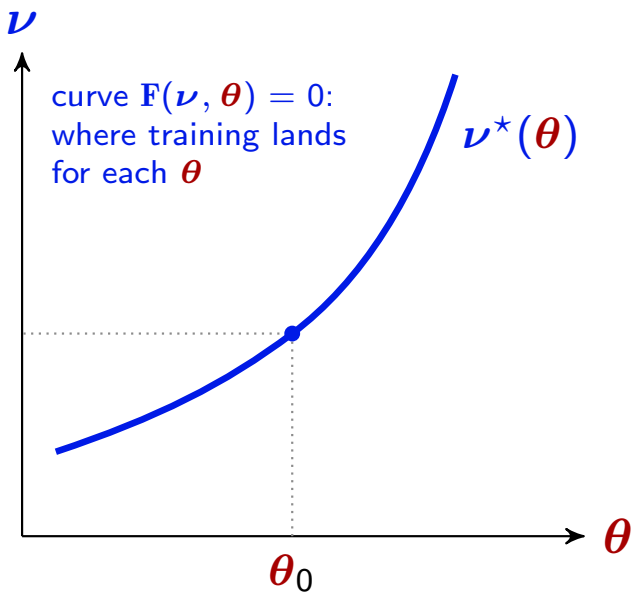
Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

LR: $\mathbb{E}\big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}\big] = 0$ (PPO)

PD: $\mathbb{E}_\omega\big[\sum_k \gamma^k \big(\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}}\, \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}}\, \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}\big)\big] = 0$ (SVG)



Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

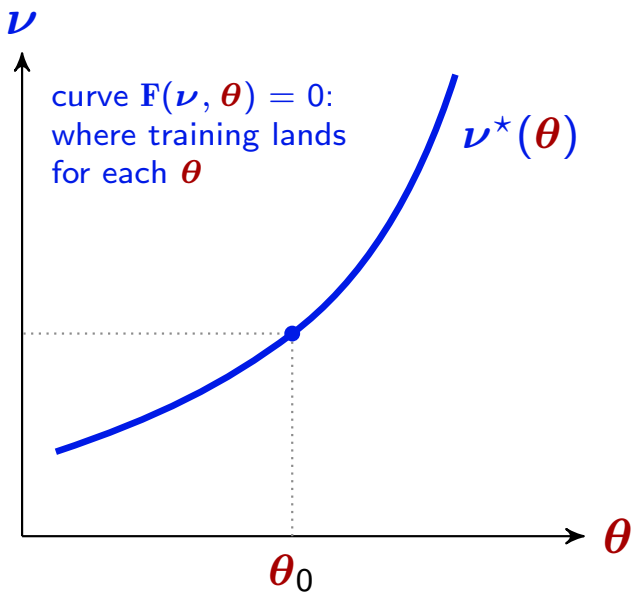
$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

LR: $\mathbb{E}\big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}} \, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}\big] = 0$ (PPO)

PD: $\mathbb{E}_\omega\big[\sum_k \gamma^k \big(\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}\big)\big] = 0$ (SVG)

Assumptions

1. **Smooth**: $\mathbf{F}$ is continuously differentiable in $(\boldsymbol{\nu}, \boldsymbol{\theta})$, e.g. $\pi_{\boldsymbol{\nu}}, \rho_{\boldsymbol{\theta}}$ (**LR**) or $g_{\boldsymbol{\nu}}, f_{\boldsymbol{\theta}}$ (**PD**) and $L_{\boldsymbol{\theta}}$ twice differentiable, with ∇ and $\mathbb{E}$ interchangeable



Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

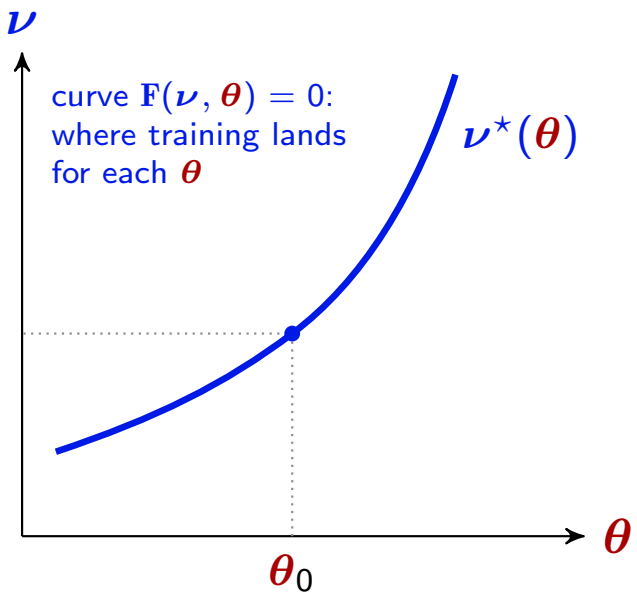
$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

LR: $\mathbb{E}\big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}\big] = 0$ (PPO)

PD: $\mathbb{E}_\omega\big[\sum_k \gamma^k \big(\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}\big)\big] = 0$ (SVG)

Assumptions

1. **Smooth**: $\mathbf{F}$ is continuously differentiable in $(\boldsymbol{\nu}, \boldsymbol{\theta})$, e.g. $\pi_{\boldsymbol{\nu}}, \rho_{\boldsymbol{\theta}}$ (**LR**) or $g_{\boldsymbol{\nu}}, f_{\boldsymbol{\theta}}$ (**PD**) and $L_{\boldsymbol{\theta}}$ twice differentiable, with ∇ and $\mathbb{E}$ interchangeable
2. **Converged**: training reached $\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) = 0$; a *local* optimum is enough



Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

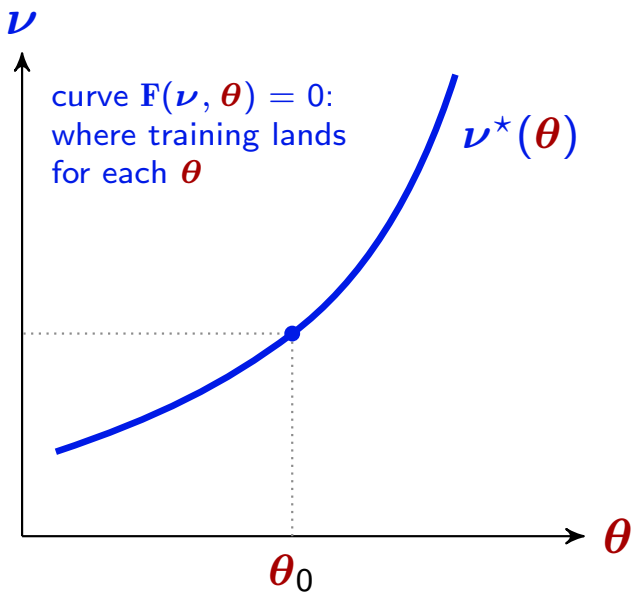
$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

LR: $\mathbb{E}\big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}\big] = 0$ (PPO)

PD: $\mathbb{E}_\omega\big[\sum_k \gamma^k \big(\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}\big)\big] = 0$ (SVG)

Assumptions

1. **Smooth**: $\mathbf{F}$ is continuously differentiable in $(\boldsymbol{\nu}, \boldsymbol{\theta})$, e.g. $\pi_{\boldsymbol{\nu}}, \rho_{\boldsymbol{\theta}}$ (**LR**) or $g_{\boldsymbol{\nu}}, f_{\boldsymbol{\theta}}$ (**PD**) and $L_{\boldsymbol{\theta}}$ twice differentiable, with ∇ and $\mathbb{E}$ interchangeable
2. **Converged**: training reached $\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) = 0$; a *local* optimum is enough
3. **Regular**: $\nabla_{\boldsymbol{\nu}} \mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta})$ is invertible (e.g. positive definite at a local minimum)



Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

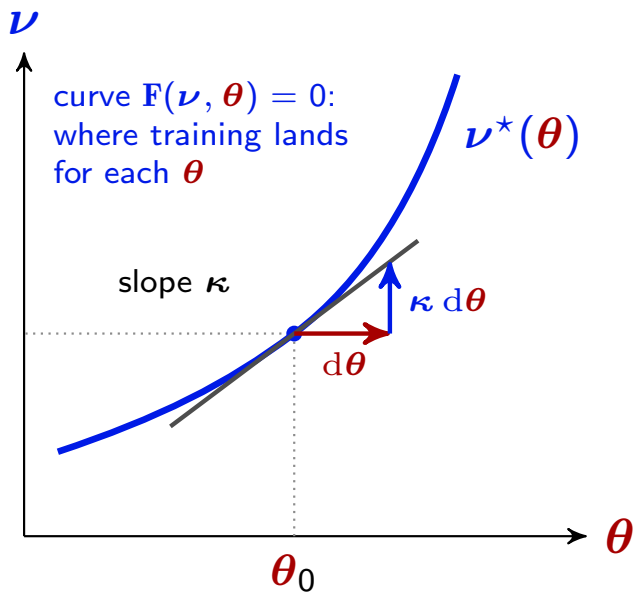
$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

LR: $\mathbb{E}\big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}\big] = 0$ (PPO)

PD: $\mathbb{E}_\omega\big[\sum_k \gamma^k \big(\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}\big)\big] = 0$ (SVG)

Assumptions

1. **Smooth**: $\mathbf{F}$ is continuously differentiable in $(\boldsymbol{\nu}, \boldsymbol{\theta})$, e.g. $\pi_{\boldsymbol{\nu}}, \rho_{\boldsymbol{\theta}}$ (**LR**) or $g_{\boldsymbol{\nu}}, f_{\boldsymbol{\theta}}$ (**PD**) and $L_{\boldsymbol{\theta}}$ twice differentiable, with ∇ and $\mathbb{E}$ interchangeable
2. **Converged**: training reached $\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) = 0$; a *local* optimum is enough
3. **Regular**: $\nabla_{\boldsymbol{\nu}} \mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta})$ is invertible (e.g. positive definite at a local minimum)



IFT: differentiate $\mathbf{F}\big(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\big) \equiv 0 \;\Rightarrow\; \nabla_{\boldsymbol{\nu}} \mathbf{F}\, \boldsymbol{\kappa} + \nabla_{\boldsymbol{\theta}} \mathbf{F} = 0 \;\Rightarrow\; \boxed{\boldsymbol{\kappa} = -(\nabla_{\boldsymbol{\nu}} \mathbf{F})^{-1} \nabla_{\boldsymbol{\theta}} \mathbf{F}}$

Computing $\nabla_{\boldsymbol{\nu}}\mathbf{F}$ and $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: PD (SVG)

$\mathbf{F} = \nabla_{\boldsymbol{\nu}}\hat{J}_{\boldsymbol{\theta}}$ is the policy gradient (a vector, the size of $\boldsymbol{\nu}$). Its derivatives are matrices:

- $\nabla_{\boldsymbol{\nu}}\mathbf{F}$: how $\mathbf{F}$ changes when the **policy** $\boldsymbol{\nu}$ changes (the Hessian of $\hat{J}_{\boldsymbol{\theta}}$)
- $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: how $\mathbf{F}$ changes when the **simulator** $\boldsymbol{\theta}$ changes

A. S. Anand et al., *Gradient-Based Co-Design of RL Policies with Differentiable Simulators*, TMLR 2026.

Computing $\nabla_{\boldsymbol{\nu}}\mathbf{F}$ and $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: PD (SVG)

$\mathbf{F} = \nabla_{\boldsymbol{\nu}}\hat{J}_{\boldsymbol{\theta}}$ is the policy gradient (a vector, the size of $\boldsymbol{\nu}$). Its derivatives are matrices:

- $\nabla_{\boldsymbol{\nu}}\mathbf{F}$: how $\mathbf{F}$ changes when the **policy** $\boldsymbol{\nu}$ changes (the Hessian of $\hat{J}_{\boldsymbol{\theta}}$)
- $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: how $\mathbf{F}$ changes when the **simulator** $\boldsymbol{\theta}$ changes

$\boldsymbol{\kappa}$ is only applied to vectors, so we need only **products**, never the matrices. A product is the gradient of a scalar (v fixed):

$$(\nabla_{\boldsymbol{\nu}}\mathbf{F})^\top v = \nabla_{\boldsymbol{\nu}}\left(\mathbf{F}^\top v\right), \qquad (\nabla_{\boldsymbol{\theta}}\mathbf{F})^\top v = \nabla_{\boldsymbol{\theta}}\left(\mathbf{F}^\top v\right)$$

A. S. Anand et al., *Gradient-Based Co-Design of RL Policies with Differentiable Simulators*, TMLR 2026.

Computing $\nabla_{\boldsymbol{\nu}}\mathbf{F}$ and $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: PD (SVG)

$\mathbf{F} = \nabla_{\boldsymbol{\nu}}\hat{J}_{\boldsymbol{\theta}}$ is the policy gradient (a vector, the size of $\boldsymbol{\nu}$). Its derivatives are matrices:

- $\nabla_{\boldsymbol{\nu}}\mathbf{F}$: how $\mathbf{F}$ changes when the **policy** $\boldsymbol{\nu}$ changes (the Hessian of $\hat{J}_{\boldsymbol{\theta}}$)
- $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: how $\mathbf{F}$ changes when the **simulator** $\boldsymbol{\theta}$ changes

$\boldsymbol{\kappa}$ is only applied to vectors, so we need only **products**, never the matrices. A product is the gradient of a scalar (v fixed):

$$(\nabla_{\boldsymbol{\nu}}\mathbf{F})^\top v = \nabla_{\boldsymbol{\nu}}\left(\mathbf{F}^\top v\right), \qquad (\nabla_{\boldsymbol{\theta}}\mathbf{F})^\top v = \nabla_{\boldsymbol{\theta}}\left(\mathbf{F}^\top v\right)$$

PD in code: three passes through the same rollout

1. **Forward**: fix the noise, simulate $\mathbf{a}_k = g_{\boldsymbol{\nu}}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$, add up the cost $J_\omega(\boldsymbol{\nu}, \boldsymbol{\theta})$

A. S. Anand et al., *Gradient-Based Co-Design of RL Policies with Differentiable Simulators*, TMLR 2026.

Computing $\nabla_{\boldsymbol{\nu}}\mathbf{F}$ and $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: PD (SVG)

$\mathbf{F} = \nabla_{\boldsymbol{\nu}}\hat{J}_{\boldsymbol{\theta}}$ is the policy gradient (a vector, the size of $\boldsymbol{\nu}$). Its derivatives are matrices:

- $\nabla_{\boldsymbol{\nu}}\mathbf{F}$: how $\mathbf{F}$ changes when the **policy** $\boldsymbol{\nu}$ changes (the Hessian of $\hat{J}_{\boldsymbol{\theta}}$)
- $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: how $\mathbf{F}$ changes when the **simulator** $\boldsymbol{\theta}$ changes

$\boldsymbol{\kappa}$ is only applied to vectors, so we need only **products**, never the matrices. A product is the gradient of a scalar (v fixed):

$$(\nabla_{\boldsymbol{\nu}}\mathbf{F})^\top v = \nabla_{\boldsymbol{\nu}}\left(\mathbf{F}^\top v\right), \qquad (\nabla_{\boldsymbol{\theta}}\mathbf{F})^\top v = \nabla_{\boldsymbol{\theta}}\left(\mathbf{F}^\top v\right)$$

PD in code: three passes through the same rollout

1. **Forward**: fix the noise, simulate $\mathbf{a}_k = g_{\boldsymbol{\nu}}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$, add up the cost $J_\omega(\boldsymbol{\nu}, \boldsymbol{\theta})$
2. **Backward**: $\mathbf{F} = \mathtt{grad}_{\boldsymbol{\nu}} J_\omega$, averaged over the noise draws; keep its AD graph (PyTorch: `create_graph=True`; JAX: nest `grad`)

A. S. Anand et al., *Gradient-Based Co-Design of RL Policies with Differentiable Simulators*, TMLR 2026.

Computing $\nabla_{\nu}\mathbf{F}$ and $\nabla_{\theta}\mathbf{F}$: PD (SVG)

$\mathbf{F} = \nabla_{\nu}\hat{J}_{\theta}$ is the policy gradient (a vector, the size of ν). Its derivatives are matrices:

- $\nabla_{\nu}\mathbf{F}$: how $\mathbf{F}$ changes when the **policy** ν changes (the Hessian of $\hat{J}_{\theta}$)
- $\nabla_{\theta}\mathbf{F}$: how $\mathbf{F}$ changes when the **simulator** θ changes

κ is only applied to vectors, so we need only **products**, never the matrices. A product is the gradient of a scalar (v fixed):

$$(\nabla_{\nu}\mathbf{F})^{\top} v = \nabla_{\nu}\left(\mathbf{F}^{\top} v\right), \qquad (\nabla_{\theta}\mathbf{F})^{\top} v = \nabla_{\theta}\left(\mathbf{F}^{\top} v\right)$$

PD in code: three passes through the same rollout

1. **Forward**: fix the noise, simulate $\mathbf{a}_k = g_{\nu}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\theta}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$, add up the cost $J_{\omega}(\nu, \theta)$
2. **Backward**: $\mathbf{F} = \mathtt{grad}_{\nu} J_{\omega}$, averaged over the noise draws; keep its AD graph (PyTorch: `create_graph=True`; JAX: nest `grad`)
3. **Second backward**: differentiate the scalar $\mathbf{F}^{\top} v$ again, w.r.t. ν and w.r.t. θ: the two products above

A. S. Anand et al., *Gradient-Based Co-Design of RL Policies with Differentiable Simulators*, TMLR 2026.

Computing $\nabla_{\nu}\mathbf{F}$ and $\nabla_{\theta}\mathbf{F}$: PD (SVG)

$\mathbf{F} = \nabla_{\nu}\hat{J}_{\theta}$ is the policy gradient (a vector, the size of ν). Its derivatives are matrices:

- $\nabla_{\nu}\mathbf{F}$: how $\mathbf{F}$ changes when the **policy** ν changes (the Hessian of $\hat{J}_{\theta}$)
- $\nabla_{\theta}\mathbf{F}$: how $\mathbf{F}$ changes when the **simulator** θ changes

κ is only applied to vectors, so we need only **products**, never the matrices. A product is the gradient of a scalar (v fixed):

$$(\nabla_{\nu}\mathbf{F})^{\top} v = \nabla_{\nu}\left(\mathbf{F}^{\top} v\right), \qquad (\nabla_{\theta}\mathbf{F})^{\top} v = \nabla_{\theta}\left(\mathbf{F}^{\top} v\right)$$

PD in code: three passes through the same rollout

1. **Forward**: fix the noise, simulate $\mathbf{a}_k = g_{\nu}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\theta}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$, add up the cost $J_{\omega}(\nu, \theta)$
2. **Backward**: $\mathbf{F} = \mathtt{grad}_{\nu} J_{\omega}$, averaged over the noise draws; keep its AD graph (PyTorch: `create_graph=True`; JAX: nest `grad`)
3. **Second backward**: differentiate the scalar $\mathbf{F}^{\top} v$ again, w.r.t. ν and w.r.t. θ: the two products above

With the noise fixed, the whole rollout is a function of (ν, θ); the second pass differentiates it again, so all second derivatives of g_{ν}, f_{θ}, L_{θ} (in ν, in θ, mixed) are included

A. S. Anand et al., *Gradient-Based Co-Design of RL Policies with Differentiable Simulators*, TMLR 2026.

Computing $\nabla_{\boldsymbol{\nu}} \mathbf{F}$ and $\nabla_{\boldsymbol{\theta}} \mathbf{F}$: LR (PPO, A2C)

The **LR** policy gradient, with $\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k | \mathbf{s}_k)$ and Q_k the cost-to-go:

$$\mathbf{F}(\boldsymbol{\nu}, \boldsymbol{\theta}) = \mathbb{E}_{\tau \sim p_{\boldsymbol{\nu}, \boldsymbol{\theta}}} \left[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k \right]$$

Computing $\nabla_{\boldsymbol{\nu}}\mathbf{F}$ and $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: LR (PPO, A2C)

The **LR** policy gradient, with $\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$ and Q_k the cost-to-go:

$$\mathbf{F}(\boldsymbol{\nu}, \boldsymbol{\theta}) = \mathbb{E}_{\tau \sim p_{\boldsymbol{\nu},\boldsymbol{\theta}}}\left[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k\right]$$

$(\boldsymbol{\nu}, \boldsymbol{\theta})$ enter in **three** places: $\nabla_{\boldsymbol{\nu}} \log \pi_k$, Q_k and the sampling $p_{\boldsymbol{\nu},\boldsymbol{\theta}}$. $x = \boldsymbol{\nu}$ or $x = \boldsymbol{\theta}$:

$$\nabla_x \mathbf{F} = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla_x \nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k}_{\text{1. policy curvature}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \; \nabla_x Q_k^\top}_{\text{2. critic sensitivity}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k \; \nabla_x \log p(\tau_{\leq k})^\top}_{\text{3. sampling}} \Big)\Big]$$

$p(\tau_{\leq k})$: probability of the sampled history up to step k

Computing $\nabla_{\boldsymbol{\nu}}\mathbf{F}$ and $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: LR (PPO, A2C)

The **LR** policy gradient, with $\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$ and Q_k the cost-to-go:

$$\mathbf{F}(\boldsymbol{\nu},\boldsymbol{\theta}) = \mathbb{E}_{\tau\sim p_{\boldsymbol{\nu},\boldsymbol{\theta}}}\Big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k\Big]$$

$(\boldsymbol{\nu},\boldsymbol{\theta})$ enter in **three** places: $\nabla_{\boldsymbol{\nu}} \log \pi_k$, Q_k and the sampling $p_{\boldsymbol{\nu},\boldsymbol{\theta}}$. $x = \boldsymbol{\nu}$ or $x = \boldsymbol{\theta}$:

$$\nabla_x \mathbf{F} = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla_x \nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k}_{\text{1. policy curvature}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \; \nabla_x Q_k^\top}_{\text{2. critic sensitivity}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k \; \nabla_x \log p(\tau_{\leq k})^\top}_{\text{3. sampling}} \Big)\Big]$$

$p(\tau_{\leq k})$: probability of the sampled history up to step k

	in $\nabla_{\boldsymbol{\nu}}\mathbf{F}$	in $\nabla_{\boldsymbol{\theta}}\mathbf{F}$	computed by
1. policy curvature	$\nabla^2_{\boldsymbol{\nu}} \log \pi_k$	0	one more backward pass

Computing $\nabla_{\boldsymbol{\nu}}\mathbf{F}$ and $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: LR (PPO, A2C)

The **LR** policy gradient, with $\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$ and Q_k the cost-to-go:

$$\mathbf{F}(\boldsymbol{\nu},\boldsymbol{\theta}) = \mathbb{E}_{\tau\sim p_{\boldsymbol{\nu},\boldsymbol{\theta}}}\Big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k\Big]$$

$(\boldsymbol{\nu},\boldsymbol{\theta})$ enter in **three** places: $\nabla_{\boldsymbol{\nu}} \log \pi_k$, Q_k and the sampling $p_{\boldsymbol{\nu},\boldsymbol{\theta}}$. $x = \boldsymbol{\nu}$ or $x = \boldsymbol{\theta}$:

$$\nabla_x \mathbf{F} = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla_x \nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k}_{\text{1. policy curvature}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \; \nabla_x Q_k^\top}_{\text{2. critic sensitivity}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k \; \nabla_x \log p(\tau_{\leq k})^\top}_{\text{3. sampling}} \Big)\Big]$$

$p(\tau_{\leq k})$: probability of the sampled history up to step k

	in $\nabla_{\boldsymbol{\nu}}\mathbf{F}$	in $\nabla_{\boldsymbol{\theta}}\mathbf{F}$	computed by
1. policy curvature	$\nabla^2_{\boldsymbol{\nu}} \log \pi_k$	0	one more backward pass
2. critic sensitivity	$\nabla_{\boldsymbol{\nu}} Q_k$	$\nabla_{\boldsymbol{\theta}} Q_k$	differentiated Bellman eq.

Computing $\nabla_{\boldsymbol{\nu}}\mathbf{F}$ and $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: LR (PPO, A2C)

The **LR** policy gradient, with $\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$ and Q_k the cost-to-go:

$$\mathbf{F}(\boldsymbol{\nu},\boldsymbol{\theta}) = \mathbb{E}_{\tau\sim p_{\boldsymbol{\nu},\boldsymbol{\theta}}}\Big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k\Big]$$

$(\boldsymbol{\nu},\boldsymbol{\theta})$ enter in **three** places: $\nabla_{\boldsymbol{\nu}} \log \pi_k$, Q_k and the sampling $p_{\boldsymbol{\nu},\boldsymbol{\theta}}$. $x = \boldsymbol{\nu}$ or $x = \boldsymbol{\theta}$:

$$\nabla_x \mathbf{F} = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla_x \nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k}_{\text{1. policy curvature}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \; \nabla_x Q_k^\top}_{\text{2. critic sensitivity}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k \; \nabla_x \log p(\tau_{\leq k})^\top}_{\text{3. sampling}} \Big)\Big]$$

$p(\tau_{\leq k})$: probability of the sampled history up to step k

	in $\nabla_{\boldsymbol{\nu}}\mathbf{F}$	in $\nabla_{\boldsymbol{\theta}}\mathbf{F}$	computed by
1. policy curvature	$\nabla^2_{\boldsymbol{\nu}} \log \pi_k$	0	one more backward pass
2. critic sensitivity	$\nabla_{\boldsymbol{\nu}} Q_k$	$\nabla_{\boldsymbol{\theta}} Q_k$	differentiated Bellman eq.
3. sampling	$\sum_{j\leq k} \nabla_{\boldsymbol{\nu}} \log \pi_j$	$\sum_{j<k} \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{j+1}\|\mathbf{s}_j,\mathbf{a}_j)$	sampled log-probabilities

Computing $\nabla_{\boldsymbol{\nu}}\mathbf{F}$ and $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: LR (PPO, A2C)

The **LR** policy gradient, with $\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$ and Q_k the cost-to-go:

$$\mathbf{F}(\boldsymbol{\nu}, \boldsymbol{\theta}) = \mathbb{E}_{\tau \sim p_{\boldsymbol{\nu},\boldsymbol{\theta}}}\Big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k\Big]$$

$(\boldsymbol{\nu}, \boldsymbol{\theta})$ enter in **three** places: $\nabla_{\boldsymbol{\nu}} \log \pi_k$, Q_k and the sampling $p_{\boldsymbol{\nu},\boldsymbol{\theta}}$. $x = \boldsymbol{\nu}$ or $x = \boldsymbol{\theta}$:

$$\nabla_x \mathbf{F} = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla_x \nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k}_{\text{1. policy curvature}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \; \nabla_x Q_k^\top}_{\text{2. critic sensitivity}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k \; \nabla_x \log p(\tau_{\leq k})^\top}_{\text{3. sampling}} \Big)\Big]$$

$p(\tau_{\leq k})$: probability of the sampled history up to step k

	in $\nabla_{\boldsymbol{\nu}}\mathbf{F}$	in $\nabla_{\boldsymbol{\theta}}\mathbf{F}$	computed by
1. policy curvature	$\nabla^2_{\boldsymbol{\nu}} \log \pi_k$	0	one more backward pass
2. critic sensitivity	$\nabla_{\boldsymbol{\nu}} Q_k$	$\nabla_{\boldsymbol{\theta}} Q_k$	differentiated Bellman eq.
3. sampling	$\sum_{j \leq k} \nabla_{\boldsymbol{\nu}} \log \pi_j$	$\sum_{j<k} \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{j+1}\|\mathbf{s}_j, \mathbf{a}_j)$	sampled log-probabilities

- Unlike **PD**, AD gives **only term 1** (policy curvature)
  - The samples are fixed data; AD misses that $(\boldsymbol{\nu}, \boldsymbol{\theta})$ change **which** ones are drawn

Computing $\nabla_{\boldsymbol{\nu}}\mathbf{F}$ and $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: LR (PPO, A2C)

The **LR** policy gradient, with $\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$ and Q_k the cost-to-go:

$$\mathbf{F}(\boldsymbol{\nu},\boldsymbol{\theta}) = \mathbb{E}_{\tau\sim p_{\boldsymbol{\nu},\boldsymbol{\theta}}}\Big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k\Big]$$

$(\boldsymbol{\nu},\boldsymbol{\theta})$ enter in **three** places: $\nabla_{\boldsymbol{\nu}} \log \pi_k$, Q_k and the sampling $p_{\boldsymbol{\nu},\boldsymbol{\theta}}$. $x = \boldsymbol{\nu}$ or $x = \boldsymbol{\theta}$:

$$\nabla_x \mathbf{F} = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla_x \nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k}_{\text{1. policy curvature}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \; \nabla_x Q_k^\top}_{\text{2. critic sensitivity}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \; Q_k \; \nabla_x \log p(\tau_{\le k})^\top}_{\text{3. sampling}} \Big)\Big]$$

$p(\tau_{\le k})$: probability of the sampled history up to step k

	in $\nabla_{\boldsymbol{\nu}}\mathbf{F}$	in $\nabla_{\boldsymbol{\theta}}\mathbf{F}$	computed by
1. policy curvature	$\nabla^2_{\boldsymbol{\nu}} \log \pi_k$	0	one more backward pass
2. critic sensitivity	$\nabla_{\boldsymbol{\nu}} Q_k$	$\nabla_{\boldsymbol{\theta}} Q_k$	differentiated Bellman eq.
3. sampling	$\sum_{j\le k} \nabla_{\boldsymbol{\nu}} \log \pi_j$	$\sum_{j<k} \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{j+1}\|\mathbf{s}_j,\mathbf{a}_j)$	sampled log-probabilities

- Unlike **PD**, AD gives **only term 1** (policy curvature)
  - The samples are fixed data; AD misses that $(\boldsymbol{\nu},\boldsymbol{\theta})$ change **which** ones are drawn
- Terms 2 and 3 are **estimated separately**; term 3 needs the simulator density $\rho_{\boldsymbol{\theta}}$

Computing $\nabla_{\boldsymbol{\nu}}\mathbf{F}$ and $\nabla_{\boldsymbol{\theta}}\mathbf{F}$: LR (PPO, A2C)

The **LR** policy gradient, with $\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$ and Q_k the cost-to-go:

$$\mathbf{F}(\boldsymbol{\nu},\boldsymbol{\theta}) = \mathbb{E}_{\tau\sim p_{\boldsymbol{\nu},\boldsymbol{\theta}}}\Big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log\pi_k \; Q_k\Big]$$

$(\boldsymbol{\nu},\boldsymbol{\theta})$ enter in **three** places: $\nabla_{\boldsymbol{\nu}}\log\pi_k$, Q_k and the sampling $p_{\boldsymbol{\nu},\boldsymbol{\theta}}$. $x = \boldsymbol{\nu}$ or $x = \boldsymbol{\theta}$:

$$\nabla_x\mathbf{F} = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla_x\nabla_{\boldsymbol{\nu}}\log\pi_k\, Q_k}_{\text{1. policy curvature}} + \underbrace{\nabla_{\boldsymbol{\nu}}\log\pi_k\,\nabla_x Q_k^\top}_{\text{2. critic sensitivity}} + \underbrace{\nabla_{\boldsymbol{\nu}}\log\pi_k\, Q_k\,\nabla_x \log p(\tau_{\le k})^\top}_{\text{3. sampling}}\Big)\Big]$$

$p(\tau_{\le k})$: probability of the sampled history up to step k

	in $\nabla_{\boldsymbol{\nu}}\mathbf{F}$	in $\nabla_{\boldsymbol{\theta}}\mathbf{F}$	computed by
1. policy curvature	$\nabla^2_{\boldsymbol{\nu}}\log\pi_k$	0	one more backward pass
2. critic sensitivity	$\nabla_{\boldsymbol{\nu}} Q_k$	$\nabla_{\boldsymbol{\theta}} Q_k$	differentiated Bellman eq.
3. sampling	$\sum_{j\le k}\nabla_{\boldsymbol{\nu}}\log\pi_j$	$\sum_{j<k}\nabla_{\boldsymbol{\theta}}\log\rho_{\boldsymbol{\theta}}(\mathbf{s}_{j+1}\|\mathbf{s}_j,\mathbf{a}_j)$	sampled log-probabilities

- Unlike **PD**, AD gives **only term 1** (policy curvature)
  - The samples are fixed data; AD misses that $(\boldsymbol{\nu},\boldsymbol{\theta})$ change **which** ones are drawn
- Terms 2 and 3 are **estimated separately**; term 3 needs the simulator density $\rho_{\boldsymbol{\theta}}$
- **Critic sensitivity: difficult, but possible**
  - **Main issue**: the critic network $Q_w(\mathbf{s},\mathbf{a})$ has no input $\boldsymbol{\nu}$, $\boldsymbol{\theta}$, so AD cannot give ∇Q_k
  - Fix: differentiating $Q_k = L_{\boldsymbol{\theta}} + \gamma\,\mathbb{E}[Q_{k+1}]$ gives $\nabla Q_k = c_k + \gamma\,\mathbb{E}[\nabla Q_{k+1}]$: the **same Bellman structure** with a new cost c_k, so ∇Q_k is learned like Q_k

Where do we use κ? (1) Sim-to-real: tune the simulator

Motivation: a policy is trained in a simulator, then deployed on a real system.

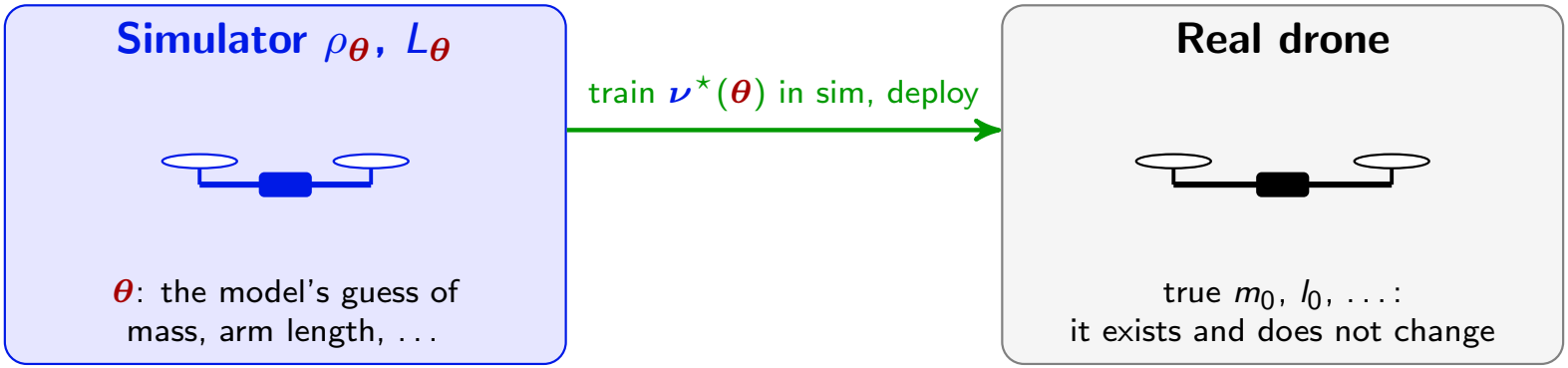
Simulator ρ_{θ}, L_{θ}



θ: the model's guess of mass, arm length, ...

Where do we use κ? (1) Sim-to-real: tune the simulator

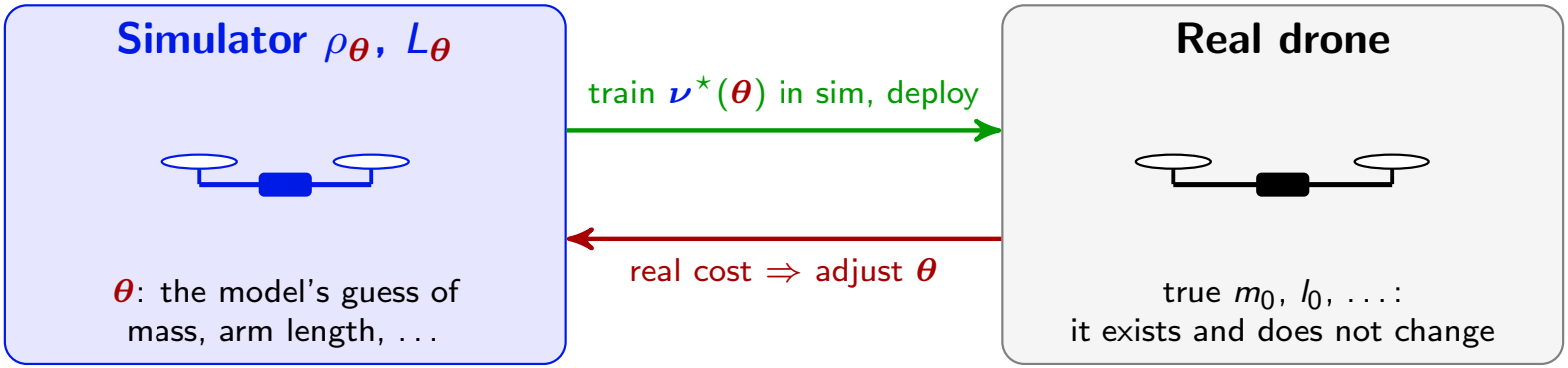
Motivation: a policy is trained in a simulator, then deployed on a real system.



- Changing θ changes the **simulator**, our model of it

Where do we use κ? (1) Sim-to-real: tune the simulator

Motivation: a policy is trained in a simulator, then deployed on a real system.

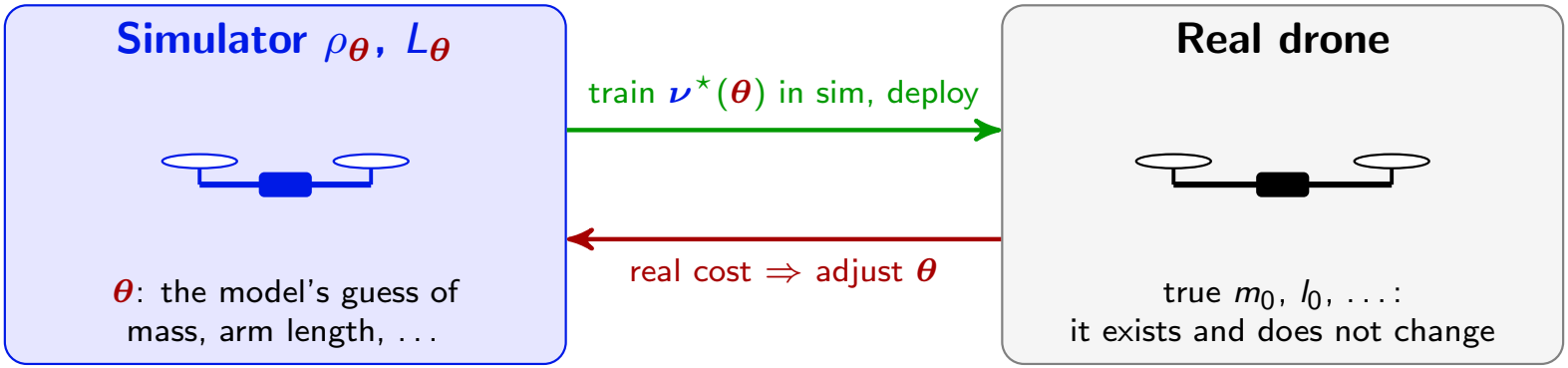


- Changing θ changes the **simulator**, our model of it
- Goal: the policy trained in the simulator should do **best on the real robot**

$$\min_{\theta} \; J_{\text{real}}\big(\nu^{\star}(\theta)\big), \qquad \nu^{\star}(\theta) \text{ trained in the simulator with parameters } \theta$$

Where do we use κ? (1) Sim-to-real: tune the simulator

Motivation: a policy is trained in a simulator, then deployed on a real system.



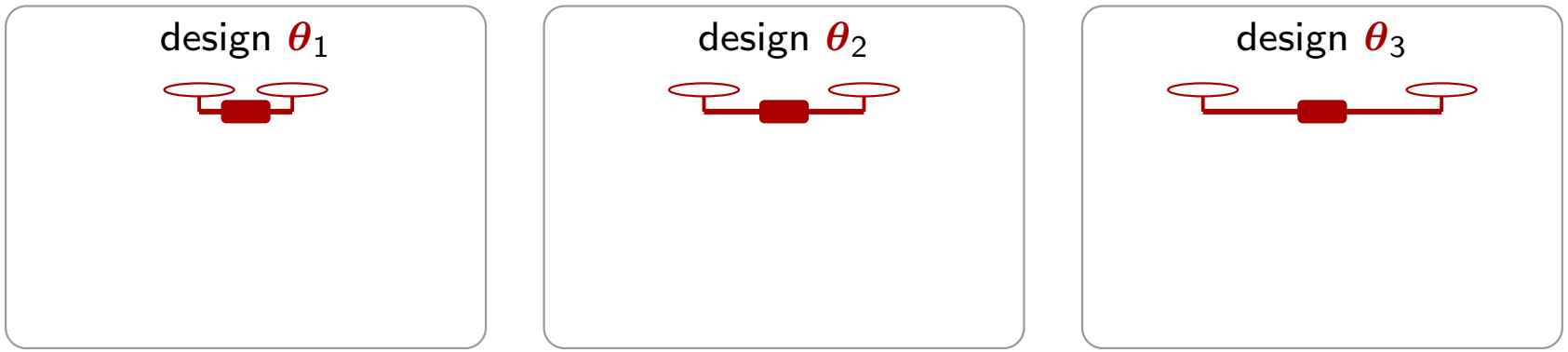
- Changing θ changes the **simulator**, our model of it
- Goal: the policy trained in the simulator should do **best on the real robot**

$$\min_{\theta} \; J_{\text{real}}\big(\nu^\star(\theta)\big), \qquad \nu^\star(\theta) \text{ trained in the simulator with parameters } \theta$$

- Not system identification: the best θ for **decisions** need not be the most accurate model

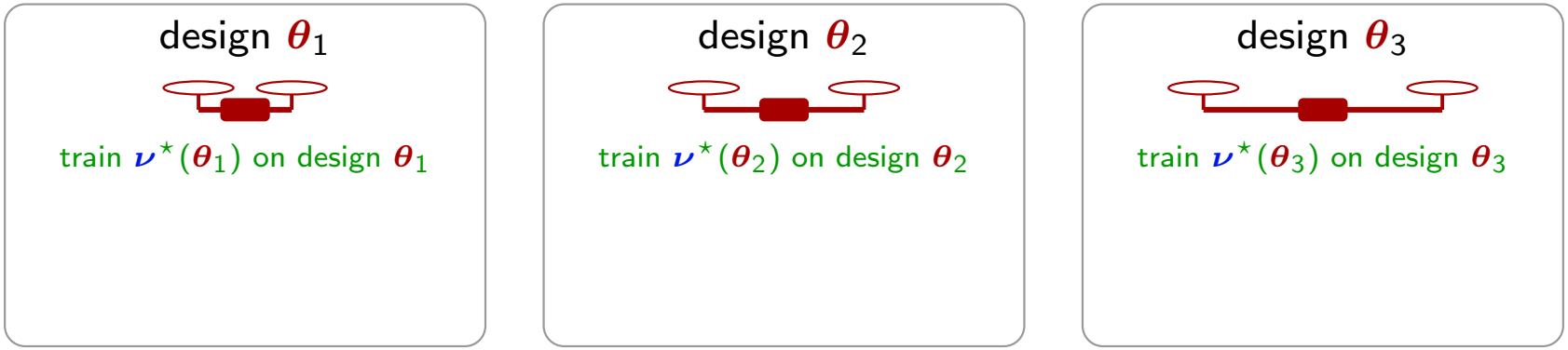
Where do we use κ? (2) Co-design: design the system and its policy

The system is **to be designed**: $\boldsymbol{\theta}$ as the **design space** (arm length, motor size; battery size, ...)



Where do we use κ? (2) Co-design: design the system and its policy

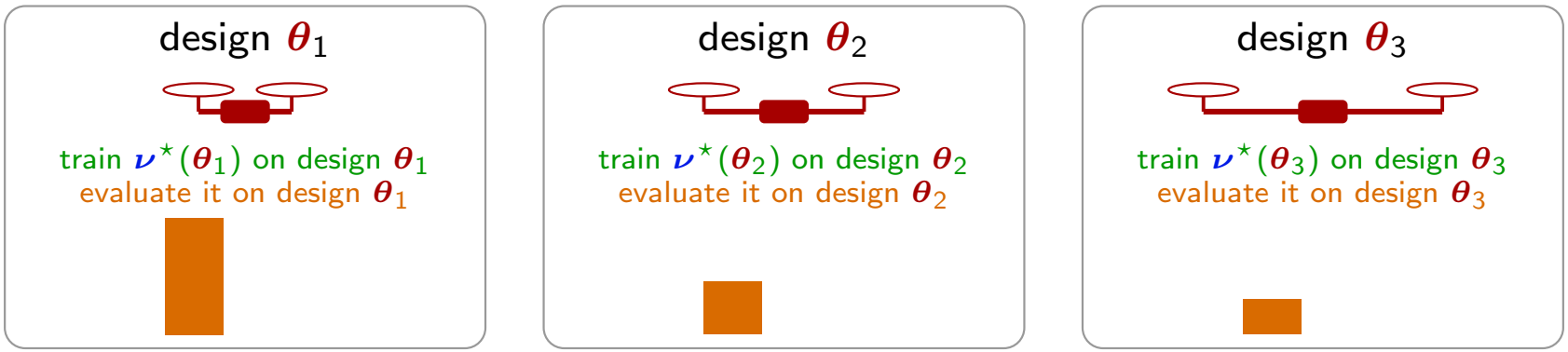
The system is **to be designed**: $\boldsymbol{\theta}$ as the **design space** (arm length, motor size; battery size, ...)



- Changing $\boldsymbol{\theta}$ changes the **real system** itself, hence its MDP: the policy must be **re-trained**, $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$

Where do we use κ? (2) Co-design: design the system and its policy

The system is **to be designed**: $\boldsymbol{\theta}$ as the **design space** (arm length, motor size; battery size, ...)

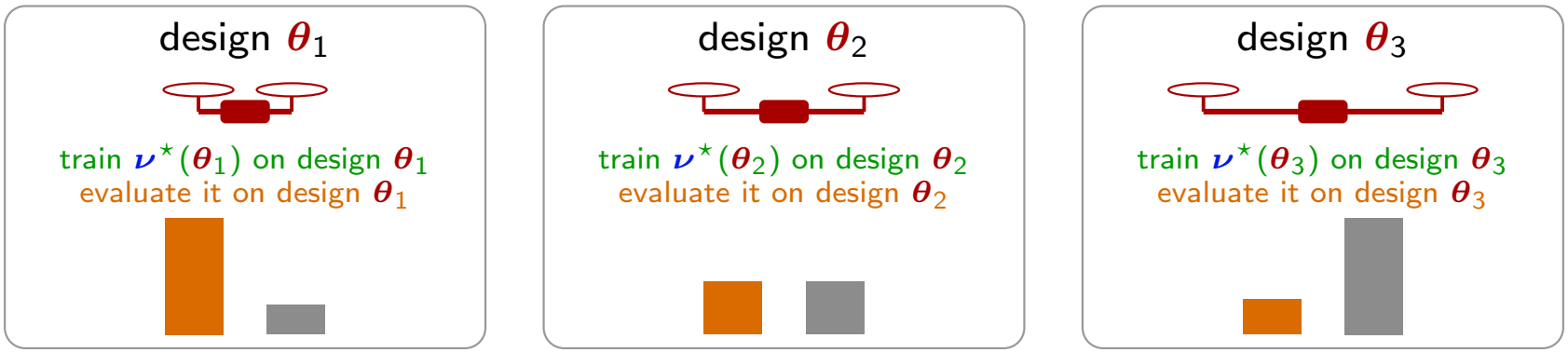


task cost of $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$ evaluated on its own design, $J^D(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta})$

- Changing $\boldsymbol{\theta}$ changes the **real system** itself, hence its MDP: the policy must be **re-trained**, $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$
- Each trained policy is **evaluated on its own design**: the system it is judged on changes with $\boldsymbol{\theta}$ ...

Where do we use κ? (2) Co-design: design the system and its policy

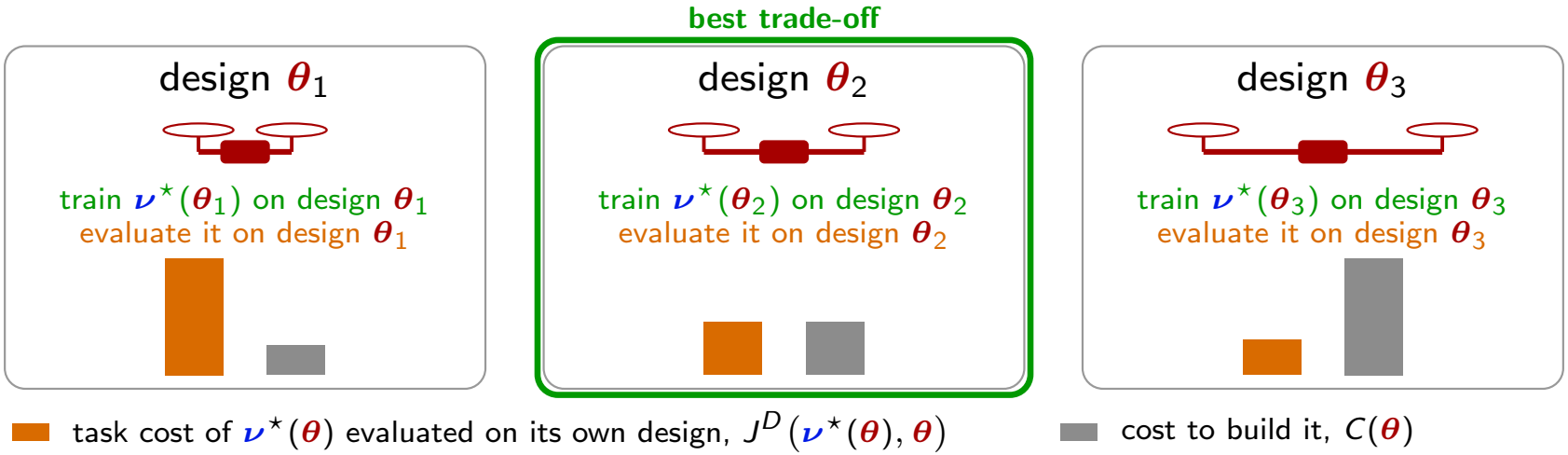
The system is **to be designed**: $\boldsymbol{\theta}$ as the **design space** (arm length, motor size; battery size, ...)



- Changing $\boldsymbol{\theta}$ changes the **real system** itself, hence its MDP: the policy must be **re-trained**, $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$
- Each trained policy is **evaluated on its own design**: the system it is judged on changes with $\boldsymbol{\theta}$ ...
- ... plus what it costs to build: $\min_{\boldsymbol{\theta}} \; J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta})$

Where do we use κ? (2) Co-design: design the system and its policy

The system is **to be designed**: $\boldsymbol{\theta}$ as the **design space** (arm length, motor size; battery size, . . .)



- Changing $\boldsymbol{\theta}$ changes the **real system** itself, hence its MDP: the policy must be **re-trained**, $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$
- Each trained policy is **evaluated on its own design**: the system it is judged on changes with $\boldsymbol{\theta}$ . . .
- . . . plus what it costs to build: $\min_{\boldsymbol{\theta}} \; J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta})$

Sim-to-real: $\boldsymbol{\theta}$ changes only the **model**. Co-design: $\boldsymbol{\theta}$ changes the **robot itself**.

Using κ: the gradient of the outer objective

In both problems we tune $\boldsymbol{\theta}$ with the gradient of an **outer objective** $J^{\text{out}}\big(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\big)$ that **judges** the trained policy. The blocker was $\nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\, \boldsymbol{\kappa}$: now computable.

$$\nabla_{\boldsymbol{\theta}} J^{\text{out}}\big(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\big) = \underbrace{\partial_{\boldsymbol{\theta}} J^{\text{out}}}_{\text{direct: } \boldsymbol{\nu} \text{ fixed}} + \underbrace{\nabla_{\boldsymbol{\nu}} J^{\text{out}}\, \boldsymbol{\kappa}}_{\text{indirect: the policy re-adapts}}, \qquad \boldsymbol{\kappa} = -(\nabla_{\boldsymbol{\nu}} \mathbf{F})^{-1} \nabla_{\boldsymbol{\theta}} \mathbf{F}$$

	Sim-to-real	**Co-design**

Using κ: the gradient of the outer objective

In both problems we tune $\boldsymbol{\theta}$ with the gradient of an **outer objective** $J^{\text{out}}\big(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\big)$ that **judges** the trained policy. The blocker was $\nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\, \boldsymbol{\kappa}$: now computable.

$$\nabla_{\boldsymbol{\theta}} J^{\text{out}}\big(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\big) = \underbrace{\partial_{\boldsymbol{\theta}} J^{\text{out}}}_{\text{direct: } \boldsymbol{\nu} \text{ fixed}} + \underbrace{\nabla_{\boldsymbol{\nu}} J^{\text{out}}\, \boldsymbol{\kappa}}_{\text{indirect: the policy re-adapts}}, \qquad \boldsymbol{\kappa} = -(\nabla_{\boldsymbol{\nu}} \mathbf{F})^{-1} \nabla_{\boldsymbol{\theta}} \mathbf{F}$$

	Sim-to-real	**Co-design**
$\boldsymbol{\theta}$	simulator parameters	design of the system

Using κ: the gradient of the outer objective

In both problems we tune $\boldsymbol{\theta}$ with the gradient of an **outer objective** $J^{\text{out}}\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right)$ that **judges** the trained policy. The blocker was $\nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\, \boldsymbol{\kappa}$: now computable.

$$\nabla_{\boldsymbol{\theta}} J^{\text{out}}\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) = \underbrace{\partial_{\boldsymbol{\theta}} J^{\text{out}}}_{\text{direct: } \boldsymbol{\nu} \text{ fixed}} + \underbrace{\nabla_{\boldsymbol{\nu}} J^{\text{out}}\, \boldsymbol{\kappa}}_{\text{indirect: the policy re-adapts}}, \qquad \boldsymbol{\kappa} = -(\nabla_{\boldsymbol{\nu}} \mathbf{F})^{-1} \nabla_{\boldsymbol{\theta}} \mathbf{F}$$

	Sim-to-real	**Co-design**
$\boldsymbol{\theta}$	simulator parameters	design of the system
J^{out}	$J_{\text{real}}(\boldsymbol{\nu}^\star)$, on the real robot	$J^D(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) + C(\boldsymbol{\theta})$, in the simulator

Using κ: the gradient of the outer objective

In both problems we tune θ with the gradient of an **outer objective** $J^{\text{out}}\big(\nu^\star(\theta), \theta\big)$ that **judges** the trained policy. The blocker was $\nabla_\theta \pi_\theta = \nabla_\nu \pi_\nu \, \kappa$: now computable.

$$\nabla_\theta J^{\text{out}}\big(\nu^\star(\theta), \theta\big) = \underbrace{\partial_\theta J^{\text{out}}}_{\text{direct: } \nu \text{ fixed}} + \underbrace{\nabla_\nu J^{\text{out}} \, \kappa}_{\text{indirect: the policy re-adapts}}, \qquad \kappa = -(\nabla_\nu \mathbf{F})^{-1} \nabla_\theta \mathbf{F}$$

	Sim-to-real	**Co-design**
θ	simulator parameters	design of the system
J^{out}	$J_{\text{real}}(\nu^\star)$, on the real robot	$J^D(\nu^\star, \theta) + C(\theta)$, in the simulator
policy judged on	the fixed real robot	its own design θ (changes with θ)

Using κ: the gradient of the outer objective

In both problems we tune θ with the gradient of an **outer objective** $J^{\text{out}}\big(\nu^\star(\theta), \theta\big)$ that **judges** the trained policy. The blocker was $\nabla_\theta \pi_\theta = \nabla_\nu \pi_\nu \, \kappa$: now computable.

$$\nabla_\theta J^{\text{out}}\big(\nu^\star(\theta), \theta\big) = \underbrace{\partial_\theta J^{\text{out}}}_{\text{direct: } \nu \text{ fixed}} + \underbrace{\nabla_\nu J^{\text{out}} \, \kappa}_{\text{indirect: the policy re-adapts}}, \qquad \kappa = -(\nabla_\nu \mathbf{F})^{-1} \nabla_\theta \mathbf{F}$$

	Sim-to-real	**Co-design**
θ	simulator parameters	design of the system
J^{out}	$J_{\text{real}}(\nu^\star)$, on the real robot	$J^D(\nu^\star, \theta) + C(\theta)$, in the simulator
policy judged on	the fixed real robot	its own design θ (changes with θ)
direct term	0 (the real robot ignores θ)	$\neq 0$ (θ changes the system)

Using κ: the gradient of the outer objective

In both problems we tune $\boldsymbol{\theta}$ with the gradient of an **outer objective** $J^{\text{out}}\big(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\big)$ that **judges** the trained policy. The blocker was $\nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\, \boldsymbol{\kappa}$: now computable.

$$\nabla_{\boldsymbol{\theta}} J^{\text{out}}\big(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\big) = \underbrace{\partial_{\boldsymbol{\theta}} J^{\text{out}}}_{\text{direct: } \boldsymbol{\nu} \text{ fixed}} + \underbrace{\nabla_{\boldsymbol{\nu}} J^{\text{out}}\, \boldsymbol{\kappa}}_{\text{indirect: the policy re-adapts}}, \qquad \boldsymbol{\kappa} = -(\nabla_{\boldsymbol{\nu}} \mathbf{F})^{-1} \nabla_{\boldsymbol{\theta}} \mathbf{F}$$

	Sim-to-real	**Co-design**
$\boldsymbol{\theta}$	simulator parameters	design of the system
J^{out}	$J_{\text{real}}(\boldsymbol{\nu}^\star)$, on the real robot	$J^D(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) + C(\boldsymbol{\theta})$, in the simulator
policy judged on	the fixed real robot	its own design $\boldsymbol{\theta}$ (changes with $\boldsymbol{\theta}$)
direct term	0 (the real robot ignores $\boldsymbol{\theta}$)	$\neq 0$ ($\boldsymbol{\theta}$ changes the system)
$\nabla_{\boldsymbol{\nu}} J^{\text{out}}$	**LR**, from real rollouts	**PD** (AD), in the simulator

Using κ: the gradient of the outer objective

In both problems we tune θ with the gradient of an **outer objective** $J^{\text{out}}\big(\nu^\star(\theta), \theta\big)$ that **judges** the trained policy. The blocker was $\nabla_\theta \pi_\theta = \nabla_\nu \pi_\nu \, \kappa$: now computable.

$$\nabla_\theta J^{\text{out}}\big(\nu^\star(\theta), \theta\big) = \underbrace{\partial_\theta J^{\text{out}}}_{\text{direct: } \nu \text{ fixed}} + \underbrace{\nabla_\nu J^{\text{out}} \, \kappa}_{\text{indirect: the policy re-adapts}}, \qquad \kappa = -(\nabla_\nu \mathbf{F})^{-1} \nabla_\theta \mathbf{F}$$

	Sim-to-real	**Co-design**
θ	simulator parameters	design of the system
J^{out}	$J_{\text{real}}(\nu^\star)$, on the real robot	$J^D(\nu^\star, \theta) + C(\theta)$, in the simulator
policy judged on	the fixed real robot	its own design θ (changes with θ)
direct term	0 (the real robot ignores θ)	$\neq 0$ (θ changes the system)
$\nabla_\nu J^{\text{out}}$	**LR**, from real rollouts	**PD** (AD), in the simulator

Adjoint trick (never form κ): with $g = \nabla_\nu J^{\text{out}}$, $g^\top \kappa = -g^\top (\nabla_\nu \mathbf{F})^{-1} \nabla_\theta \mathbf{F}$, so

1. solve $(\nabla_\nu \mathbf{F})^\top \lambda = g$: one matrix-free linear solve in dim ν (CG, MINRES, GMRES)

Using κ: the gradient of the outer objective

In both problems we tune θ with the gradient of an **outer objective** $J^{\text{out}}\big(\nu^\star(\theta), \theta\big)$ that **judges** the trained policy. The blocker was $\nabla_\theta \pi_\theta = \nabla_\nu \pi_\nu \, \kappa$: now computable.

$$\nabla_\theta J^{\text{out}}\big(\nu^\star(\theta), \theta\big) = \underbrace{\partial_\theta J^{\text{out}}}_{\text{direct: } \nu \text{ fixed}} + \underbrace{\nabla_\nu J^{\text{out}} \, \kappa}_{\text{indirect: the policy re-adapts}}, \qquad \kappa = -(\nabla_\nu \mathbf{F})^{-1} \nabla_\theta \mathbf{F}$$

	Sim-to-real	**Co-design**
θ	simulator parameters	design of the system
J^{out}	$J_{\text{real}}(\nu^\star)$, on the real robot	$J^D(\nu^\star, \theta) + C(\theta)$, in the simulator
policy judged on	the fixed real robot	its own design θ (changes with θ)
direct term	0 (the real robot ignores θ)	$\neq 0$ (θ changes the system)
$\nabla_\nu J^{\text{out}}$	**LR**, from real rollouts	**PD** (AD), in the simulator

Adjoint trick (never form κ): with $g = \nabla_\nu J^{\text{out}}$, $g^\top \kappa = -g^\top (\nabla_\nu \mathbf{F})^{-1} \nabla_\theta \mathbf{F}$, so

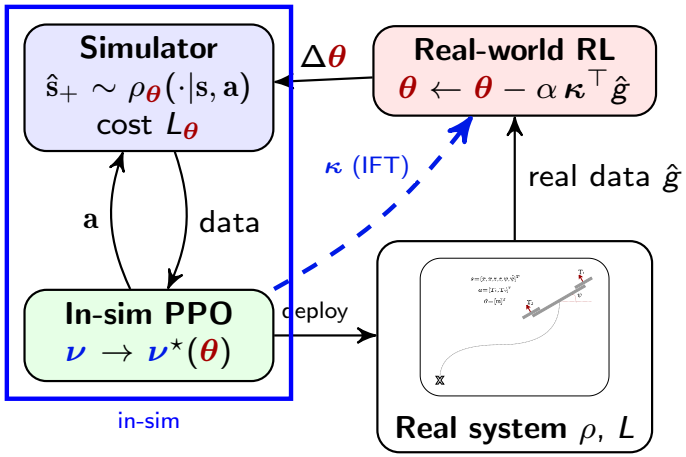
1. solve $(\nabla_\nu \mathbf{F})^\top \lambda = g$: one matrix-free linear solve in dim ν (CG, MINRES, GMRES)
2. indirect term $= -(\nabla_\theta \mathbf{F})^\top \lambda$: one vector–Jacobian product ($> 100\times$ faster)

Outline

1. Decisions from data
2. AI models for Decision
3. Differentiating Policy Training
4. Examples



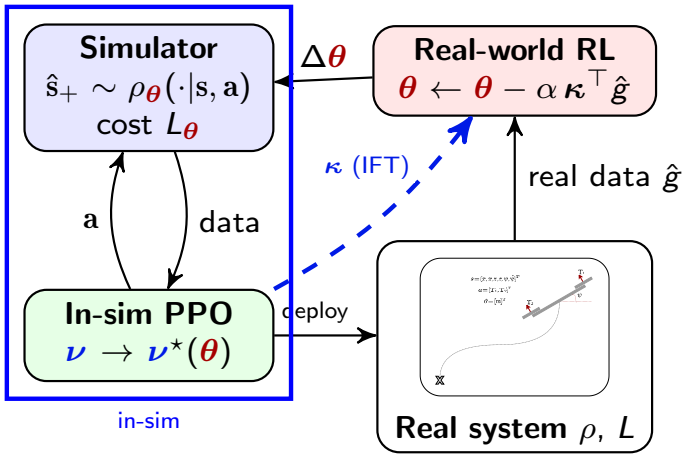
Example 1: Sim-to-real with bi-level PPO



Recall (slide 25): tune θ so that the policy trained in sim is best on the robot

$$\min_{\theta} \; J_{\text{real}}\big(\nu^\star(\theta)\big) \quad \text{s.t.} \quad \nabla_\nu \hat{J}_\theta(\nu^\star) = 0$$

Example 1: Sim-to-real with bi-level PPO

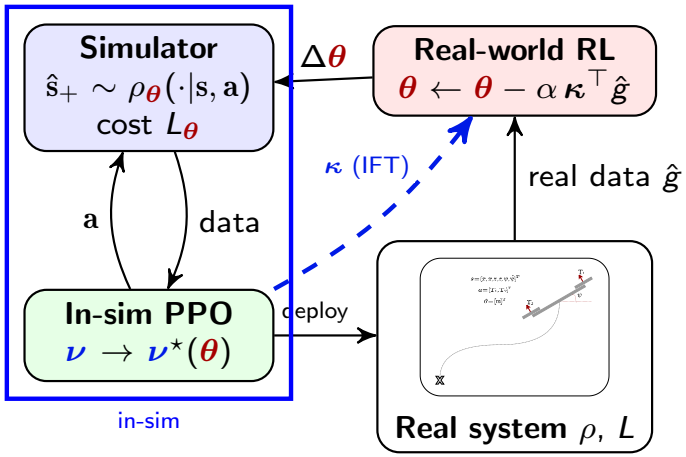


Recall (slide 25): tune $\boldsymbol{\theta}$ so that the policy trained in sim is best on the robot

$$\min_{\boldsymbol{\theta}} \; J_{\text{real}}\big(\boldsymbol{\nu}^\star(\boldsymbol{\theta})\big) \quad \text{s.t.} \quad \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

- **In sim**: PPO trains $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$ (warm start)

Example 1: Sim-to-real with bi-level PPO

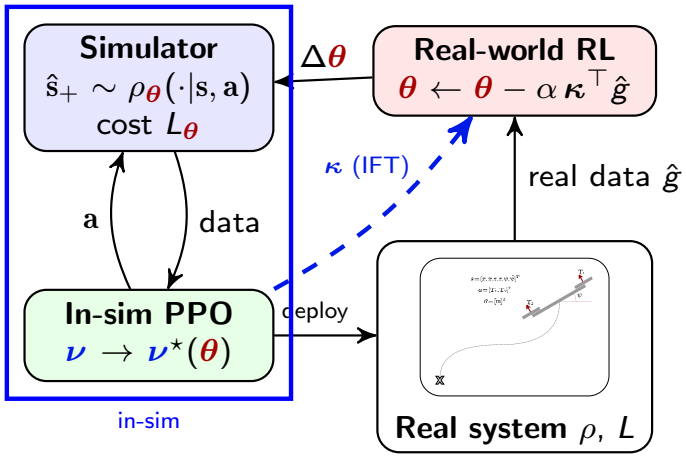


Recall (slide 25): tune θ so that the policy trained in sim is best on the robot

$$\min_{\theta} \; J_{\text{real}}\left(\nu^\star(\theta)\right) \quad \text{s.t.} \quad \nabla_\nu \hat{J}_\theta(\nu^\star) = 0$$

- **In sim**: PPO trains $\nu^\star(\theta)$ (warm start)
- **On the robot**: **LR** gradient $\hat{g}$, $\theta \leftarrow \theta - \alpha \kappa^\top \hat{g}$

Example 1: Sim-to-real with bi-level PPO

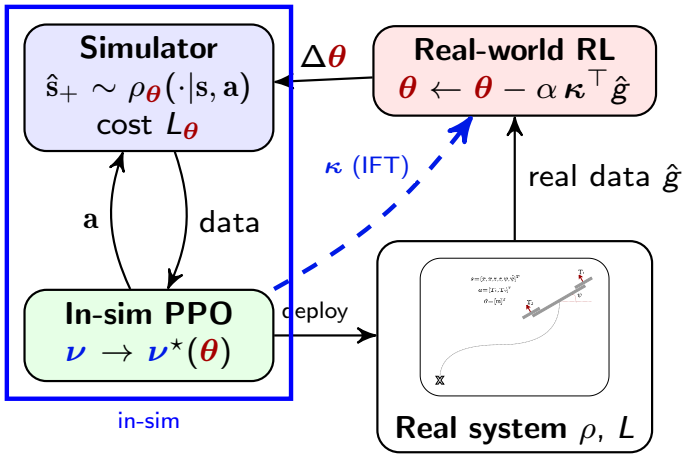


Recall (slide 25): tune θ so that the policy trained in sim is best on the robot

$$\min_{\theta} \; J_{\text{real}}\big(\nu^\star(\theta)\big) \quad \text{s.t.} \quad \nabla_\nu \hat{J}_\theta(\nu^\star) = 0$$

- **In sim**: PPO trains $\nu^\star(\theta)$ (warm start)
- **On the robot**: **LR** gradient $\hat{g}$, $\theta \leftarrow \theta - \alpha \kappa^\top \hat{g}$
- **Converges** a.s. (critics fast, policy medium, θ slow)

Example 1: Sim-to-real with bi-level PPO

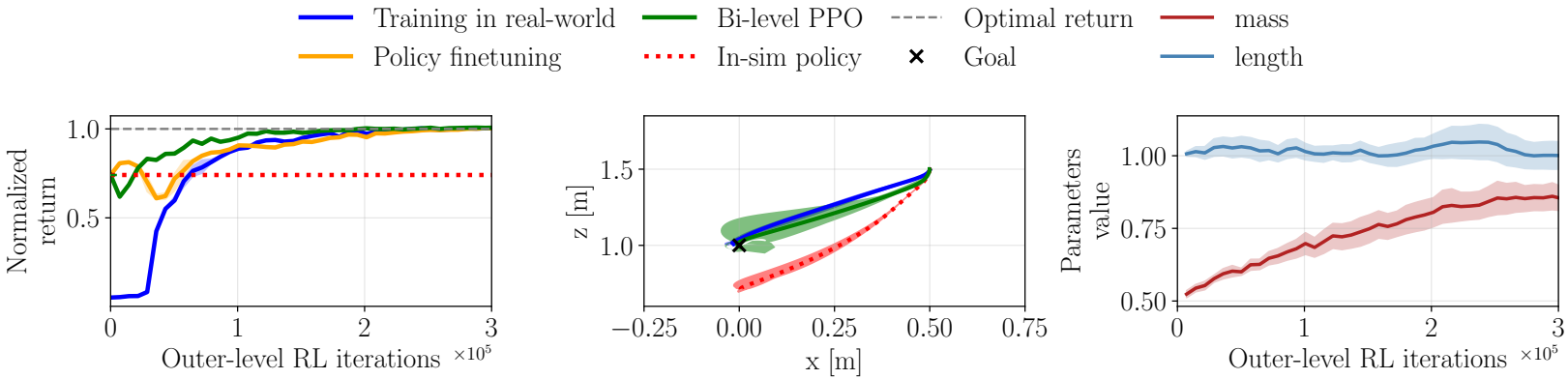


Recall (slide 25): tune θ so that the policy trained in sim is best on the robot

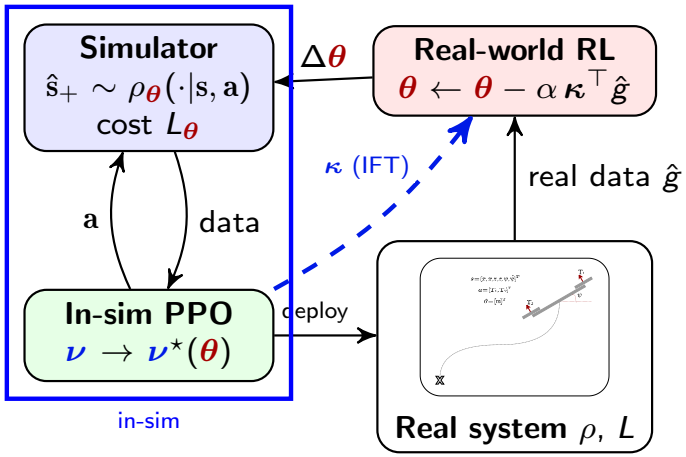
$$\min_{\theta} \; J_{\text{real}}\left(\nu^\star(\theta)\right) \quad \text{s.t.} \quad \nabla_\nu \hat{J}_\theta(\nu^\star) = 0$$

- **In sim**: PPO trains $\nu^\star(\theta)$ (warm start)
- **On the robot**: LR gradient $\hat{g}$, $\theta \leftarrow \theta - \alpha \kappa^\top \hat{g}$
- **Converges** a.s. (critics fast, policy medium, θ slow)

2D quadrotor (safe-control-gym): the simulator starts with wrong mass and arm length



Example 1: Sim-to-real with bi-level PPO

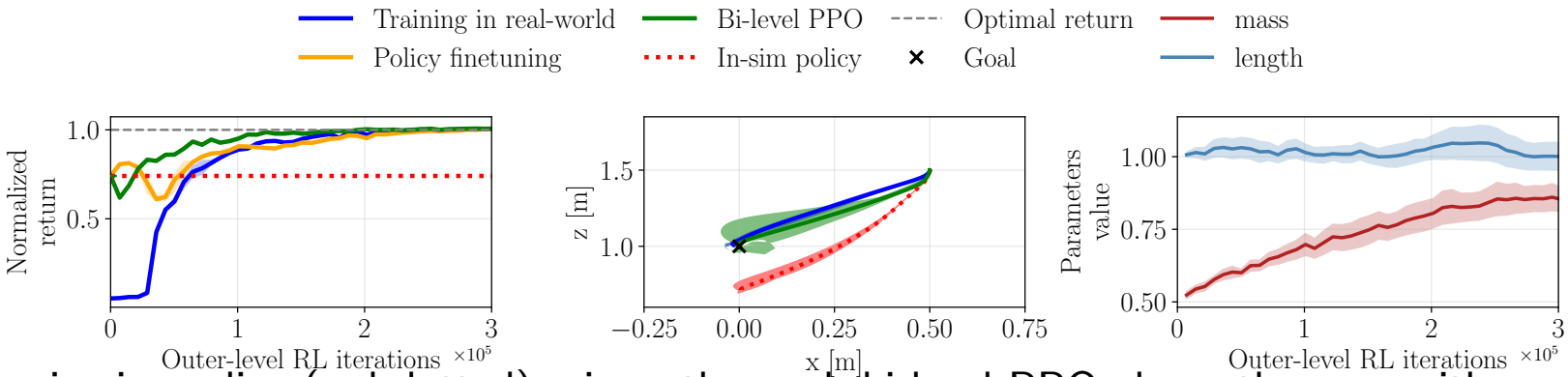


Recall (slide 25): tune θ so that the policy trained in sim is best on the robot

$$\min_\theta \; J_{\text{real}}\big(\nu^\star(\theta)\big) \quad \text{s.t.} \quad \nabla_\nu \hat{J}_\theta(\nu^\star) = 0$$

- **In sim**: PPO trains $\nu^\star(\theta)$ (warm start)
- **On the robot**: LR gradient $\hat{g}$, $\theta \leftarrow \theta - \alpha \kappa^\top \hat{g}$
- **Converges** a.s. (critics fast, policy medium, θ slow)

2D quadrotor (safe-control-gym): the simulator starts with wrong mass and arm length



the in-sim policy (red dotted) misses the goal; bi-level PPO closes the gap, with mass $\to \approx 0.85\, m_0$: **a wrong model, but the optimal policy**

Example 2: Co-design of a system and its policy

Each design $\boldsymbol{\theta}$ needs its own trained policy

Bi-level problem

$$\begin{aligned} \min_{\boldsymbol{\theta}} \quad & J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta}) \\ \text{s.t.} \quad & \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta}) \end{aligned}$$

Example 2: Co-design of a system and its policy

Each design $\boldsymbol{\theta}$ needs its own trained policy

Bi-level problem

$$\begin{aligned} \min_{\boldsymbol{\theta}} \quad & J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta}) \\ \text{s.t.} \quad & \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta}) \end{aligned}$$

- J^π trains the policy; J^D **judges** the design; $C(\boldsymbol{\theta})$: cost to build it

Example 2: Co-design of a system and its policy

Each design $\boldsymbol{\theta}$ needs its own trained policy

Bi-level problem

$$\begin{aligned} \min_{\boldsymbol{\theta}} \quad & J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta}) \\ \text{s.t.} \quad & \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta}) \end{aligned}$$

- J^π trains the policy; J^D **judges** the design; $C(\boldsymbol{\theta})$: cost to build it
- usual practice: BO or ES around full RL runs, scales poorly with dim $\boldsymbol{\theta}$

Example 2: Co-design of a system and its policy

Each design $\boldsymbol{\theta}$ needs its own trained policy

Bi-level problem

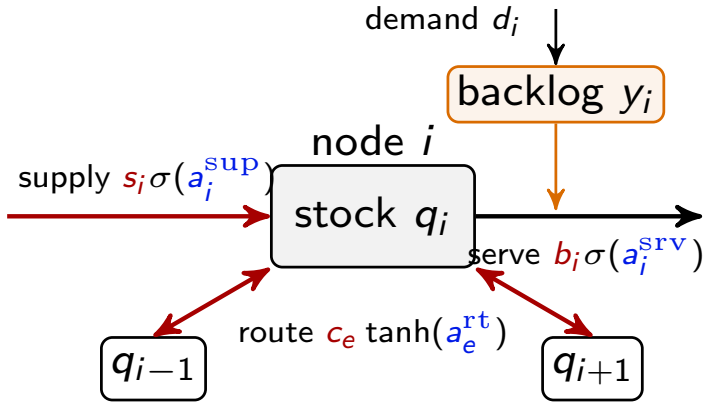
$$\min_{\boldsymbol{\theta}} \quad J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta})$$

$$\text{s.t.} \quad \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta})$$

- J^π trains the policy; J^D **judges** the design; $C(\boldsymbol{\theta})$: cost to build it
- usual practice: BO or ES around full RL runs, scales poorly with $\dim \boldsymbol{\theta}$

Example: inventory–routing network

$\boldsymbol{\theta}$ = capacities, $\dim \boldsymbol{\theta}$ from 15 to 150



the design bounds the actions;
$J^D = J^\pi + \lambda_y \cdot$backlog

Example 2: Co-design of a system and its policy

Each design $\boldsymbol{\theta}$ needs its own trained policy

Bi-level problem

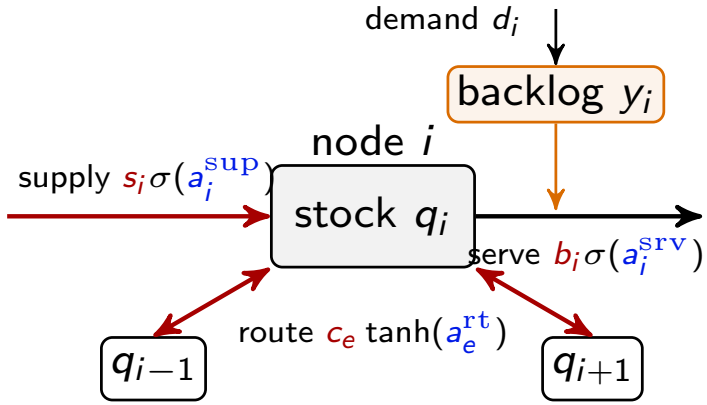
$$\min_{\boldsymbol{\theta}} \quad J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta})$$

$$\text{s.t.} \quad \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta})$$

- J^π trains the policy; J^D **judges** the design; $C(\boldsymbol{\theta})$: cost to build it
- usual practice: BO or ES around full RL runs, scales poorly with $\dim \boldsymbol{\theta}$

Example: inventory–routing network

$\boldsymbol{\theta}$ = capacities, $\dim \boldsymbol{\theta}$ from 15 to 150



the design bounds the actions;

$J^D = J^\pi + \lambda_y \cdot \text{backlog}$

Algorithm: Gradient-based co-design (differentiable simulator: **PD** everywhere)

for $k = 0, 1, 2, \ldots$ **do**

a few warm-started SVG steps on $J^\pi(\cdot, \boldsymbol{\theta}_k)$: they only have to **track** $\boldsymbol{\nu}^\star(\boldsymbol{\theta}_k)$;

$g \leftarrow \partial_{\boldsymbol{\theta}} J^D + \nabla_{\boldsymbol{\nu}} J^D \, \boldsymbol{\kappa} + \nabla_{\boldsymbol{\theta}} C$ (`jax.grad` and the adjoint trick, slide 27);

$\boldsymbol{\theta}_{k+1} \leftarrow \Pi_\Theta(\boldsymbol{\theta}_k - \alpha\, g)$; `// projection onto the bounds`

Example 2: Co-design of a system and its policy

Each design $\boldsymbol{\theta}$ needs its own trained policy

Bi-level problem

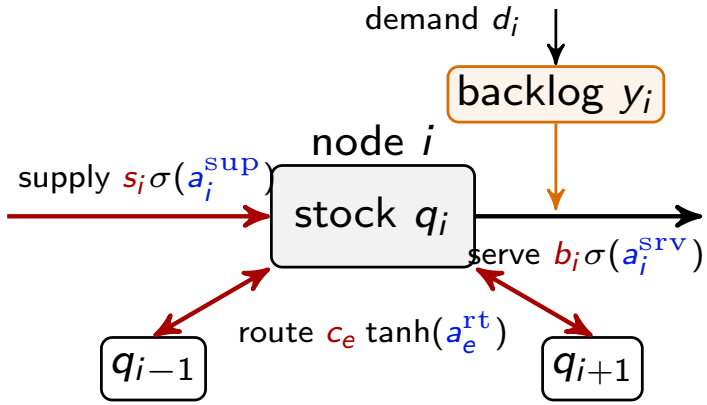
$$\min_{\boldsymbol{\theta}} \quad J^D\big(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\big) + C(\boldsymbol{\theta})$$

$$\text{s.t.} \quad \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta})$$

- J^π trains the policy; J^D **judges** the design; $C(\boldsymbol{\theta})$: cost to build it
- usual practice: BO or ES around full RL runs, scales poorly with $\dim \boldsymbol{\theta}$

Example: inventory–routing network

$\boldsymbol{\theta}$ = capacities, $\dim \boldsymbol{\theta}$ from 15 to 150



the design bounds the actions;

$J^D = J^\pi + \lambda_y \cdot \text{backlog}$

Algorithm: Gradient-based co-design (differentiable simulator: **PD** everywhere)

---

for $k = 0, 1, 2, \ldots$ **do**

a few warm-started SVG steps on $J^\pi(\cdot, \boldsymbol{\theta}_k)$: they only have to **track** $\boldsymbol{\nu}^\star(\boldsymbol{\theta}_k)$;

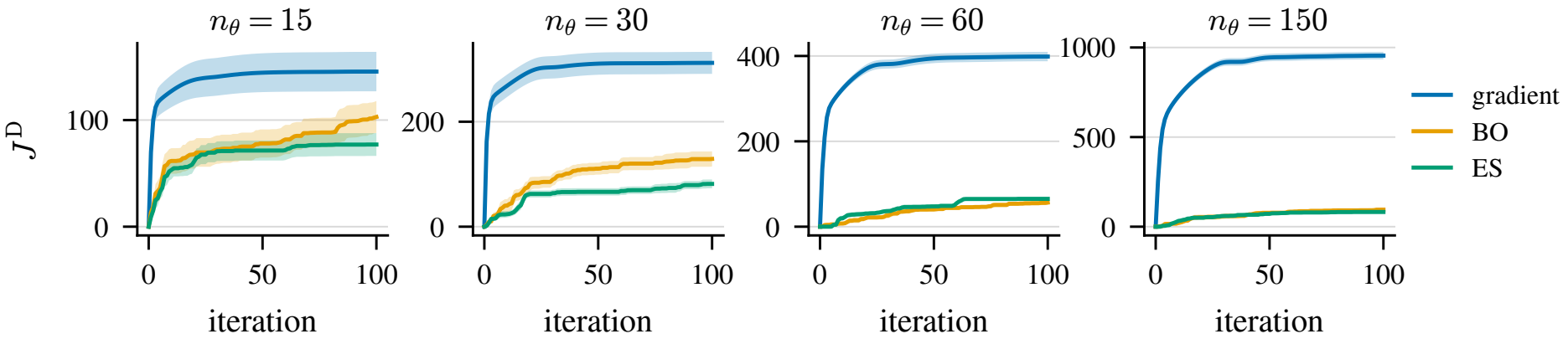
$g \leftarrow \partial_{\boldsymbol{\theta}} J^D + \nabla_{\boldsymbol{\nu}} J^D \, \boldsymbol{\kappa} + \nabla_{\boldsymbol{\theta}} C$ (`jax.grad` and the adjoint trick, slide 27);

$\boldsymbol{\theta}_{k+1} \leftarrow \Pi_\Theta(\boldsymbol{\theta}_k - \alpha\, g)$; `// projection onto the bounds`

Converges to an approximately stationary design for small α: error $\mathcal{O}\big(\frac{1}{\alpha N}\big) + \mathcal{O}(\alpha\sigma^2)$+ bias

Co-design: inventory–routing network

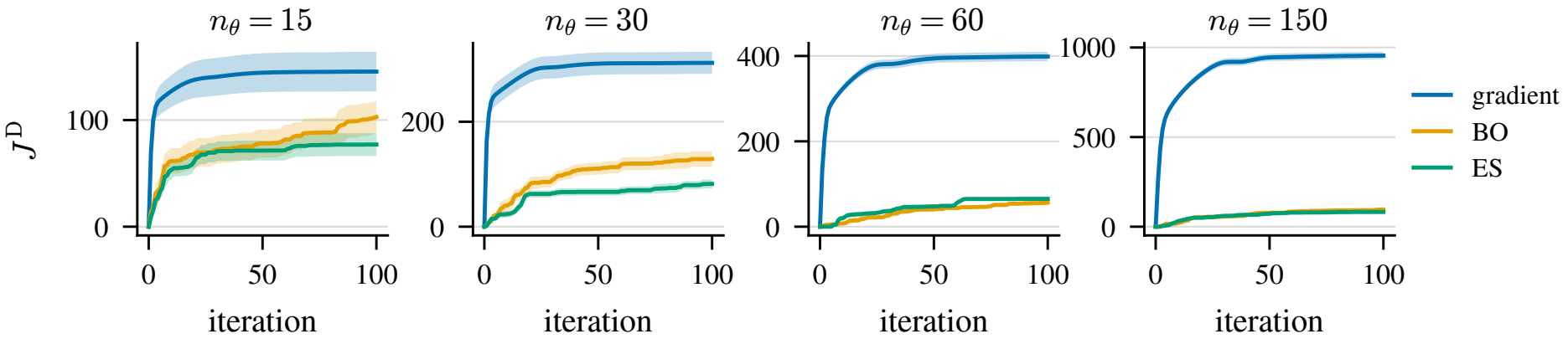
Against black-box search: 100 design evaluations each, matched objectives ($\lambda_y = 0$, so the κ-term is 0 and only the direct gradient is used)



Plots show the design *return* $-(J^D + C)$: higher is better.

Co-design: inventory–routing network

Against black-box search: 100 design evaluations each, matched objectives ($\lambda_y = 0$, so the κ-term is 0 and only the direct gradient is used)

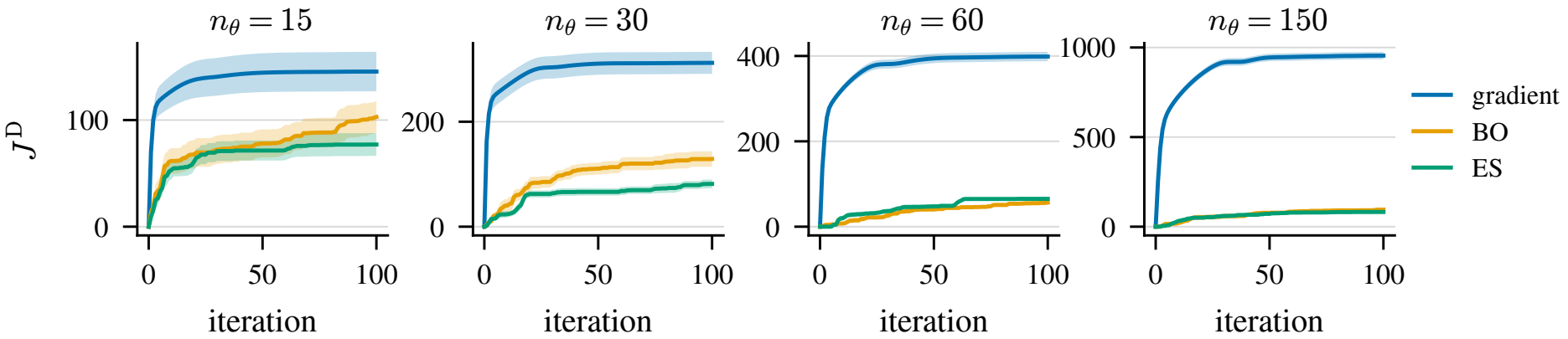


- the gradient approach outperforms BO and ES at every size, and the gap grows with dim $\boldsymbol{\theta}$ (15 to 150)

Plots show the design *return* $-(J^D + C)$: higher is better.

Co-design: inventory–routing network

Against black-box search: 100 design evaluations each, matched objectives ($\lambda_y = 0$, so the κ-term is 0 and only the direct gradient is used)

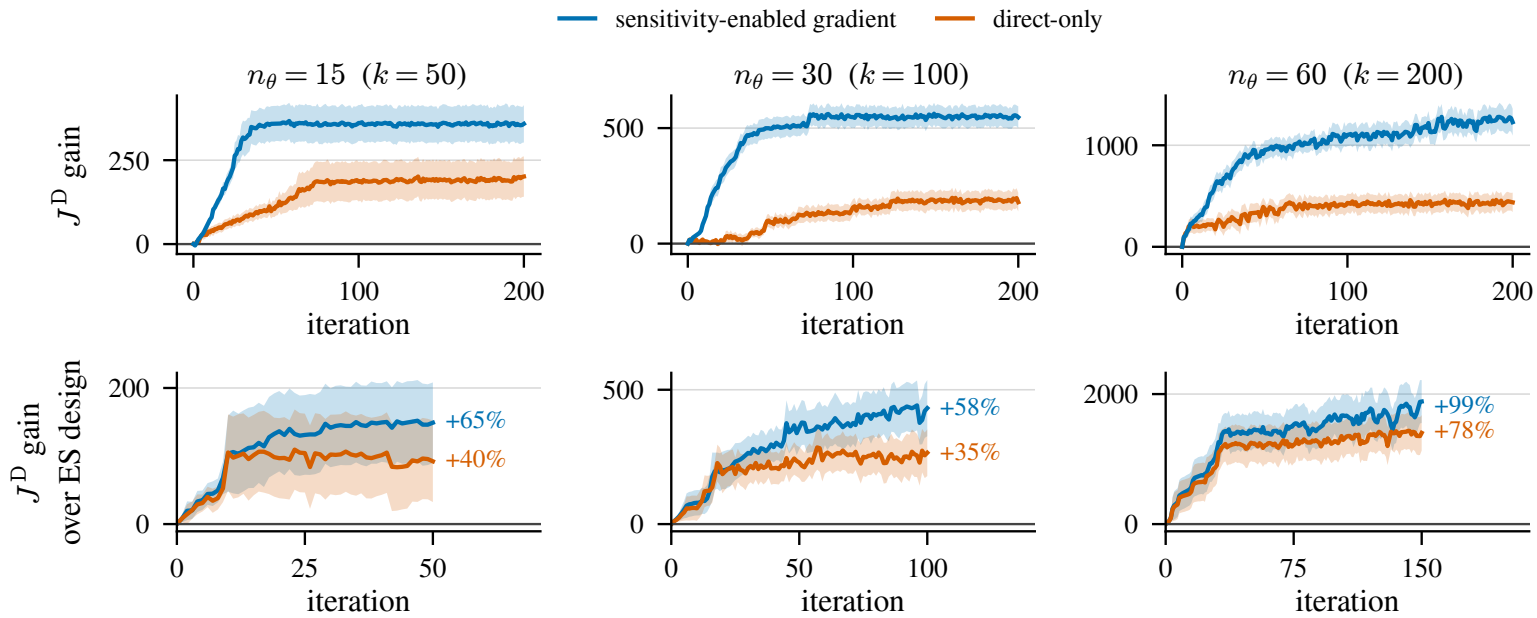


- the gradient approach outperforms BO and ES at every size, and the gap grows with dim $\boldsymbol{\theta}$ (15 to 150)
- cost per design evaluation: $\approx 2\,\mathrm{s}$ for the gradient, $\approx 2\,\mathrm{s}$ ES, $\approx 3\,\mathrm{s}$ BO

Plots show the design *return* $-(J^D + C)$: higher is better.

Co-design: inventory–routing network

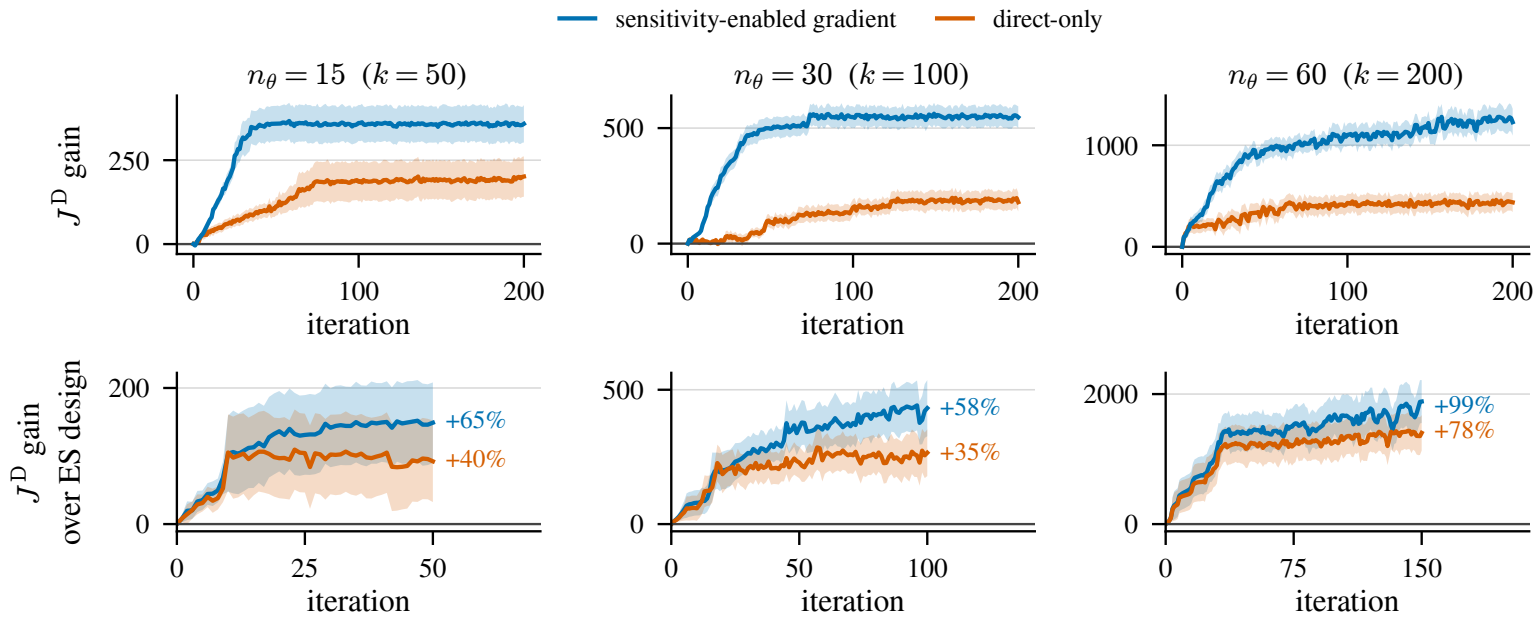
Effect of the κ-term: mismatched objectives ($\lambda_y = 0.1$, $J^D \neq J^\pi$)



Plots show the design *return* $-(J^D + C)$: higher is better.

Co-design: inventory–routing network

Effect of the κ-term: mismatched objectives ($\lambda_y = 0.1$, $J^D \neq J^\pi$)

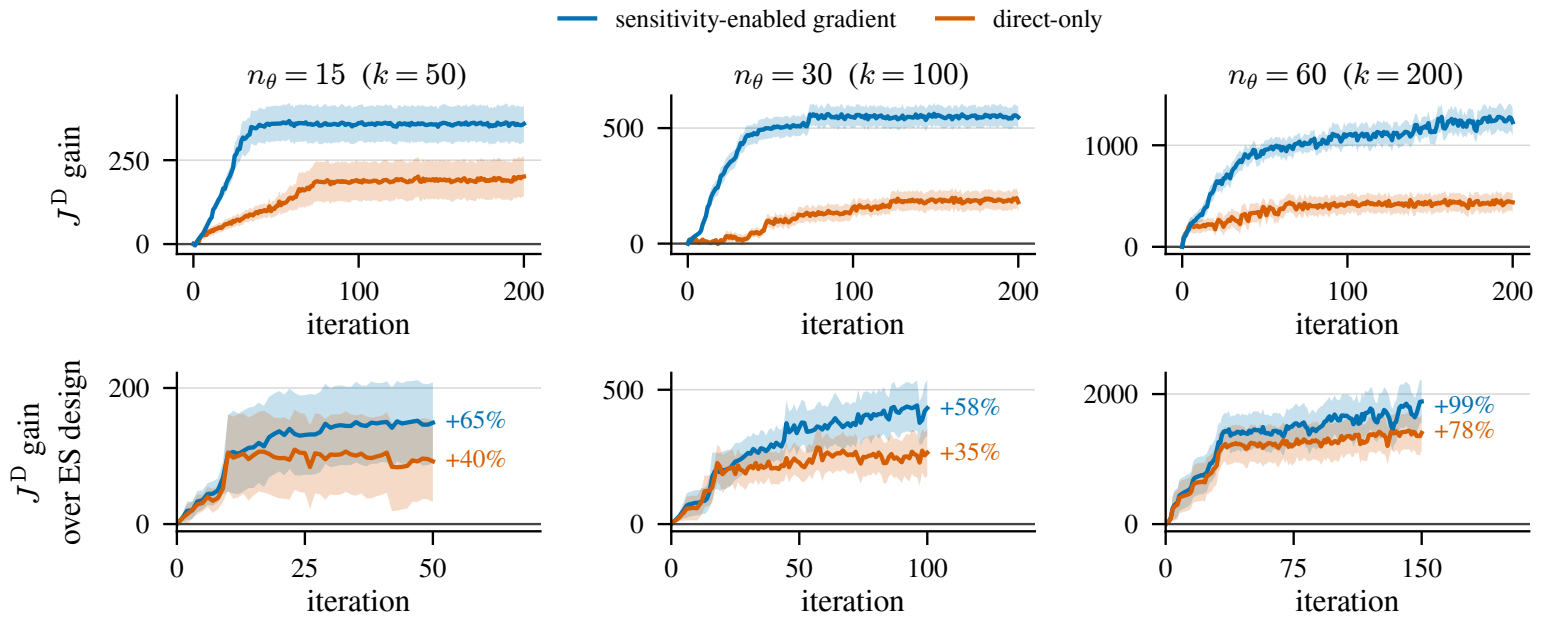


- **top**: sensitivity-enabled vs direct-only gradient, from the same initial design: the κ-term helps, more in higher dimension

Plots show the design *return* $-(J^D + C)$: higher is better.

Co-design: inventory–routing network

Effect of the κ-term: mismatched objectives ($\lambda_y = 0.1$, $J^D \neq J^\pi$)

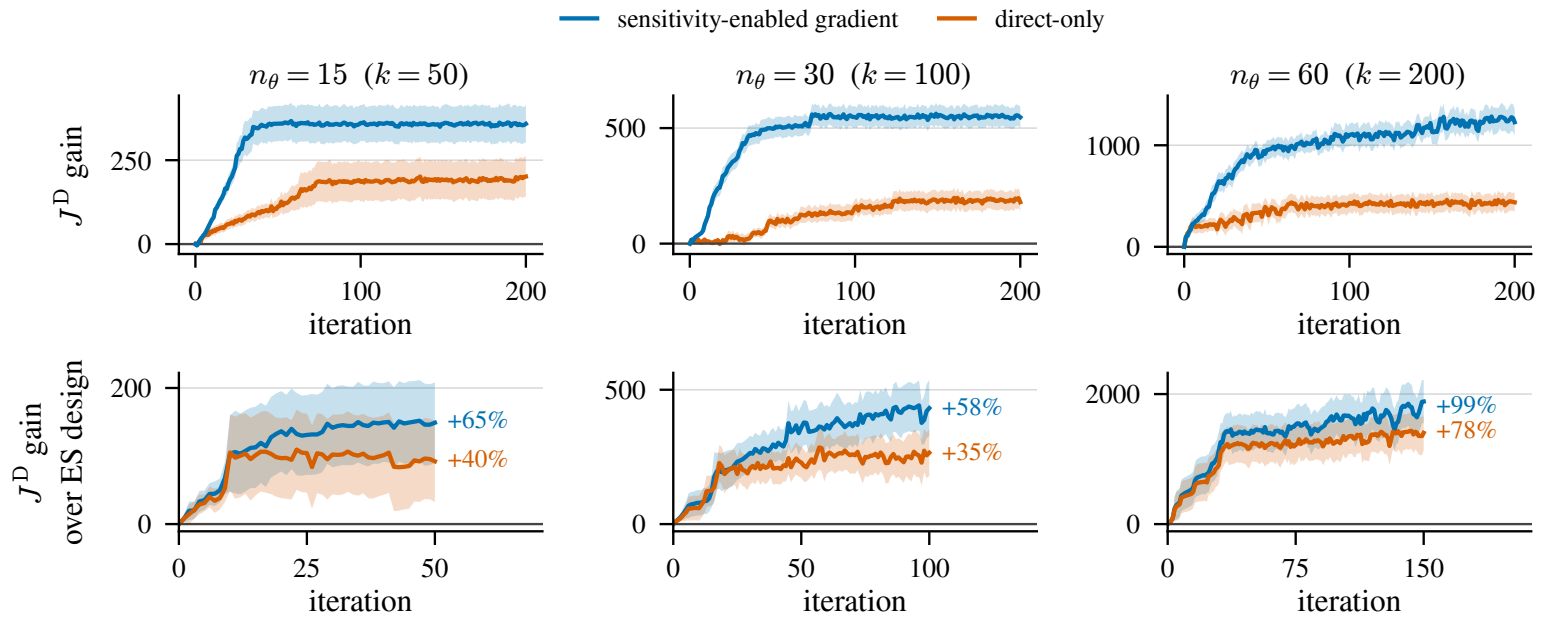


- **top**: sensitivity-enabled vs direct-only gradient, from the same initial design: the κ-term helps, more in higher dimension
- **bottom**: starting from the best design of a converged ES search: the gradient still improves it

Plots show the design *return* $-(J^D + C)$: higher is better.

Co-design: inventory–routing network

Effect of the κ-term: mismatched objectives ($\lambda_y = 0.1$, $J^D \neq J^\pi$)



- **top**: sensitivity-enabled vs direct-only gradient, from the same initial design: the κ-term helps, more in higher dimension
- **bottom**: starting from the best design of a converged ES search: the gradient still improves it
- cost: $\approx 6.5\,\mathrm{s}$ per design evaluation with a matrix-free κ-solve (vs $\approx 2\,\mathrm{s}$ direct-only)

Plots show the design *return* $-(J^D + C)$: higher is better.

Take-aways

1. **Models for decisions**: a policy optimal for an approximate model $\rho_{\boldsymbol{\theta}}$ is in general not optimal in reality, and standard model fitting does not fix this

Take-aways

1. **Models for decisions**: a policy optimal for an approximate model $\rho_{\boldsymbol{\theta}}$ is in general not optimal in reality, and standard model fitting does not fix this
2. **MDP equivalence**: some (non-unique) models and costs give the real optimal policy even if $\rho_{\boldsymbol{\theta}}$ is inaccurate $\Rightarrow$ tune $\boldsymbol{\theta}$ for **closed-loop performance**

Take-aways

1. **Models for decisions**: a policy optimal for an approximate model $\rho_{\boldsymbol{\theta}}$ is in general not optimal in reality, and standard model fitting does not fix this
2. **MDP equivalence**: some (non-unique) models and costs give the real optimal policy even if $\rho_{\boldsymbol{\theta}}$ is inaccurate $\Rightarrow$ tune $\boldsymbol{\theta}$ for **closed-loop performance**
3. **The obstacle**: this needs $\boldsymbol{\kappa} = \mathrm{d}\boldsymbol{\nu}^\star / \mathrm{d}\boldsymbol{\theta}$, the sensitivity of the trained policy

Take-aways

1. **Models for decisions**: a policy optimal for an approximate model $\rho_{\boldsymbol{\theta}}$ is in general not optimal in reality, and standard model fitting does not fix this
2. **MDP equivalence**: some (non-unique) models and costs give the real optimal policy even if $\rho_{\boldsymbol{\theta}}$ is inaccurate $\Rightarrow$ tune $\boldsymbol{\theta}$ for **closed-loop performance**
3. **The obstacle**: this needs $\boldsymbol{\kappa} = \mathrm{d}\boldsymbol{\nu}^\star / \mathrm{d}\boldsymbol{\theta}$, the sensitivity of the trained policy
4. **LR re-weights** paths (a surrogate loss, black-box simulator), **PD moves** them (backpropagation through the simulator); both are a forward and a backward pass, neither always has lower variance

Take-aways

1. **Models for decisions**: a policy optimal for an approximate model $\rho_{\boldsymbol{\theta}}$ is in general not optimal in reality, and standard model fitting does not fix this
2. **MDP equivalence**: some (non-unique) models and costs give the real optimal policy even if $\rho_{\boldsymbol{\theta}}$ is inaccurate $\Rightarrow$ tune $\boldsymbol{\theta}$ for **closed-loop performance**
3. **The obstacle**: this needs $\boldsymbol{\kappa} = \mathrm{d}\boldsymbol{\nu}^\star / \mathrm{d}\boldsymbol{\theta}$, the sensitivity of the trained policy
4. **LR re-weights** paths (a surrogate loss, black-box simulator), **PD moves** them (backpropagation through the simulator); both are a forward and a backward pass, neither always has lower variance
5. **RL policy training can be differentiated**: under smoothness, convergence and regularity assumptions, the trained policy is a stationary point and the IFT gives a meaningful gradient, $\boldsymbol{\kappa} = -(\nabla_{\boldsymbol{\nu}} \mathbf{F})^{-1} \nabla_{\boldsymbol{\theta}} \mathbf{F}$, from the end point of training alone

Take-aways

1. **Models for decisions**: a policy optimal for an approximate model $\rho_{\boldsymbol{\theta}}$ is in general not optimal in reality, and standard model fitting does not fix this
2. **MDP equivalence**: some (non-unique) models and costs give the real optimal policy even if $\rho_{\boldsymbol{\theta}}$ is inaccurate $\Rightarrow$ tune $\boldsymbol{\theta}$ for **closed-loop performance**
3. **The obstacle**: this needs $\boldsymbol{\kappa} = \mathrm{d}\boldsymbol{\nu}^\star / \mathrm{d}\boldsymbol{\theta}$, the sensitivity of the trained policy
4. **LR re-weights** paths (a surrogate loss, black-box simulator), **PD moves** them (backpropagation through the simulator); both are a forward and a backward pass, neither always has lower variance
5. **RL policy training can be differentiated**: under smoothness, convergence and regularity assumptions, the trained policy is a stationary point and the IFT gives a meaningful gradient, $\boldsymbol{\kappa} = -(\nabla_{\boldsymbol{\nu}} \mathbf{F})^{-1} \nabla_{\boldsymbol{\theta}} \mathbf{F}$, from the end point of training alone
6. **Outer gradient** $=$ direct $+ \nabla_{\boldsymbol{\nu}} J^{\mathrm{out}} \boldsymbol{\kappa}$: tune a simulator for reality (sim-to-real) or a system with its policy (co-design)

Take-aways

1. **Models for decisions**: a policy optimal for an approximate model $\rho_{\boldsymbol{\theta}}$ is in general not optimal in reality, and standard model fitting does not fix this
2. **MDP equivalence**: some (non-unique) models and costs give the real optimal policy even if $\rho_{\boldsymbol{\theta}}$ is inaccurate $\Rightarrow$ tune $\boldsymbol{\theta}$ for **closed-loop performance**
3. **The obstacle**: this needs $\boldsymbol{\kappa} = \mathrm{d}\boldsymbol{\nu}^\star / \mathrm{d}\boldsymbol{\theta}$, the sensitivity of the trained policy
4. **LR re-weights** paths (a surrogate loss, black-box simulator), **PD moves** them (backpropagation through the simulator); both are a forward and a backward pass, neither always has lower variance
5. **RL policy training can be differentiated**: under smoothness, convergence and regularity assumptions, the trained policy is a stationary point and the IFT gives a meaningful gradient, $\boldsymbol{\kappa} = -(\nabla_{\boldsymbol{\nu}} \mathbf{F})^{-1} \nabla_{\boldsymbol{\theta}} \mathbf{F}$, from the end point of training alone
6. **Outer gradient** $=$ direct $+ \nabla_{\boldsymbol{\nu}} J^{\mathrm{out}} \boldsymbol{\kappa}$: tune a simulator for reality (sim-to-real) or a system with its policy (co-design)

Proof of concept established on sim-to-real and co-design: a viable pathway towards **decision-oriented learning of MDPs**.

Take-aways

1. **Models for decisions**: a policy optimal for an approximate model $\rho_{\boldsymbol{\theta}}$ is in general not optimal in reality, and standard model fitting does not fix this
2. **MDP equivalence**: some (non-unique) models and costs give the real optimal policy even if $\rho_{\boldsymbol{\theta}}$ is inaccurate $\Rightarrow$ tune $\boldsymbol{\theta}$ for **closed-loop performance**
3. **The obstacle**: this needs $\boldsymbol{\kappa} = \mathrm{d}\boldsymbol{\nu}^\star / \mathrm{d}\boldsymbol{\theta}$, the sensitivity of the trained policy
4. **LR re-weights** paths (a surrogate loss, black-box simulator), **PD moves** them (backpropagation through the simulator); both are a forward and a backward pass, neither always has lower variance
5. **RL policy training can be differentiated**: under smoothness, convergence and regularity assumptions, the trained policy is a stationary point and the IFT gives a meaningful gradient, $\boldsymbol{\kappa} = -(\nabla_{\boldsymbol{\nu}} \mathbf{F})^{-1} \nabla_{\boldsymbol{\theta}} \mathbf{F}$, from the end point of training alone
6. **Outer gradient** $=$ direct $+ \nabla_{\boldsymbol{\nu}} J^{\mathrm{out}} \boldsymbol{\kappa}$: tune a simulator for reality (sim-to-real) or a system with its policy (co-design)

Proof of concept established on sim-to-real and co-design: a viable pathway towards **decision-oriented learning of MDPs**.

Open questions: scaling the critic sensitivities, conditioning of $\nabla_{\boldsymbol{\nu}} \mathbf{F}$ for deep networks, offline real-world updates, latent-space model learning under partial observability

Thank you!

Questions and discussion

Papers

- *Predicting What Matters: Training AI Models for Better Decisions*, IEEE TNNLS, 2025
  Akhil S. Anand, Shambhuraj Sawant, Dirk Peter Reinhardt, Sebastien Gros
- *Bi-level RL pathway for sim-to-real optimality*, RLC 2026
  Akhil S. Anand, Shambhuraj Sawant, Paavo Parmas, Jasper Hoffmann, Dirk Peter Reinhardt, Sebastien Gros
- *Gradient-based co-design of RL policies with differentiable simulators*, TMLR 2026
  Akhil S. Anand, Xun Shen, Marc Peter Deisenroth, Sebastien Gros

Backup slides

details of the policy sensitivities $\nabla_{\nu}\mathbf{F}$, $\nabla_{\theta}\mathbf{F}$ and of the critic sensitivities

Backup: computing $\nabla_{\boldsymbol{\nu}}\mathbf{F}$ and $\nabla_{\boldsymbol{\theta}}\mathbf{F}$ for **LR** policies (PPO, A2C)

Recall (slide 19): $\mathbf{F} = \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}\big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k\big]$, with $\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$, $Q_k = Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)$ (PPO: $Q_k \to$ advantage)

$(\boldsymbol{\nu}, \boldsymbol{\theta})$ act on $\mathbf{F}$ in **three** ways:

1. the policy term $\nabla_{\boldsymbol{\nu}} \log \pi_k$ itself — **policy curvature**
2. the cost-to-go Q_k — **critic sensitivity**
3. which states and actions are sampled (re-weighting, slide 18) — **sampling**

Differentiate once more (Anand et al., 2026, Theorem 1)

$$\nabla_{\boldsymbol{\nu}}\mathbf{F} = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla^2_{\boldsymbol{\nu}\boldsymbol{\nu}} \log \pi_k \, Q_k}_{\text{policy curvature}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, (\nabla_{\boldsymbol{\nu}} Q_k)^\top}_{\text{critic sensitivity}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \, (W_k^{\boldsymbol{\nu}})^\top}_{\text{sampling}} \Big)\Big]$$

$$\nabla_{\boldsymbol{\theta}}\mathbf{F} = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, (\nabla_{\boldsymbol{\theta}} Q_k)^\top}_{\text{critic sensitivity}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \, (W_k^{\boldsymbol{\theta}})^\top}_{\text{sampling}} \Big)\Big]$$

history score: $W_k^{\boldsymbol{\nu}} = \sum_{i \le k} \nabla_{\boldsymbol{\nu}} \log \pi_i$, $\quad W_k^{\boldsymbol{\theta}} = \sum_{i<k} \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{i+1}|\mathbf{s}_i, \mathbf{a}_i)$

π_k does not depend on $\boldsymbol{\theta}$, so $\nabla_{\boldsymbol{\theta}}\mathbf{F}$ has no curvature term; critic sensitivities ∇Q_k: see the critic backup slides

A. S. Anand et al., *Bi-Level Reinforcement Learning Pathway for Sim-to-Real Optimality*, RLC 2026.

Backup: $\nabla_{\boldsymbol{\nu}}\mathbf{F}$ and $\nabla_{\boldsymbol{\theta}}\mathbf{F}$ for **LR** policies in code

Goal: the IFT only needs products $(\nabla_{\boldsymbol{\nu}}\mathbf{F})^\top v$, $(\nabla_{\boldsymbol{\theta}}\mathbf{F})^\top w$ (slide 27), never the matrices

1. **Forward**: the same rollouts as PPO; record $\log \pi_k$ and $\log \rho_{\boldsymbol{\theta}}$ of every step as functions of $(\boldsymbol{\nu}, \boldsymbol{\theta})$; the critic gives Q_k; $\nabla_{\boldsymbol{\nu}} Q_k$, $\nabla_{\boldsymbol{\theta}} Q_k$ from the critic backup slides
2. **Pitfall**: `grad` of the usual PPO gradient gives **only** the policy curvature: samples and critic are constants in the AD graph, so it even returns $\nabla_{\boldsymbol{\theta}}\mathbf{F} = 0$
3. **Trick**: with $\mathrm{stop}(x)$ the value of x seen as a constant by AD (`detach`, `stop_gradient`), $x - \mathrm{stop}(x)$ is 0 in value but has derivative 1 $\Rightarrow$ add the missing derivatives without changing any value

   sampling: multiply step k by $\exp\big(\ell_k - \mathrm{stop}(\ell_k)\big)$,
   $\ell_k = \log p(\text{history up to } k)$: value 1, derivative W_k
   critic: use $Q_k + (\nabla_{\boldsymbol{\nu}} Q_k)^\top \big(\boldsymbol{\nu} - \mathrm{stop}(\boldsymbol{\nu})\big) + (\nabla_{\boldsymbol{\theta}} Q_k)^\top \big(\boldsymbol{\theta} - \mathrm{stop}(\boldsymbol{\theta})\big)$:
   value Q_k, derivative ∇Q_k
4. **Backward**: with this $\mathbf{F}$, $(\nabla_{\boldsymbol{\nu}}\mathbf{F})^\top v = \nabla_{\boldsymbol{\nu}} \langle \mathbf{F}, v \rangle$ and $(\nabla_{\boldsymbol{\theta}}\mathbf{F})^\top w = \nabla_{\boldsymbol{\theta}} \langle \mathbf{F}, w \rangle$: one backward pass each

Same samples and same value of $\mathbf{F}$, but AD now sees all three effects

J. Foerster et al., *DiCE: The Infinitely Differentiable Monte Carlo Estimator*, ICML 2018 (the $\exp(\ell - \mathrm{stop}(\ell))$ trick).

Backup: differentiating the critic, $\nabla_{\boldsymbol{\nu}} Q_k$ and $\nabla_{\boldsymbol{\theta}} Q_k$

Why not AD of the critic network? The network $Q_w(\mathbf{s}, \mathbf{a})$ has no input $\boldsymbol{\nu}$ or $\boldsymbol{\theta}$: AD gives only $\nabla_{\mathbf{s}} Q$, $\nabla_{\mathbf{a}} Q$, $\nabla_w Q$. Its weights w come out of training, so we differentiate what Q **satisfies**, the Bellman equation:

$$Q_k = L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k) + \gamma\, \mathbb{E}\big[Q_{k+1}\big], \qquad \mathbf{s}_{k+1} \sim \rho_{\boldsymbol{\theta}}(\cdot|\mathbf{s}_k, \mathbf{a}_k), \quad \mathbf{a}_{k+1} \sim \pi_{\boldsymbol{\nu}}(\cdot|\mathbf{s}_{k+1})$$

Differentiate it like the **LR** gradient: product rule, and a sampling distribution gives a score

$$\nabla_{\boldsymbol{\theta}} Q_k = \mathbb{E}\Big[\underbrace{\nabla_{\boldsymbol{\theta}} L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k) + \gamma\, \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)\, Q_{k+1}}_{c_k^{\boldsymbol{\theta}}} + \gamma\, \nabla_{\boldsymbol{\theta}} Q_{k+1}\Big]$$

$$\nabla_{\boldsymbol{\nu}} Q_k = \mathbb{E}\Big[\underbrace{\gamma\, \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_{k+1}|\mathbf{s}_{k+1})\, Q_{k+1}}_{c_k^{\boldsymbol{\nu}}} + \gamma\, \nabla_{\boldsymbol{\nu}} Q_{k+1}\Big]$$

$\boldsymbol{\theta}$: in the cost and in the next state (score of $\rho_{\boldsymbol{\theta}}$); $\boldsymbol{\nu}$: only in the next action (score of $\pi_{\boldsymbol{\nu}}$)

∇Q_k satisfies **its own Bellman equation**, with the **sensitivity cost** c_k in place of $L_{\boldsymbol{\theta}}$:

$$\nabla Q_k = c_k + \gamma\, \mathbb{E}\big[\nabla Q_{k+1}\big] = \mathbb{E}\big[\textstyle\sum_{t \geq k} \gamma^{t-k} c_t\big]$$

A. S. Anand et al., *Bi-Level Reinforcement Learning Pathway for Sim-to-Real Optimality*, RLC 2026 (Lemma 2 and appendix).

Backup: estimating the critic sensitivity ∇Q_k in practice

∇Q_k is the cost-to-go of the sensitivity cost c_k, so it is **estimated like a critic**, on the same in-sim rollouts (Q_{k+1} inside c_k comes from the usual critic):

- **Monte Carlo**: along each sampled trajectory, backwards like the return:

$$G_N = 0, \qquad G_k = c_k + \gamma\, G_{k+1}, \qquad \nabla Q_k \approx G_k$$

- **TD**: train a network $G(\mathbf{s}, \mathbf{a}) \approx \nabla Q$ (one output per parameter) on the target $c_k + \gamma\, G(\mathbf{s}_{k+1}, \mathbf{a}_{k+1})$, next to the usual critic; fine for a few $\boldsymbol{\theta}$, costly for a large $\boldsymbol{\nu}$
- **Variance**: c_k contains scores $\Rightarrow$ subtract baselines, as A2C/PPO do for Q
- **Timescales**: the critic and ∇Q are learned fastest, then the policy, then $\boldsymbol{\theta}$ (convergence of the bi-level scheme)

PD (SVG, SHAC) with a terminal critic $\gamma^H V(\mathbf{s}_H)$

AD differentiates V **through the state** $\mathbf{s}_H$ (as part of the rollout). The critic's own dependence on $\boldsymbol{\theta}$ is usually dropped: the critic is frozen, which gives a **semi-gradient** whose error is scaled by γ^H

A. S. Anand et al., *Bi-Level Reinforcement Learning Pathway for Sim-to-Real Optimality*, RLC 2026; *Gradient-Based Co-Design of RL Policies with Differentiable Simulators*, TMLR 2026 (**PD**).

Backup: computing $\nabla_{\nu}\mathbf{F}$ and $\nabla_{\theta}\mathbf{F}$ for **PD** policies (SVG)

Recall (slide 20): with the noise ω fixed, $\mathbf{F}$ is itself the output of a backward pass through the rollout cost: $\mathbf{F}(\nu,\theta) = \mathbb{E}_{\omega}\left[\nabla_{\nu} J_{\omega}(\nu,\theta)\right]$

In code: differentiate the backward pass once more (`grad` of `grad`)

gradient $\mathbf{F}(\nu,\theta)$ = average over noise draws ω_i of $\texttt{grad}_{\nu}\, J_{\omega_i}(\nu,\theta)$

in ν $(\nabla_{\nu}\mathbf{F})^{\top} v = \nabla_{\nu}\langle \mathbf{F}(\nu,\theta), v\rangle$ (a Hessian–vector product)

in θ $(\nabla_{\theta}\mathbf{F})^{\top} w = \nabla_{\theta}\langle \mathbf{F}(\nu,\theta), w\rangle$

one more backward pass each; no pitfall, as the noise does not depend on (ν,θ)

What AD computes for you (Anand et al., 2026): per step, $\mathbf{F}$ adds (slope of the cost) $\times$ (how the path moves with ν) $= \nabla_z L_{\theta}(z_k)\frac{\mathrm{d}z_k}{\mathrm{d}\nu}$, $z_k = (\mathbf{s}_k, \mathbf{a}_k)$. Product rule:

$$\frac{\mathrm{d}}{\mathrm{d}\theta}\left(\nabla_z L_{\theta}(z_k)\,\frac{\mathrm{d}z_k}{\mathrm{d}\nu}\right) = \underbrace{\left(\frac{\mathrm{d}}{\mathrm{d}\theta}\nabla_z L_{\theta}(z_k)\right)}_{\theta \text{ in the cost, the path moves}}\frac{\mathrm{d}z_k}{\mathrm{d}\nu} + \nabla_z L_{\theta}(z_k)\,\underbrace{\frac{\mathrm{d}^2 z_k}{\mathrm{d}\nu\,\mathrm{d}\theta}}_{\theta \text{ bends the path's response}}$$

second derivatives of L_{θ}, f_{θ}, g_{ν} along the path, all from AD; $\nabla_{\nu}\mathbf{F}$: same with $\theta \to \nu$

In practice $\nabla_{\nu}\mathbf{F}$ may be ill-conditioned or indefinite: damp it $(+\varepsilon I)$ or truncate its eigendecomposition

A. S. Anand et al., *Gradient-Based Co-Design of Reinforcement Learning Policies with Differentiable Simulators*, TMLR 2026